

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG

ĐỒ ÁN TỐT NGHIỆP

**THIẾT KẾ HỆ THỐNG GIÁM SÁT, THU THẬP
DỮ LIỆU VÀ CHẨN ĐOÁN LỖI CỦA XE Ô TÔ
THÔNG QUA CỔNG OBD-II**

NGÀNH HỆ THỐNG NHÚNG VÀ IOT

Sinh viên: **VÕ XUÂN LỘC**

MSSV: 22139040

VÕ QUỐC TRƯỜNG

MSSV: 22139074

Hướng dẫn: **ThS NGUYỄN VĂN PHÚC**

TP. HỒ CHÍ MINH – 06/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT THÀNH PHỐ HỒ CHÍ MINH
KHOA ĐIỆN ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG

ĐỒ ÁN TỐT NGHIỆP

**THIẾT KẾ HỆ THỐNG GIÁM SÁT, THU THẬP
DỮ LIỆU VÀ CHẨN ĐOÁN LỖI CỦA XE Ô TÔ
THÔNG QUA CÔNG OBD-II**

NGÀNH HỆ THỐNG NHÚNG VÀ IOT

Sinh viên: **VÕ XUÂN LỘC**

MSSV: 22139040

VÕ QUỐC TRƯỜNG

MSSV: 22139074

Hướng dẫn: **ThS NGUYỄN VĂN PHÚC**

TP. HỒ CHÍ MINH – 06/2026

THÔNG TIN KHÓA LUẬN TỐT NGHIỆP

1. Thông tin sinh viên

Họ và tên sinh viên: Võ Xuân Lộc

MSSV: 22139040

Email: 22139040@student.hcmute.edu.vn

Điện thoại: 0979400237

Họ và tên sinh viên: Võ Quốc Trường

MSSV: 22139074

Email: 22139074@student.hcmute.edu.vn

Điện thoại: 0984317621

2. Thông tin đề tài

- Tên đề tài: THIẾT KẾ HỆ THỐNG GIÁM SÁT, THU THẬP DỮ LIỆU VÀ CHẨN ĐOÁN LỖI CỦA XE Ô TÔ THÔNG QUA CỔNG OBD-II.
- Đơn vị quản lý: Bộ môn Kỹ Thuật Máy Tính - Viễn Thông, Khoa Điện Điện Tử, Trường Đại Học Công Nghệ Kỹ Thuật Thành phố Hồ Chí Minh.
- Thời gian thực hiện: Từ ngày 11/03/2026 đến ngày 26/06/2026.
- Thời gian bảo vệ trước hội đồng: Ngày 03/07/2026.

TP.HCM, ngày ... tháng ... năm 2026

Sinh viên thực hiện đồ án

(Ký và ghi rõ họ tên)

Giảng viên hướng dẫn xác nhận quyền báo cáo đã được chỉnh sửa theo đề nghị được ghi trong biên bản của Hội đồng đánh giá Khóa luận tốt nghiệp.

.....

TP.HCM, ngày ... tháng ... năm 2026

Xác nhận của Bộ Môn

Giảng viên hướng dẫn

(Ký và ghi rõ họ tên và học hàm - học vị)

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP (Dành cho giảng viên hướng dẫn)

Đề tài: Thiết kế hệ thống giám sát, thu thập dữ liệu và chẩn đoán lỗi của xe ô tô thông qua cổng OBD-II

Sinh viên: Võ Xuân Lộc
Võ Quốc Trường

MSSV: 22139040
MSSV: 22139074

Hướng dẫn: Ths. Nguyễn Văn Phúc

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:
Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.
- Đặt vấn đề rõ ràng, mục tiêu cụ thể
2. Phương pháp thực hiện/ phân tích/ thiết kế:
Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.
- Phương pháp hợp lý
3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:
Phù hợp với mục tiêu đề tài; phân tích và đánh giá / kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.
- Kết quả phù hợp với mục tiêu
4. Kết luận và đề xuất:
Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.
- Đề xuất mang tính cải tiến
5. Hình thức trình bày và bố cục báo cáo:
Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.
- Bố cục rõ ràng, đúng mẫu
6. Kỹ năng chuyên nghiệp và tính sáng tạo:
Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.
- Kỹ năng làm việc nhóm tốt
7. Tài liệu trích dẫn
Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.
- Có thực hiện trích dẫn tài liệu tham khảo
8. Đánh giá về sự trùng lặp của đề tài
Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.
- Tỉ lệ đạo văn đạt yêu cầu
9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

- Tinh thần và thái độ học tập nghiêm túc, khả năng nghiên cứu tốt

Đề nghị của giảng viên hướng dẫn

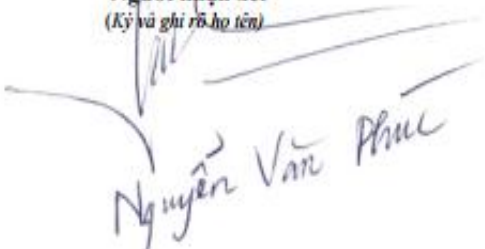
Ghi rõ: "Báo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp"

Báo cáo đạt yêu cầu, nhóm sinh viên được phép bảo vệ khóa luận tốt nghiệp

Tp. HCM, ngày 26 tháng 06 năm 2026

Người nhận xét

(Ký và ghi rõ họ tên)


Nguyễn Văn Phúc

LỜI CẢM ƠN

Lời đầu tiên, nhóm thực hiện xin chân thành cảm ơn Quý Thầy Cô Khoa Điện - Điện tử, Trường Đại học Công Nghệ Kỹ Thuật Thành phố Hồ Chí Minh đã tận tình chỉ dạy kiến thức, kinh nghiệm và tạo điều kiện cho chúng em trong suốt quá trình học tập.

Đặc biệt, nhóm thực hiện xin chân thành cảm ơn sâu sắc nhất đến Thầy – Ths Nguyễn Văn Phúc, người đã hướng dẫn, định hướng, chỉ dẫn và giúp đỡ nhóm giải quyết các vấn đề kỹ thuật để hoàn thành tốt đề tài nghiên cứu này. Thầy không chỉ giúp nhóm chỉnh sửa những ý chưa đúng, mà còn hướng dẫn cách trình bày bố cục hợp lý, rõ ràng, giúp nhóm nâng cao tư duy và kỹ năng làm việc nhóm cũng như kỹ năng trình bày một đề tài báo cáo.

Nhóm thực hiện kính chúc Quý Thầy Cô đang công tác tại trường Đại Học Công Nghệ Kỹ thuật Thành phố Hồ Chí Minh lời chúc sức khỏe và gặt hái được nhiều thành công. Cuối cùng, nhóm thực hiện xin gửi lời cảm ơn tới các bậc phụ huynh, gia đình đã tạo điều kiện nền tảng tốt nhất để có thể hoàn thành đề tài.

Đề tài nghiên cứu vẫn còn nhiều khuyết điểm và chắc chắn không tránh khỏi những thiếu sót ngoài ý muốn. Nhóm thực hiện rất mong nhận được những ý kiến đóng góp trân quý của Quý Thầy Cô trong Hội đồng để đề tài được hoàn thiện và có tính thực tiễn cao hơn.

Nhóm thực hiện xin chân thành cảm ơn.

LỜI CAM ĐOAN

Nhóm thực hiện cam kết rằng đồ án tốt nghiệp này là kết quả nghiên cứu và thực hiện riêng của nhóm dưới sự hướng dẫn của giảng viên hướng dẫn. Các tài liệu tham khảo, số liệu, hình ảnh và kết quả nghiên cứu của người khác đều được trích dẫn đầy đủ theo đúng quy định. Nhóm thực hiện xin hoàn toàn chịu trách nhiệm trước Nhà trường về tính trung thực và tính hợp pháp của nội dung đồ án.

Sinh viên thực hiện đồ án
(Ký và ghi rõ họ tên)

TÓM TẮT

Đề tài “Thiết kế hệ thống giám sát, thu thập dữ liệu và chẩn đoán lỗi của xe ô tô thông qua cổng OBD-II” hướng đến mục tiêu thiết kế và chế tạo một thiết bị cầm tay hoạt động ổn định có vỏ bảo vệ để trích xuất dữ liệu từ các dòng xe có hỗ trợ cổng OBD-II. Thông qua giao thức tầng mạng CAN TP theo tiêu chuẩn ISO 15765-2, hệ thống có thể đọc và giải mã chính xác các thông số động học thời gian thực như: Tốc độ xe, vòng tua, mức nhiên liệu, nhiệt độ nước làm mát, tải động cơ, vị trí bướm ga, danh sách mã lỗi hệ thống và mã định danh phương tiện.

Mục tiêu chính của đề tài hướng đến bao gồm:

- Đọc các thông tin của xe thông qua cổng OBD-II bằng các Mode 01, 03, 07, 09, 0A.
- Đẩy dữ liệu trích xuất được lên cơ sở dữ liệu Firebase.
- Xây dựng ứng dụng di động Android kết nối đồng bộ dữ liệu với Firebase nhằm trực quan hoá dữ liệu đọc được từ xe ô tô, chủ động gửi yêu cầu đọc dữ liệu từ xa và có cảnh báo tự động khi các thông số vượt ngưỡng.
- Kiểm thử trực tiếp trên nhiều dòng xe thật. Sử dụng mạch phân tích xung Logic Analyzer để bắt luồng log dữ liệu thô, phục vụ quá trình phân tích truyền nhận dữ liệu CAN và đối chiếu trực tiếp với các tiêu chuẩn ISO 15031-5 và SAE J1979 để đảm bảo tính đúng đắn của luồng truyền nhận trên mạng CAN.

MỤC LỤC

DANH MỤC HÌNH	x
DANH MỤC BẢNG	xiii
CÁC TỪ VIẾT TẮT	xiv
CHƯƠNG 1	1
GIỚI THIỆU	1
1. Tình hình nghiên cứu	1
2. Lý do chọn đề tài	1
3. Mục tiêu của đề tài	2
4. Giới hạn của đề tài	3
5. Bố cục của đề tài	3
CHƯƠNG 2	5
CƠ SỞ LÝ THUYẾT	5
2.1 Giao thức truyền thông CAN	5
2.2.1 Tổng quan về mạng CAN	5
2.2.2 Kiến trúc mạng CAN	6
2.2.3 Cấu trúc khung tin CAN	7
2.2.3 Nguyên lý hoạt động	9
2.2 Giao thức CAN TP (ISO 15765-2)	12
2.2.1 Tổng quan về CAN TP	12
2.2.2 Các loại frame trong CAN	12
2.2.3 Nguyên lý hoạt động của CAN TP	12
2.2.3.1 Truyền dữ liệu khung đơn SF	12
2.2.3.2 Truyền dữ liệu đa khung	13
2.2.3.3 Cơ chế điều khiển luồng	14
2.3 Tiêu chuẩn chẩn đoán OBD-II	14
2.3.1 Tổng quan về OBD-II	14
2.3.2 Cổng kết nối OBD-II theo tiêu chuẩn ISO 15031-3	14
2.3.3 Các chế độ chẩn đoán	16
2.3.4 Cấu trúc thông điệp chẩn đoán OBD-II trên mạng CAN	17
2.3.5 Quy trình đóng gói thông điệp chẩn đoán chi tiết theo từng Mode ...	18

2.3.5.1 Quy trình đóng gói và truyền nhận Mode 01	18
2.3.5.2 Quy trình đóng gói và truyền nhận Mode 03, Mode 07 và Mode 0A	20
2.3.5.3 Quy trình đóng gói và truyền nhận Mode 09	26
2.4 Chuẩn giao tiếp SPI	28
2.4.1 Tổng quan về SPI	28
2.4.2 Cấu trúc tín hiệu trong	28
2.4.3 Nguyên lý hoạt động của SPI	29
2.5 Nền tảng phát triển ứng dụng Flutter	29
2.6 Hệ điều hành thời gian thực (RTOS)	30
2.6.1 Tổng quan về RTOS	30
2.6.2 Kiến trúc cơ bản của RTOS	30
2.6.3 Nguyên lý hoạt động của	31
2.6.4 Các cơ chế chính trong RTOS	31
CHƯƠNG 3	32
THIẾT KẾ VÀ THI CÔNG HỆ THỐNG	32
3.1 Đặc tả hệ thống	32
3.1.1 Yêu cầu chức năng	32
3.1.2 Yêu cầu phi chức năng	32
3.1.3 Yêu cầu phần cứng	33
3.1.4 Yêu cầu phần mềm	33
3.2 Sơ đồ khối và chức năng của các khối	34
3.3 Phân tích và thiết kế các khối phần cứng	35
3.3.1 Khối xử lý trung tâm	35
3.3.2 Khối truyền thông	37
3.3.3 Khối hiển thị	40
3.3.4 Khối nút nhấn	43
3.3.5 Khối nguồn	44
3.4 Sơ đồ nguyên lý toàn hệ thống	49
3.5 Thiết kế phần mềm và ứng dụng di động	50
3.5.1 Thiết kế cơ sở dữ liệu Firebase	50

3.5.2 Thiết kế giao diện ứng di động	51
3.6 Lưu đồ giải thuật	51
3.6.1 Lưu đồ giải thuật hệ thống	51
3.6.2 Lưu đồ giải thuật từng chức năng	53
3.6.2.1 Lưu đồ giải thuật Mode 01	53
3.6.2.2 Lưu đồ giải thuật Mode 03, 07, 0A	55
3.6.2.3 Lưu đồ giải thuật Mode 09	57
3.6.2.4 Lưu đồ giải thuật tiến trình đồng bộ dữ liệu lên Firebase	58
3.6.2.5 Lưu đồ giải thuật UI	59
CHƯƠNG 4	60
KẾT QUẢ THỰC HIỆN	60
4.1 Kết quả thực hiện trên phần cứng	60
4.2 Kết quả thực hiện trên phần mềm	62
4.3 Kết quả thực hiện quy trình kiểm thử	66
4.3.1 Thử nghiệm trên bộ mô phỏng giả lập	66
4.3.2 Thử nghiệm thực tế trên phương tiện và phân tích kết quả	68
4.3.2.1 Kết quả kiểm thử Mode 01	69
4.3.2.2 Kết quả kiểm thử Mode 03, Mode 07	78
4.3.2.3 Kết quả kiểm thử Mode 0A	80
4.3.2.4 Kết quả kiểm thử Mode 09	82
4.4 Đánh giá và nhận xét hệ thống	87
CHƯƠNG 5	90
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	90
TÀI LIỆU THAM KHẢO	92

DANH MỤC HÌNH

Hình 2.1. Tổng quan mạng CAN trong ô tô	5
Hình 2.2. Kiến trúc mạng CAN.....	6
Hình 2.3. Tốc độ truyền và chiều dài của CAN tiêu chuẩn	7
Hình 2.4. Hai dây xoắn đôi CAN_H và CAN_L	7
Hình 2.5. Cấu trúc khung dữ liệu chuẩn (Standard Data frame)	8
Hình 2.6. Cấu trúc khung dữ liệu mở rộng (Extended Data frame)	8
Hình 2.7. Ví dụ về cơ chế phân xử trên mạng CAN	10
Hình 2.8. Quan hệ giữa trạng thái logic (dominant/recessive) và điện áp trên 2 dây CAN trong mạng High-speed CAN (ISO 11898-2)	11
Hình 2.9. Truyền dữ liệu sử dụng SF trong CAN TP	13
Hình 2.10. Cơ chế truyền nhiều frame trong CAN TP	13
Hình 2.11. Đầu nối OBD-II (Female) trên ô tô	15
Hình 2.12. Male OBD-II connector	15
Hình 2.13. Cấu trúc thông điệp chẩn đoán OBD-II trên CAN	17
Hình 2.14. Ví dụ cấu trúc bit mã lỗi DTC P0143	25
Hình 2.15. Giao tiếp SPI giữa Master và nhiều Slave	29
Hình 3.1. Sơ đồ khối hệ thống	34
Hình 3.2. Module ESP32 devkit v1	35
Hình 3.3. Sơ đồ chân module ESP32 devkit v1	36
Hình 3.4. Sơ đồ nguyên lý khối điều khiển trung tâm	36
Hình 3.5. Bảng trạng thái điều khiển chân EN, IO0 và IO2 từ tín hiệu DTR, RTS	37
Hình 3.6. Module MCP2515 CAN Bus	38
Hình 3.7. Sơ đồ nguyên lý khối truyền thông	39
Hình 3.8. Màn hình cảm ứng TFT 2.8 inch SPI ILI934	41
Hình 3.9. Mặt sau và sơ đồ ký hiệu chân cắm thực tế của module màn hình TFT 2.8" SPI	41
Hình 3.10. Sơ đồ nguyên lý khối hiển thị	42
Hình 3.11. Nút nhấn 6x6mm cao 6mm 4 chân xuyên lỗ	43
Hình 3.12. Sơ đồ nguyên lý khối nút nhấn	44
Hình 3.13. Sơ đồ nguyên lý khối nút nhấn	45
Hình 3.14. Module sạc pin TP4056	45
Hình 3.15. Module tăng áp MT3608	46
Hình 3.16. IC ổn áp LM 1117 3.3V	47
Hình 3.17. Sơ đồ nguyên lý khối nguồn	48
Hình 3.18. Sơ đồ nguyên lý toàn hệ thống	49
Hình 3.19. Cấu trúc cây dữ liệu thực tế trên Firebase	50
Hình 3.20. Tổng quan về giao diện ứng dụng	51

Hình 3.21. Lưu đồ giải thuật toàn hệ thống	52
Hình 3.22. Lưu đồ giải thuật Mode 01	54
Hình 3.23. Lưu đồ giải thuật Mode 03, 7, 0A	55
Hình 3.24. Lưu đồ giải thuật Mode 09	57
Hình 3.25. Lưu đồ giải thuật tiến trình đồng bộ dữ liệu lên Firebase	58
Hình 3.26. Lưu đồ giải thuật UI	59
Hình 4.1. Bản vẽ thiết kế đường mạch in PCB	60
Hình 4.2. a) Mô hình 3D mặt trước và b) mô hình 3D mặt sau của thiết bị	61
Hình 4.3. a) Mô hình mặt trước và b) mặt sau thực tế của thiết bị	61
Hình 4.4. Thiết bị hoàn thiện được đóng vỏ bảo vệ	62
Hình 4.5. a) Giao diện đăng nhập và b) Giao diện đăng ký tài khoản	63
Hình 4.6. a) Giao diện các Mode 03, b) Giao diện Mode 07 và c) Giao diện Mode 0A	64
Hình 4.7. Giao diện Mode 09	65
Hình 4.8. a) Giao diện thông tin của admin b) Giao diện thông tin của User và c) Giao diện quản lý tài khoản người dùng	65
Hình 4.9. Hệ thống mô phỏng giả lập	67
Hình 4.10. Thử nghiệm thực tế trên xe ô tô Mazda 3 2022	68
Hình 4.11. Kết nối vật lý giữa thiết bị chẩn đoán với cổng OBD-II của xe Mazda 3	68
Hình 4.12. Ảnh chụp màn hình LCD của thiết bị kiểm thử thu thập dữ liệu Mode 01	69
Hình 4.13. Ảnh chụp cụm đồng hồ Taplo trên xe ô tô tại thời điểm thực nghiệm đối chiếu	69
Hình 4.14. Giải đồ xung của lệnh yêu cầu dữ liệu tốc độ vòng tua của Mode 01	70
Hình 4.15. Giải đồ xung phản hồi tốc độ vòng tua máy từ ECU	70
Hình 4.16. Giải đồ xung của lệnh yêu cầu dữ liệu tốc độ xe Mode 01	71
Hình 4.17. Giải đồ xung phản hồi tốc độ xe từ ECU	72
Hình 4.18. Giải đồ xung của lệnh yêu cầu nhiệt độ nước làm mát động cơ	73
Hình 4.19. Giải đồ xung phản hồi nhiệt độ nước từ ECU	73
Hình 4.20. Giải đồ xung của lệnh yêu cầu dữ liệu vị trí bướm ga tuyệt đối	74
Hình 4.21. Giải đồ xung phản hồi vị trí bướm ga tuyệt đối từ ECU	75
Hình 4.22. Giải đồ xung của lệnh yêu cầu dữ liệu mức nhiên liệu	76
Hình 4.23. Giải đồ xung phản hồi mức nhiên liệu từ ECU	76
Hình 4.24. Giải đồ xung của lệnh yêu cầu dữ liệu tải trọng động cơ	77
Hình 4.25. Giải đồ xung phản hồi tải trọng động cơ từ ECU	78
Hình 4.26. Ảnh chụp màn hình LCD của thiết bị chẩn đoán đọc mã lỗi	79
Hình 4.27. Giải đồ xung của lệnh yêu cầu mã lỗi đến ECU động cơ Mode 03, 07	79

Hình 4.28. Giản đồ xung của lệnh yêu cầu mã lỗi đến ECU hộp số Mode 03, 07	79
Hình 4.29. Giản đồ xung phản hồi từ ECU động cơ Mode 03, 07	80
Hình 4.30. Giản đồ xung phản hồi từ ECU hộp số Mode 03, 07	80
Hình 4.31. Giản đồ xung của lệnh yêu cầu mã lỗi đến ECU động cơ Mode 0A ..	81
Hình 4.32. Giản đồ xung của lệnh yêu cầu mã lỗi đến ECU hộp số Mode 0A ..	81
Hình 4.33. Giản đồ xung phản hồi của ECU động cơ Mode 0A	81
Hình 4.34. Giản đồ xung phản hồi của ECU hộp số Mode 0A	82
Hình 4.35. Ảnh chụp màn hình LCD của thiết bị chẩn đoán thu thập dữ liệu Mode 09	83
Hình 4.36. Kết quả đối chiếu thông tin mã VIN trên hệ thống tra cứu cơ sở dữ liệu xe	83
Hình 4.37. Giản đồ xung của lệnh yêu cầu thông tin mã VIN	84
Hình 4.38. Giản đồ xung phản hồi khung đầu tiên FF từ ECU	84
Hình 4.39. Giản đồ xung khung điều khiển luồng FC từ thiết bị chẩn đoán gửi đi	85
Hình 4.40. Giản đồ xung phản hồi nối tiếp CF1	85
Hình 4.41. Giản đồ xung phản hồi nối tiếp CF2	86

DANH MỤC BẢNG

Bảng 2. 1. Định danh cấu trúc các byte dữ liệu trong khung yêu cầu Mode 01 .	18
Bảng 2. 2. Định danh cấu trúc các byte dữ liệu trong khung phản hồi Mode 01	19
Bảng 2. 3. Định nghĩa và công thức giải mã các thông số Mode 01 PID của hệ thống.....	20
Bảng 2. 4. Định danh cấu trúc các byte dữ liệu trong khung yêu cầu Mode 03,07 và 0A	20
Bảng 2. 5. Định danh cấu trúc các byte trong khung phản hồi tiêu cực Mode 03,07 và 0A	21
Bảng 2. 6. Danh sách các mã phản hồi tiêu cực NRC	22
Bảng 2. 7. Định danh cấu trúc byte phản hồi khi phương tiện không có mã lỗi	22
Bảng 2. 8. Định danh cấu trúc toàn bộ các byte phản hồi khi có ít hơn 2 mã lỗi	22
Bảng 2. 9. Định danh cấu trúc khung đầu tiên từ ECU Mode 03,07 và 0A	23
Bảng 2. 10. Định danh cấu trúc khung điều khiển luồng FC từ thiết bị kiểm thử Mode 03,07 và 0A	24
Bảng 2. 11. Định danh cấu trúc khung nối tiếp CF từ ECU Mode 03,07 và 0A	24
Bảng 2. 12. Quy luật bóc tách Bitmasking và giải mã mã lỗi DTC P0143	25
Bảng 2. 13. Định danh cấu trúc các byte dữ liệu trong khung yêu cầu Mode 09	26
Bảng 2. 14. Định danh cấu trúc khung đầu tiên FF chứa mã VIN từ ECU	27
Bảng 2. 15. Định danh cấu trúc khung điều khiển luồng FC Mode 09	27
Bảng 2. 16. Định danh cấu trúc khung nối tiếp số CF1 Mode 09	28
Bảng 2. 17. Định danh cấu trúc khung nối tiếp số CF2 Mode 09	28
Bảng 3. 1. Thông số kỹ thuật module ESP32 devkit v1	35
Bảng 3. 2. Thông số kỹ thuật module MCP2515 CAN Bus	38
Bảng 3. 3 Thông số kỹ thuật màn hình cảm ứng TFT SPI ILI9341	41
Bảng 3. 4. Mô tả chức năng các chân cắm của module màn hình	41
Bảng 3. 5. Thông số kỹ thuật nút nhấn	43
Bảng 3. 6 Thông số kỹ thuật module sạc pin TP4056	45
Bảng 3. 7. Thông số kỹ thuật module tăng áp MT3608	46
Bảng 3. 8. Thông số kỹ thuật IC ổn áp LM1117 3.3V	47

CÁC TỪ VIẾT TẮT

OBD-II	On-Board Diagnostics II
CAN	Controller Area Network
ECU	Engine Control Unit
SF	Single Frame
FF	First Frame
CF	Consecutive Frame
FC	Flow Control
PID	Parameter Identification
NRC	Negative Respond Code
SPI	Serial Peripheral Interface
DTC	Diagnostic Trouble Code
MIL	Malfunction Indicator Lamp
SAE	Society of Automotive Engineers
ISO	International Organization for Standardization
ECM	Engine Control Module
TCM	Transmission Control Module
CAN TP	CAN Transport Protocol
CEL	Check Engine Light
RTOS	Real-Time Operation System

CHƯƠNG 1

GIỚI THIỆU

1. Tình hình nghiên cứu

Hiện nay, sự phát triển mạnh mẽ của hệ thống điện tử trên ô tô làm tăng nhu cầu về các nghiên cứu giám sát và chẩn đoán lỗi qua cổng OBD-II. Việc này không chỉ giúp theo dõi tình trạng xe, phát hiện sớm hư hỏng để hỗ trợ bảo trì mà còn có ý nghĩa lớn trong đào tạo, giúp sinh viên nắm vững các giao thức giao tiếp phần mềm ô tô.

Đề tài [1] đã triển khai hệ thống đọc thông số thời gian thực và mã lỗi DTC qua CAN bus, hiển thị trên LabVIEW, đồng thời tự làm ECU mô phỏng để kiểm thử. Tuy nhiên, nghiên cứu này mới dừng lại ở mô phỏng, chưa được đánh giá trên các dòng xe thực tế.

Đề tài [2] cũng nghiên cứu hệ thống chẩn đoán qua CAN bus với chức năng tương tự đề tài [1]. Dù vậy, nghiên cứu chủ yếu dừng ở mức phân tích công nghệ và đề xuất giải pháp, chưa làm rõ quá trình triển khai phần cứng lẫn kiểm thử trên xe thật.

Đề tài [3] đề xuất hệ thống chẩn đoán từ xa dựa trên công nghệ IoT với tính năng rất đầy đủ, phù hợp định hướng phát triển. Nhược điểm lớn nhất là hệ thống sử dụng phần cứng khá cồng kềnh và chi phí cao, chưa tối ưu được kích thước và giá thành.

Từ những thiếu sót của các đề tài trên, nhóm quyết định thực hiện đề tài “Thiết kế hệ thống giám sát, thu thập dữ liệu và chẩn đoán lỗi của xe ô tô thông qua cổng OBD-II” nhằm phát triển một thiết bị phần cứng nhỏ gọn, thử nghiệm thực tế trên nhiều dòng xe thật, đi kèm ứng dụng di động hỗ trợ giám sát thời gian thực và điều khiển đọc dữ liệu từ xa.”

2. Lý do chọn đề tài

Hiện nay, người dùng chủ yếu nhận biết lỗi động cơ qua đèn cảnh báo MIL trên bảng điều khiển. Tuy nhiên, đây chỉ là đèn báo tổng quát, chỉ bật khi thông số vượt ngưỡng mà không cung cấp thông tin chi tiết. Khi đèn sáng, chủ xe không thể biết chính xác vị trí, nguyên nhân hay mức độ nghiêm trọng của hư hỏng, dẫn đến tâm lý hoang mang và bị động trong việc xử lý.

Bên cạnh đó, dù tiêu chuẩn OBD-II bắt buộc trên các dòng xe hiện đại giúp giám sát và giải mã lỗi, nhưng đa số chủ xe vẫn chưa biết đến giao thức này. Thêm vào đó, các máy chẩn đoán chuyên dụng trên thị trường tuy đầy đủ chức năng nhưng có giá thành rất cao, khiến người dùng phổ thông khó tiếp cận.

Xuất phát từ thực tiễn trên, đề tài "Thiết kế hệ thống giám sát, thu thập dữ liệu và chẩn đoán lỗi của xe ô tô thông qua cổng OBD-II" được thực hiện nhằm mang đến một giải pháp hiệu quả với chi phí thấp. Điểm nhấn của đề tài là việc nghiên cứu và chế tạo một thiết bị phần cứng sở hữu các tính năng chẩn đoán cốt lõi với giá thành tối ưu, nhưng vẫn đáp ứng tốt nhu cầu thu thập và giám sát dữ liệu theo thời gian thực. Hệ thống cung cấp trải nghiệm trực quan, linh hoạt thông qua một màn hình hiển thị được sử dụng để chọn các chức năng, cùng với một ứng dụng di động đi kèm giúp người dùng dễ dàng theo dõi chi tiết tình trạng phương tiện. Giải pháp này không chỉ tháo gỡ rào cản về chi phí so với các máy chẩn đoán chuyên nghiệp, mà còn giúp người dùng chủ động nắm bắt chính xác các vấn đề kỹ thuật, từ đó tối ưu hóa quy trình bảo trì và giảm thiểu tối đa chi phí.

3. Mục tiêu của đề tài

Để giải quyết các vấn đề đã đặt ra, đề tài hướng tới 5 mục tiêu chính:

- Thiết kế hệ thống thu thập dữ liệu qua OBD-II: Trích xuất dữ liệu xe theo thời gian thực, bao gồm các Mode hoạt động như sau:
 - Mode 01: Đọc các dữ liệu theo thời gian thực như: Tốc độ xe, vòng tua máy, vị trí bướm ga, tải động cơ, nhiệt độ nước làm mát, nhiên liệu.
 - Mode 03: Truy xuất các mã lỗi liên quan đến động cơ và khí thải được lưu trữ trên hộp ECU của xe khi xảy ra lỗi liên quan đến đèn CEL.
 - Mode 07: Tương tự Mode 03, nhưng Mode này truy xuất các mã lỗi DTC đang chờ xử lý, tức là đã phát hiện ra lỗi nhưng chưa đủ điều kiện bật đèn CEL.
 - Mode 09: Đọc mã định danh của phương tiện VIN.
 - Mode 0A: Truy xuất các mã lỗi không thể xóa được bằng các thao tác xóa thông qua Mode 4, các mã lỗi này các mã lỗi được tự xóa sau nhiều lần tự xác nhận.

- Để phục vụ phát triển, kiểm thử và trình diễn, nhóm sử dụng thêm bộ mô phỏng ECU. Đây chỉ là công cụ hỗ trợ đánh giá thiết bị đọc OBD-II, không thuộc phạm vi thiết kế chính của đề tài.
- Xây dựng ứng dụng di động hỗ trợ giám sát các thông số xe và có thể gửi yêu cầu đọc dữ liệu từ xa thay thế cho các thao tác trực tiếp trên phần cứng.

4. Giới hạn của đề tài

Do những giới hạn về mặt thời gian, tài nguyên cũng như yêu cầu khắt khe về an toàn trong phần mềm ô tô, đề tài được giới hạn trong các phạm vi cụ thể như sau:

- Dù cổng OBD-II hỗ trợ nhiều giao thức như SAE J1850, ISO 9141-2 K-Line, nhưng nghiên cứu này chỉ tập trung thiết kế phần cứng để giao tiếp qua mạng CAN bus.
- Đề tài chỉ sử dụng 5 Mode giao tiếp để chỉ đọc dữ liệu là Mode 01, 03, 07, 09 và 0A, hoàn toàn không can thiệp vào việc chỉnh sửa, cài đặt, xóa hay nạp dữ liệu của ECU.
- Hệ thống chỉ đọc mã lỗi Mode 03, 07, 0A từ ECM và TCM nơi lưu lỗi động cơ và khí thải.
- Phạm vi dòng xe: Đề tài áp dụng cho các dòng xe động cơ xăng tuân thủ tiêu chuẩn SAE J1962.
- Hệ thống chỉ đưa ra cảnh báo tự động trên ứng dụng di động Android khi phát hiện các thông số vượt ngưỡng và mã lỗi hệ thống. Chưa mở rộng tính năng đưa ra giải pháp kỹ thuật và hướng dẫn xử lý sự cố.

5. Bố cục của đề tài

Chương 1: Giới thiệu

Giới thiệu khái quát về tình hình nghiên cứu hiện nay, lý do chọn đề tài, mục tiêu đề tài đặt ra cũng như những giới hạn đề tài đi kèm.

Chương 2: Cơ sở lý thuyết

Nêu rõ các lý thuyết về mạng CAN, tiêu chuẩn OBD-II trên phương tiện, giao tiếp SPI và hệ điều hành thời gian thực.

Chương 3: Thiết kế và thi công hệ thống

Trình bày rõ ràng về các sơ đồ khối hệ thống, chức năng của từng khối, sơ đồ nguyên lý của các linh kiện dùng trong hệ thống, lưu đồ thuật toán hệ thống và các lưu đồ con.

Chương 4: Kết quả thực hiện

Trình bày kết quả hệ thống thực hiện được trên phần cứng và giao diện ứng dụng Android, phân tích log kết quả đi kèm.

Chương 5: Kết luận và hướng phát triển

Đánh giá lại kết quả đạt được những gì, đưa ra những hạn chế và những điểm có thể phát triển sau này của hệ thống.

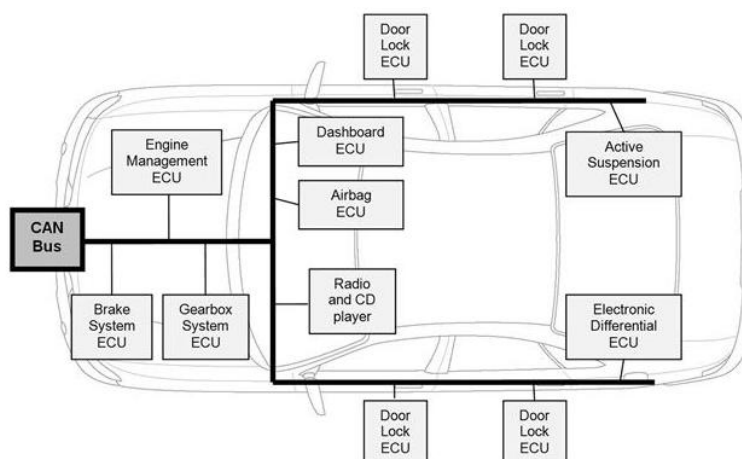
CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

2.1 Giao thức truyền thông CAN

2.2.1 Tổng quan về mạng CAN

Controller Area Network (CAN hoặc CAN Bus) là chuẩn truyền thông nối tiếp được phát triển bởi Bosch vào những năm 1980, nhằm phục vụ trao đổi dữ liệu giữa các bộ điều khiển điện tử (ECU) trong các hệ thống ô tô và công nghiệp [4].



Hình 2.1. Tổng quan mạng CAN trong ô tô

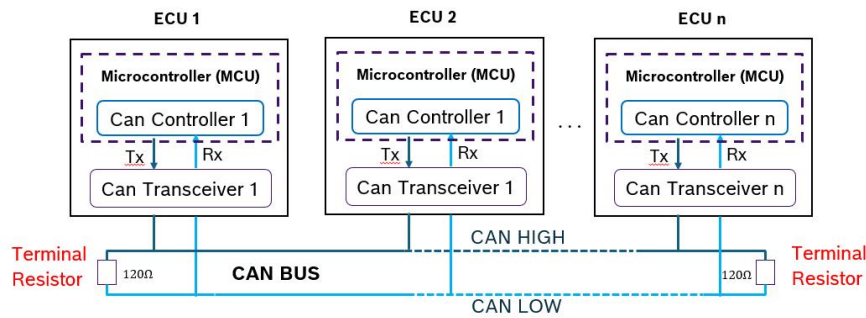
Hình 2.1 mô tả CAN cho phép nhiều ECU giao tiếp với nhau thông qua một đường truyền chung mà không cần đến bộ điều khiển trung tâm. Mỗi ECU trong hệ thống được coi là một node và tất cả các node đều có thể gửi và nhận dữ liệu trên cùng một bus.

Một đặc điểm quan trọng của CAN là cơ chế truyền thông dựa trên thông điệp, thay vì gửi dữ liệu đến một địa chỉ cụ thể như trong các giao thức truyền thông khác thì CAN sử dụng các định danh khung tin (ID) để xác định nội dung dữ liệu. Các node sẽ lọc và xử lý những thông điệp phù hợp với nhu cầu của mình.

Ngoài ra, CAN là giao thức đa chủ, cho phép bất kỳ node nào cũng có thể khởi tạo quá trình truyền dữ liệu khi bus rảnh, giúp tăng tính linh hoạt và giảm độ trễ trong hệ thống.

Nhờ các đặc tính như độ tin cậy cao, khả năng chống nhiễu tốt, chi phí thấp và dễ triển khai, CAN hiện nay được sử dụng rộng rãi trong các hệ thống ô tô hiện đại để kết nối các ECU như động cơ, phanh, hệ thống lái và các hệ thống hỗ trợ khác.

2.2.2 Kiến trúc mạng CAN



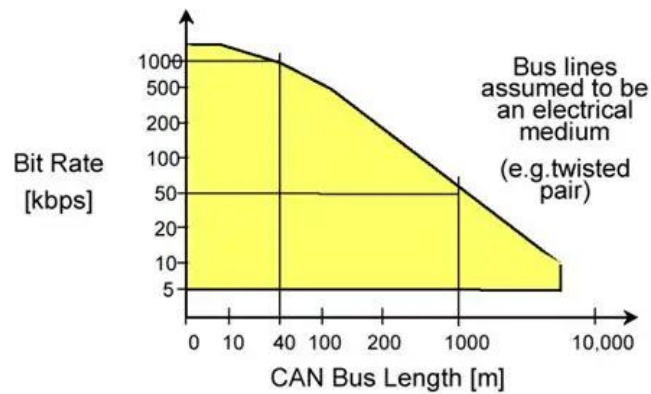
Hình 2.2. Kiến trúc mạng CAN

Trong hệ thống mạng lưới điện trên ô tô, mạng CAN thường được triển khai theo cấu trúc liên kết dạng đường trục (bus), nghĩa là các ECU thuộc cùng một phân đoạn mạng sẽ cùng chia sẻ cặp dây vi sai để trao đổi dữ liệu. Mỗi ECU tham gia mạng CAN có thể được xem như một node. Trong thực tế, không phải tất cả các ECU đều nối chung vào một CAN bus duy nhất. Thay vào đó, hệ thống thường được phân chia thành nhiều mạng con theo nhu cầu của hệ thống và phạm vi chức năng, sau đó có thể liên kết với nhau qua gateway để trao đổi dữ liệu giữa các mạng.

Hình 2.2 còn mô tả cấu trúc của một ECU, bao gồm ba thành phần chính: Microcontroller, CAN Controller và CAN Transceiver.

- MCU: là bộ não của ECU, nó diễn giải các thông điệp CAN được truyền đến và quyết định thông điệp nào cần truyền đi.
- CAN Controller: Bộ điều khiển này thường được tích hợp trong MCU và đảm bảo tất cả các giao tiếp tuân thủ giao thức CAN (mã hóa thông điệp, phát hiện lỗi, tranh chấp) giúp giảm độ phức tạp cho ECU.
- CAN Transceiver: Chuyên đổi tín hiệu logic mà vi điều khiển hiểu sang tín hiệu vi sai trên đường truyền và ngược lại.

Ngoài ra, việc chia ra nhiều mạng con này còn liên quan đến ràng buộc vật lý của bus về tốc độ truyền phụ thuộc chiều dài của đường bus. Hình 2.3 mô tả rằng tốc độ truyền càng nhanh thì chiều dài của bus càng nhỏ và ngược lại. CAN bus thường được cấu hình ở tốc độ 500 kbps để đảm bảo sự ổn định và tin cậy trong quá trình truyền dữ liệu. Trong đồ án, nhóm thực hiện cũng cấu hình các thông số tương tự, sử dụng tốc độ truyền 500 kbps để đảm bảo phù hợp với các tiêu chuẩn thực tế và đảm bảo hiệu suất hoạt động tốt nhất.



Hình 2.3. Tốc độ truyền và chiều dài của CAN tiêu chuẩn

Hình 2.4 mô tả về mặt cấu trúc vật lý, mạng CAN sử dụng hai dây truyền dẫn được xoắn đôi lại với nhau là CANH (CAN high) và CANL (CAN low), cho phép truyền dữ liệu dạng nối tiếp hai chiều với độ tin cậy cao [5].



Hình 2.4. Hai dây xoắn đôi CAN_H và CAN_L

Một yêu cầu quan trọng của kiến trúc bus là điện trở kết thúc. Mỗi mạng bus cần đặt hai điện trở $120\ \Omega$ ở hai đầu dây [5] nhằm hấp thụ năng lượng tín hiệu và hạn chế hiện tượng phản xạ tín hiệu từ đầu cáp gây nhiễu và sai lệch dữ liệu.

2.2.3 Cấu trúc khung tin CAN

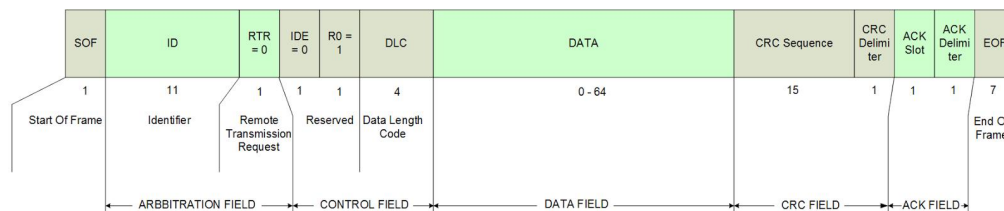
Trong mạng CAN, dữ liệu được truyền giữa các ECU dưới dạng khung tin (frame). Tùy theo mục đích sử dụng trong quá trình truyền thông, CAN định nghĩa một số loại khung tin khác nhau để truyền tải dữ liệu và điều khiển [4] bao gồm:

- Khung dữ liệu (Data frame): là khung mang dữ liệu từ một bộ truyền đến các bộ nhận. Khung này có vùng để mang các byte dữ liệu.
- Khung yêu cầu hay khung điều khiển (Remote frame): là khung được truyền từ một Node để yêu cầu Node khác truyền khung dữ liệu có ID trùng với khung yêu cầu.
- Khung báo lỗi (Error frame): là khung được truyền bởi bất kỳ Node nào khi Node đó phát hiện lỗi trên mạng bus

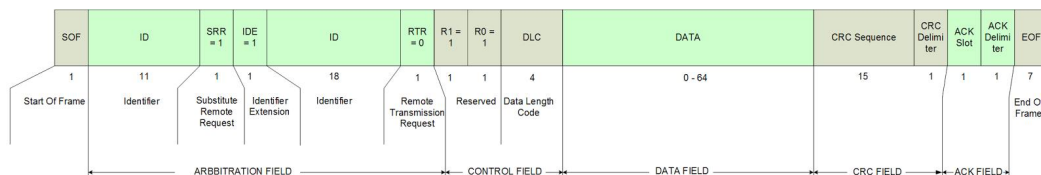
- Khung báo quá tải (Overload frame): được sử dụng để tạo thêm độ trễ giữa giữa các khung dữ liệu hoặc khung yêu cầu. Mỗi Node trong bus CAN có thể truyền bất kỳ khi nào nếu phát hiện bus rảnh, nếu một Node nhận quá nhiều dữ liệu, nó có thể dùng khung này để ngăn sự truyền tiếp theo.

Trong đó, Data frame là loại khung tin quan trọng và được sử dụng phổ biến nhất, đóng vai trò truyền dữ liệu thực tế giữa các ECU trong hệ thống. Các loại khung tin còn lại chủ yếu phục vụ các chức năng hỗ trợ như yêu cầu dữ liệu, phát hiện và xử lý lỗi trong quá trình truyền thông.

Do phạm vi của đề án tập trung vào đọc dữ liệu và đọc lỗi thông qua cổng OBD-II, do đó chủ yếu quan tâm đến các thông tin được truyền dưới dạng dữ liệu trên mạng CAN. Nội dung trong mục này sẽ tập trung phân tích cấu trúc của Data frame, trong khi các loại khung tin khác chỉ được đề cập ở mức tổng quan.



Hình 2.5. Cấu trúc khung dữ liệu chuẩn (Standard Data frame)



Hình 2.6. Cấu trúc khung dữ liệu mở rộng (Extended Data frame)

Để truyền dữ liệu trên mạng CAN, thông tin được đóng gói theo một cấu trúc khung tin xác định. Trong chuẩn CAN classic, khung dữ liệu có thể được tổ chức theo hai dạng chính là khung dữ liệu chuẩn với định danh 11 bit và khung dữ liệu mở rộng với định danh 29 bit. Hai dạng khung này về cơ bản có cấu trúc tương tự nhau chỉ khác ở độ dài trường định danh. Hình 2.5 và hình 2.6 lần lượt minh họa cấu trúc của hai loại khung dữ liệu này.

Dựa theo tiêu chuẩn ISO 11898-1, các trường trong cấu trúc một khung dữ liệu được định nghĩa chi tiết như sau [4]:

- Start of Frame (SOF): Luôn là 1 bit có giá trị “0”. Khi bus đang ở trạng thái nghỉ (recessive =1), sự xuất hiện của bit “0” giúp các node nhận biết rằng một frame mới đang bắt đầu truyền dữ liệu.
- Identifier (ID): Gồm 11 bit đối với khung dữ liệu chuẩn và 29 bit đối với khung dữ liệu mở rộng. Trường này dùng để định danh khung tin và tham gia vào cơ chế phân xử ưu tiên trên bus
- Remote Transmission Request (RTR): Dùng để phân biệt với Remote frame
- Identifier Extension (IDE): Dùng để phân biệt giữa hai loại khung dữ liệu chuẩn và mở rộng
- Reserved bit (r0): Được sử dụng để dự trữ cho các mở rộng trong tương lai của chuẩn CAN.
- Data Length Code (DLC): Gồm 4 bit, dùng để xác định độ dài vùng dữ liệu (Data field), cho phép từ 0 đến 8 byte trong CAN classic.
- Data field: Là phần chứa dữ liệu thực tế, có độ dài từ 0 đến 8 byte tùy theo giá trị DLC.
- Cycle Redundancy Check (CRC): Gồm 15 bit, được dùng để kiểm tra tính toàn vẹn của dữ liệu trong frame. Bên nhận sẽ tính toán lại CRC và so sánh với giá trị nhận được để phát hiện lỗi.
- CRC delimiter: là 1 bit “1” dùng để phân cách giữa hai trường CRC và ACK
- Acknowledgement field (ACK): Dùng để xác nhận lại gói tin truyền đi có thành công hay không, gồm 2 phần:
 - ACK slot: 1 bit, bên truyền sẽ gửi đi bit “1”, các node nhận đúng frame sẽ ghi đè thành “0”
 - ACK delimiter: 1 bit, dùng để kết thúc trường ACK
- End of Frame (EOF): Gồm 7 bit “1”, đánh dấu kết thúc hoàn chỉnh của một frame

2.2.3 Nguyên lý hoạt động

Trong mạng CAN, các ECU giao tiếp với nhau thông qua cơ chế truyền thông đa điểm, trong đó bất kỳ ECU nào cũng có thể truyền dữ liệu lên bus khi đường truyền rảnh. Quá trình hoạt động của CAN dựa trên các nguyên lý chính bao gồm truy nhập bus, cơ chế phân xử ưu tiên và logic truyền tín hiệu vi sai.

- Cơ chế truy cập bus

Mạng CAN sử dụng phương pháp Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA), nghĩa là mỗi node sẽ kiểm tra trạng thái của bus trước khi truyền. Nếu bus đang rảnh (trạng thái recessive), node có thể bắt đầu gửi khung dữ liệu. Nếu bus đang được sử dụng, node sẽ chờ cho đến khi bus rảnh mới tiếp tục truyền.

Cơ chế này giúp tránh xung đột dữ liệu ngay từ đầu và đảm bảo quá trình truyền thông diễn ra có kiểm soát.

- Cơ chế phân xử ưu tiên

Trong trường hợp có nhiều node cùng bắt đầu truyền tại cùng một thời điểm, CAN sử dụng cơ chế phân xử theo từng bit (bitwise arbitration) để xác định node nào được phép tiếp tục truyền.

Cơ chế này dựa trên hai mức logic của bus: bit dominant (logic 0) và bit recessive (logic 1). Nguyên tắc cơ bản là bit dominant luôn thắng bit recessive.

Khi bắt đầu truyền, các node sẽ gửi đồng thời ID của khung dữ liệu lên bus. Trong quá trình này, mỗi node vừa phát dữ liệu vừa đồng thời đọc lại trạng thái thực tế trên bus để so sánh với bit mà nó đã gửi đi. Nếu node gửi bit recessive nhưng đọc được trên bus là bit dominant thì node đó nhận biết rằng có node khác có độ ưu tiên cao hơn, do đó sẽ ngừng truyền ngay lập tức. Ngược lại, nếu node không bị thua trong quá trình phân xử thì sẽ tiếp tục truyền toàn bộ khung dữ liệu.

	Start Bit	ID BITS	
		10987654	
Node 15	0	00000000	
Node 16	0	00000001	
Can data	0	00000000	

	Start Bit	ID BITS	
		109876543210	
Node 15	0	000000001111	keep Transmitting !!
Node 16	0	00000001	Stopped Transmission !!
Can data	0	000000001111	

Hình 2.7. Ví dụ về cơ chế phân xử trên mạng CAN

Hình 2.7 là một ví dụ minh họa cho thấy hai node (node 15 và node 1) bắt đầu truyền đồng thời dữ liệu lên bus. Tại một thời điểm, node 16 phát bit recessive (logic 1) trong khi node 15 phát bit dominant (logic 0). Do bit dominant luôn thắng bit recessive trên bus nên node 16 sẽ dừng truyền, trong khi node 15 tiếp tục truyền toàn bộ khung dữ liệu.

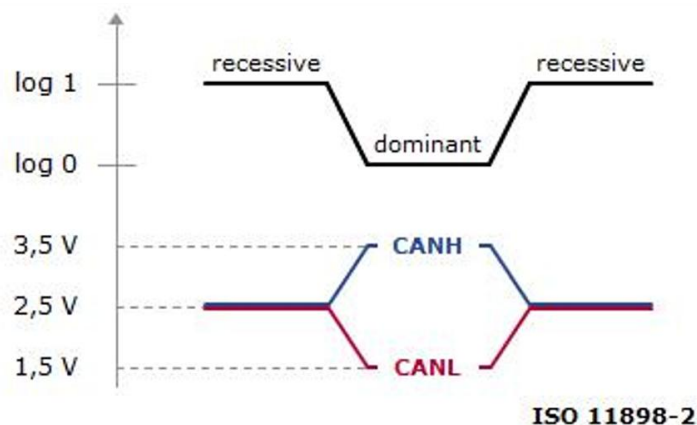
Từ cơ chế này có thể rút ra kết luận quan trọng là nếu có nhiều node truyền dữ liệu đồng thời lên đường bus, node nào có giá trị ID nhỏ hơn sẽ có độ ưu tiên cao hơn

và được truyền trước. Ưu điểm của cơ chế phân xử trong CAN là không làm mất dữ liệu. Các node thua phân xử chỉ tạm dừng và có thể truyền lại sau đó mà không gây lỗi hoặc làm hỏng khung tin đang được truyền trên bus.

- Truyền tín hiệu vi sai trên mạng CAN

Trong mạng CAN, tín hiệu được truyền theo phương pháp truyền vi sai thông qua hai dây là CAN_High và CAN_Low. Thay vì sử dụng điện áp của một dây so với Mass, CAN xác định giá trị logic dựa trên hiệu điện thế giữa hai dây này, giúp tăng độ chính xác và khả năng chống nhiễu trong môi trường làm việc nhiều nhiễu điện từ như hệ thống ô tô.

Đối với High-speed CAN (ISO 11898-2), tín hiệu trên bus có hai trạng thái như được minh họa trong hình 2.8.



Hình 2.8. Quan hệ giữa trạng thái logic (dominant/recessive) và điện áp trên 2 dây CAN trong mạng High-speed CAN (ISO 11898-2)

Hình 2.8 minh họa cho thấy trạng thái dominant tương ứng với sự chênh lệch điện áp giữa CAN_High và CAN_Low, trong khi trạng thái recessive có điện áp gần bằng nhau.

Ưu điểm của phương pháp truyền vi sai là khả năng chống nhiễu điện từ. Khi nhiễu từ môi trường tác động lên cả hai dây, mức nhiễu thường giống nhau trên cả hai dây. Do bộ thu chỉ quan tâm đến hiệu điện thế giữa hai dây, nên phần lớn nhiễu sẽ bị triệt tiêu, đảm bảo tín hiệu nhận được vẫn chính xác.

2.2 Giao thức CAN TP (ISO 15765-2)

2.2.1 Tổng quan về CAN TP

CAN Transport Protocol (CAN TP), được định nghĩa trong ISO 15765-2, là giao thức ở tầng vận chuyển và tầng mạng dùng để hỗ trợ truyền dữ liệu chẩn đoán trên mạng CAN [6]. Giao thức này được sử dụng phổ biến trong các hệ thống như OBD-II và UDS nhằm cho phép trao đổi dữ liệu giữa thiết bị chẩn đoán (tester) và ECU.

Do đặc điểm của CAN chỉ cho phép truyền dữ liệu với kích thước giới hạn (tối đa 8 byte đối với Classical CAN), CAN TP cung cấp cơ chế phân mảnh dữ liệu để truyền các bản tin có kích thước lớn hơn. Dữ liệu sẽ được chia nhỏ thành nhiều phần ở phía gửi và được ghép lại ở phía nhận, đảm bảo truyền thông tin đầy đủ và chính xác.

2.2.2 Các loại frame trong CAN

Tiêu chuẩn ISO 15765-2 định nghĩa bốn loại frame chính Network Protocol Data Unit (N_PDU) được sử dụng trong quá trình truyền dữ liệu [6]:

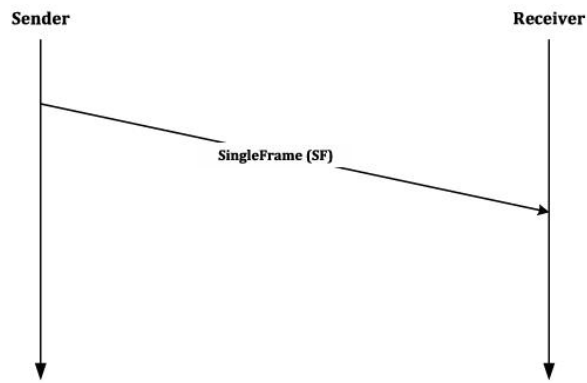
- Single Frame (SF): Dùng để truyền dữ liệu có kích thước nhỏ, nằm trong một CAN frame duy nhất
- First Frame (FF): Frame đầu tiên trong quá trình truyền dữ liệu dài, chứa thông tin tổng độ dài gói tin
- Consecutive Frame (CF): Các frame tiếp theo chứa phần dữ liệu đã được phân chia
- Flow Control (FC): Frame do phía nhận gửi để điều khiển tốc độ truyền và đảm bảo không quá tải buffer.

2.2.3 Nguyên lý hoạt động của CAN TP

2.2.3.1 Truyền dữ liệu bằng khung đơn SF

Trong trường hợp dữ liệu có kích thước nhỏ, bản tin có thể được truyền trực tiếp bằng một Single Frame mà không cần phân mảnh.

Hình 2.9 mô tả quá trình truyền gói tin không cần phân mảnh, trong đó dữ liệu được gửi trực tiếp từ phía gửi đến phía nhận thông qua một frame duy nhất.

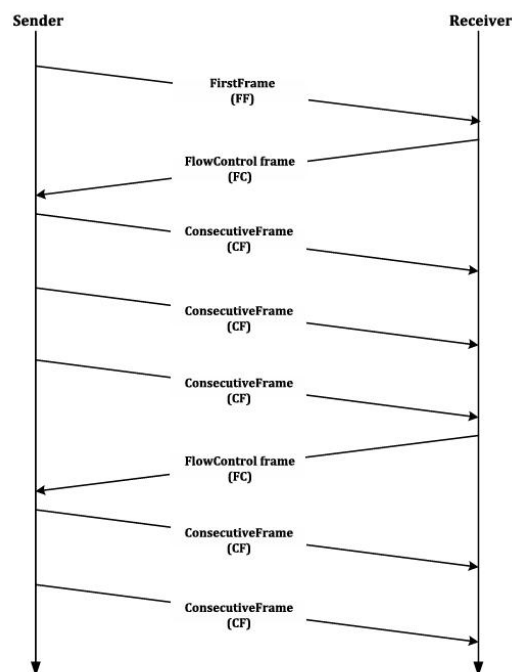


Hình 2.9. Truyền dữ liệu sử dụng SF trong CAN TP

2.2.3.2 Truyền dữ liệu đa khung

Khi dữ liệu vượt quá kích thước của một CAN frame, CAN TP sử dụng cơ chế truyền nhiều frame. Quá trình này diễn ra theo trình tự:

1. Bên gửi gửi First Frame (FF) để thông báo độ dài dữ liệu
2. Bên nhận phản hồi Flow Control (FC)
3. Bên gửi truyền các Consecutive Frame (CF)
4. Bên nhận ghép các frame để tái tạo dữ liệu ban đầu



Hình 2.10. Cơ chế truyền nhiều frame trong CAN TP

Hình 2.10 mô tả quá trình truyền dữ liệu dài với sự tham gia của các frame điều khiển luồng, giúp phía nhận kiểm soát tốc độ truyền và đảm bảo không bị quá tải bộ đệm.

2.2.3.3 Cơ chế điều khiển luồng

Flow Control cho phép phía nhận điều chỉnh quá trình truyền thông qua các tham số chính [6]:

- Flow Status (FS): trạng thái truyền (tiếp tục, chờ hoặc dừng)
- Block Size (BS): số lượng frame được phép gửi liên tiếp
- Separation Time (ST_{min}): khoảng thời gian tối thiểu giữa hai frame

Cơ chế này giúp đồng bộ tốc độ giữa sender và receiver, đảm bảo hệ thống hoạt động ổn định.

2.3 Tiêu chuẩn chẩn đoán OBD-II

2.3.1 Tổng quan về OBD-II

OBD- II (On- Board Diagnostics II) là tiêu chuẩn chẩn đoán được tích hợp trên các phương tiện ô tô nhằm giám sát trạng thái hoạt động của động cơ và các hệ thống liên quan, đặc biệt là hệ thống kiểm soát khí thải. Tiêu chuẩn này cho phép thu thập, lưu trữ và truyền các thông tin chẩn đoán phục vụ cho việc phát hiện lỗi và kiểm tra tình trạng vận hành của phương tiện.

Trong hệ thống OBD- II, các ECU có nhiệm vụ giám sát và điều khiển các hệ thống con trên xe. Một số ECU điển hình bao gồm bộ điều khiển động cơ (ECM), bộ điều khiển hộp số (TCM) và bộ điều khiển phanh điện tử (EBCM). Khi phát hiện bất thường, ECU sẽ ghi nhận lỗi và cung cấp thông tin chẩn đoán thông qua giao thức OBD- II, giúp kỹ thuật viên xác định và xử lý sự cố.

OBD- II yêu cầu các phương tiện như xe du lịch, xe tải nhẹ và xe tải hạng trung phải hỗ trợ truyền một tập hợp tối thiểu các thông tin chẩn đoán ra bên ngoài. Việc giao tiếp này được thực hiện thông qua cổng kết nối chuẩn, cho phép các thiết bị như máy đọc lỗi kết nối trực tiếp với hệ thống trên xe để truy xuất dữ liệu và mã lỗi.

Do đó, OBD- II đóng vai trò quan trọng trong việc phát hiện lỗi động cơ, giám sát hiệu suất vận hành và góp phần giảm thiểu khí thải từ phương tiện.

2.3.2 Cổng kết nối OBD-II theo tiêu chuẩn ISO 15031-3

Tiêu chuẩn OBD- II được phát triển nhằm đáp ứng yêu cầu chẩn đoán và kiểm soát khí thải trên các phương tiện ô tô. Ban đầu, chuẩn SAE J1979 được xây dựng để đáp ứng các quy định OBD tại Hoa Kỳ, áp dụng cho các dòng xe từ năm 1996 trở đi [7]. Sau đó, tiêu chuẩn ISO 15031- 5 được phát triển dựa trên SAE J1979, kết hợp các

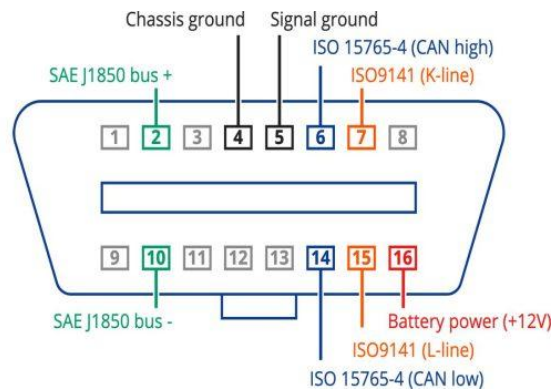
yêu cầu chẩn đoán của Hoa Kỳ và châu Âu, áp dụng cho các phương tiện từ năm 2000 trở đi [8].



Hình 2.11. Đầu nối OBD-II (Female) trên ô tô

Bên cạnh giao thức chẩn đoán, OBD- II còn quy định chuẩn về phần cứng kết nối. Tiêu chuẩn ISO 15031-3 định nghĩa đầu nối chẩn đoán Diagnostic Link Connector, bao gồm các yêu cầu về vị trí lắp đặt, khả năng truy cập, số lượng chân và đặc tính điện, cơ khí của đầu nối [9].

Đầu nối DLC theo ISO 15031-3 thường gồm 16 chân và đóng vai trò là giao diện vật lý giữa hệ thống trên xe và thiết bị chẩn đoán bên ngoài. Thông qua cổng này, thiết bị đọc lỗi có thể truy xuất dữ liệu từ ECU, bao gồm các thông số vận hành, mã lỗi DTC và các thông tin chẩn đoán khác. Trong phạm vi đồ án, chuẩn ISO 15031-3 được sử dụng làm cơ sở để xác định sơ đồ chân của cổng OBD- II, phục vụ cho việc thiết kế mạch giao tiếp giữa bộ đọc ESP32 và ECU.



Hình 2.12. Male OBD-II connector

Trong chuẩn SAE J1962 hoặc tiêu chuẩn quốc tế ISO 15031-3 [10], các chân kết nối được quy định để hỗ trợ nhiều giao thức truyền thông khác nhau. Đối với hệ thống sử dụng CAN theo tiêu chuẩn ISO 15765- 4, sơ đồ chân được quy định như sau:

- Chân 6: Tín hiệu CAN High
- Chân 14: Tín hiệu CAN Low

Ngoài ra, đầu nối còn hỗ trợ các chân nguồn và các giao thức đời cũ như:

- Chân 4 và Chân 5: Nối mass sườn xe (Chassis Ground) và mass tín hiệu (Signal Ground).
- Chân 16: Cấp nguồn dương từ ắc quy, với mức điện áp phổ biến là 12V đối với đầu nối Type A (xe du lịch) và 24V đối với Type B (xe tải hạng nặng), phục vụ cấp nguồn nuôi trực tiếp cho thiết bị chẩn đoán [10]
- ISO 9141 (K-Line): Sử dụng chân 7 và chân 15
- SAE J1850: Sử dụng chân 2 (Bus+) và chân 10 (Bus-)

Các giao thức cũ chủ yếu được sử dụng trên các dòng xe thế hệ trước khi chuẩn giao tiếp CAN tốc độ cao trở thành tiêu chuẩn bắt buộc.

2.3.3 Các chế độ chẩn đoán

Theo tiêu chuẩn SAE J1979, OBD-II hỗ trợ 10 Mode chẩn đoán tiêu chuẩn [7]:

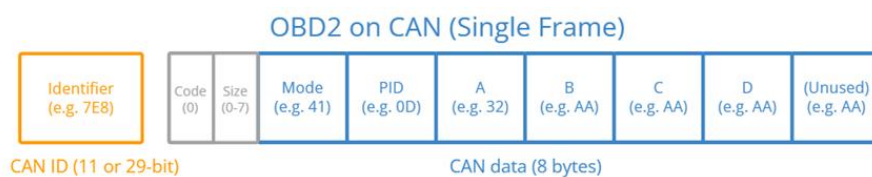
- Mode 01: Chế độ này cung cấp dữ liệu thời gian thực từ các hộp ECU trên xe, cho phép quan sát trực tiếp trạng thái hoạt động hiện tại của xe. Các thông số có thể được giám sát bao gồm: Tốc độ động cơ, nhiệt độ nước làm mát, vị trí bướm ga, tốc độ xe, ...
- Mode 02: Khi xảy ra lỗi và đèn CEL được kích hoạt, chế độ 2 sẽ lưu lại một “snapshot” trạng thái hoạt động của xe tại thời điểm lỗi xuất hiện. Dữ liệu này thường bao gồm các thông số như: Tải động cơ, tốc độ xe, nhiệt độ động cơ, trạng thái hệ thống nhiên liệu,...
- Mode 03: Chế độ này dùng để truy xuất các mã lỗi chẩn đoán DTC được lưu trong máy tính trên xe. Các mã lỗi này giúp xác định những sự cố cụ thể liên quan đến động cơ, hệ thống phát thải và các bộ phận quan trọng khác của phương tiện.
- Mode 04: Sau khi hoàn thành sửa chữa, chế độ 4 cho phép xóa các mã lỗi DTC đã được lưu trữ và tắt đèn CEL.
- Mode 05: Kiểm tra cảm biến oxy. Dịch vụ này không hỗ trợ trên chuẩn ISO 15765-4 mà được tích hợp hoàn toàn vào Mode 6.
- Mode 06: Truy xuất kết quả kiểm tra giám sát trên xe đối với các hệ thống được giám sát liên tục và không liên tục. Kết quả được trả về kèm theo các

giới hạn tối thiểu và tối đa của nhà sản xuất. Nếu bài test chưa từng được hoàn thành kể từ lần xóa lỗi gần nhất, giá trị trả về sẽ đặt về 0.

- Mode 07: Truy xuất các mã lỗi đang chờ xử lý hoặc lỗi tạm thời được phát hiện trong chu kỳ lái xe hiện tại hoặc chu kỳ gần nhất. Dịch vụ này giúp kỹ thuật viên kiểm tra nhanh kết quả sửa chữa ngay sau một chu kỳ lái xe duy nhất mà không cần phải đợi đủ điều kiện bật đèn CEL.
- Mode 08: Cho phép thiết bị kiểm tra bên ngoài kiểm tra và kiểm soát hoạt động của các hệ thống, bài test hoặc linh kiện trên xe. Các lệnh điều khiển có thể yêu cầu linh kiện: Bật, tắt hoặc kích hoạt tuần hoàn trong vòng n giây.
- Mode 09: Yêu cầu các thông tin đặc định của xe như số nhận dạng xe VIN và mã định danh hiệu chuẩn. Nếu mã VIN chưa được lập, ECU vẫn phải phản hồi bằng các ký tự byte đệm.
- Mode 0A: Hiển thị các mã lỗi DTC vĩnh viễn chỉ có thể được xóa sau khi hệ thống trên xe xác nhận vấn đề đã được giải quyết thông qua nhiều chu kỳ lái xe.

2.3.4 Cấu trúc thông điệp chẩn đoán OBD-II trên mạng CAN

Trong hệ thống OBD-II sử dụng giao thức CAN, thông điệp chẩn đoán được truyền dưới dạng khung CAN. Một thông điệp OBD-II cơ bản thường bao gồm CAN Identifier và trường dữ liệu CAN gồm 8 byte. CAN Identifier có thể sử dụng định dạng 11-bit hoặc 29-bit, dùng để xác định địa chỉ của ECU hoặc loại thông điệp được truyền trên mạng CAN.



Hình 2.13. Cấu trúc thông điệp chẩn đoán OBD-II trên CAN

Identifier: Mã định danh CAN, dùng để xác định thông điệp đến từ ECU nào hoặc gửi đến ECU nào. Ví dụ ở frame yêu cầu, ID là 0x7DF tức là gửi broadcast đến toàn mạng CAN thì ở frame phản hồi, ID có thể là 0x7E0 hoặc 0x7E8 của ECM (Engine Control Module), 0x7E1 hoặc 0x7E9 của TCM (Transmission Control Module) [11].

Code và Size: Frame CAN theo ISO-TP nên byte đầu gồm Code và Size.

- Code = 0: Single Frame
- Code = 1: First Frame, frame đầu có một dữ liệu dài
- Code = 2: Consecutive frame, các frame tiếp theo của dữ liệu dài
- Code = 3: Flow control frame, frame điều khiển luồng truyền
- Size (0 - 7): Số lượng byte phục vụ việc chẩn đoán

Mode : Theo tiêu chuẩn SAE J1979, OBD-II hỗ trợ 10 Mode với giá trị từ 0x01 - 0x0A đối với frame yêu cầu và 0x41 - 0x4A đối với frame phản hồi.

PID (Parameter Identifiers): Dùng để chỉ rõ thông số cần đọc từ ECU. Mỗi PID đại diện cho một dữ liệu cụ thể của xe đi theo từng Mode, ví dụ tốc độ xe, vòng tua động cơ hoặc nhiệt độ động cơ. Các PID tiêu chuẩn được quy định trong SAE J1979, nhưng mức độ hỗ trợ có thể khác nhau tùy từng dòng xe. Ở cả frame yêu cầu và frame phản hồi thì giá trị này đều không đổi.

2.3.5 Quy trình đóng gói thông điệp chẩn đoán chi tiết theo từng Mode

2.3.5.1 Quy trình đóng gói và truyền nhận Mode 01

Theo quy định tại mục 7.1.1 thuộc tiêu chuẩn ISO 15031-5 sửa đổi năm 2006, mục đích của chế độ này là cho phép thiết bị kiểm thử truy cập vào các giá trị dữ liệu cảm biến thời gian thực hiện tại liên quan đến khí thải của phương tiện. Tiêu chuẩn bắt buộc các dữ liệu này phải là giá trị đo thực tế, không được lấy giá trị mặc định khi có lỗi hệ thống.

Vì kích thước dữ liệu yêu cầu và phản hồi của từng thông số độc lập rất ngắn luôn nhỏ hơn hoặc bằng 7 byte, nên quy trình truyền thông giữa mạch kiểm thử và ECU xe được đóng gói hoàn toàn dưới dạng khung đơn SF theo giao thức CAN TP ISO 15765-2.

Cấu trúc đóng gói khung yêu cầu dữ liệu:

Cấu trúc khung tin yêu cầu đọc giá trị thông số PID được thiết lập nghiêm ngặt dựa trên quy định tại mục 7.1.2.3 và Bảng 127 (Table 127 – Request current powertrain diagnostic data request message) của tiêu chuẩn ISO 15031-5.

Bảng 2.1. Định danh cấu trúc các byte dữ liệu trong khung yêu cầu Mode 01

Hướng khung tin		Thiết bị kiểm thử → ECU		
ID		0x7DF		
Loại khung tin		Yêu cầu		
Số thứ tự Byte	Trường dữ liệu	Giá trị (Hex)	Ý nghĩa	

Byte 0	PCI	0x02	Khung đơn SF, thông báo độ dài dữ liệu kế tiếp là 2 byte
Byte 1	SID	0x01	Mã ID Mode 01
Byte 2	PID	0xXX	Mã thông số PID
Byte 3	Padding Byte #1	0x00/ 0xFF	Byte đệm
Byte 4	Padding Byte #2	0x00/ 0xFF	Byte đệm
Byte 5	Padding Byte #3	0x00/ 0xFF	Byte đệm
Byte 6	Padding Byte #4	0x00/ 0xFF	Byte đệm
Byte 7	Padding Byte #5	0x00/ 0xFF	Byte đệm

Cấu trúc đóng gói khung phản hồi dữ liệu:

Khi nhận được khung tin hợp lệ từ thiết bị kiểm thử, hộp ECU động cơ xử lý và phát ngược lại khung tin phản hồi tích cực lên mạng bus qua định danh địa chỉ vật lý là CAN ID 0x7E8. Cấu trúc đóng gói các byte phản hồi tuân thủ theo quy định tại Mục 7.1.2.4 và Bảng 128 của tiêu chuẩn, được mạch thiết kế lại trực quan tại Bảng 2.2

Bảng 2.2. Định danh cấu trúc các byte dữ liệu trong khung phản hồi Mode 01

Hướng khung tin		ECU → thiết bị kiểm thử	
ID		0x7E8	
Loại khung tin		Phản hồi	
Số thứ tự Byte	Trường dữ liệu	Giá trị (Hex)	Ý nghĩa
Byte 0	PCI	0x0X	Khung đơn SF, với X là tổng số byte data phía sau
Byte 1	Phản hồi SID	0x41	Mã phản hồi SID = 0x01 + 0x40
Byte 2	Xác nhận PID	0xXX	ECU xác nhận mã PID
Byte 3	Data Byte A	0x00 - 0xFF	Byte dữ liệu thô
Byte 4	Data Byte B	0x00 - 0xFF	Byte dữ liệu thô
Byte 5	Data Byte C/ Padding	0x00 - 0xFF / 0x00	Byte dữ liệu thô/ Byte đệm
Byte 6	Data Byte D / Padding	0x00 - 0xFF / 0x00	Byte dữ liệu thô/ Byte đệm
Byte 7	Padding Byte	0x00	Byte đệm

Công thức giải mã thông số PID của Mode 01

Ngay khi bóc tách được các byte dữ liệu thô CAN (Data Byte A và Data Byte B) từ khung phản hồi, tầng ứng dụng trên vi điều khiển mạch kiểm thử sẽ áp dụng trực tiếp các thuật toán và công thức toán học quy chuẩn để đổi dữ liệu Hex sang giá trị vật lý hiển thị.

Bảng 2.3. Định nghĩa và công thức giải mã các thông số Mode 01 PID của hệ thống

Mã PID (Hex)	Định nghĩa	Số lượng Byte	Ký hiệu Byte thô	Công thức giải mã	Đơn vị
04	Tải động cơ	1	A	$(A * 100) / 255$	%
05	Nhiệt độ nước làm mát động cơ	1	A	$A - 40$	°C
0C	Tốc độ vòng tua máy	2	A, B	$((A * 256) + B) / 4$	Rpm (Vòng/phút)
0D	Tốc độ xe	1	A	A	Km/h
11	Vị trí mở bướm ga tuyệt đối	1	A	$(A * 100) / 255$	%
2F	Nhiên liệu	1	A	$(A * 100) / 255$	%

2.3.5.2 Quy trình đóng gói và truyền nhận Mode 03, Mode 07 và Mode 0A

Theo tiêu chuẩn ISO 15031-5 sửa đổi năm 2006, mục đích của các chế độ này là cho phép thiết bị kiểm thử truy cập vào danh sách các mã lỗi DTC liên quan đến khí thải và lỗi động cơ được lưu trữ trên phương tiện. Tiêu chuẩn quy định rõ ràng chức năng của từng Mode: Mode 03 dùng để đọc các mã lỗi đã xác nhận, Mode 07 dùng để đọc các mã lỗi đang chờ duyệt và Mode 0A dùng để kiểm tra các mã lỗi vĩnh viễn.

Vì kích thước dữ liệu yêu cầu rất ngắn chỉ chứa mã dịch vụ chẩn đoán nhưng kích thước dữ liệu phản hồi phụ thuộc hoàn toàn vào số lượng mã lỗi thực tế hiện có trên phương tiện, nên quy trình truyền thông giữa mạch kiểm thử và ECU xe có thể linh hoạt đóng gói dưới dạng khung đơn SF hoặc dạng đa khung tuân thủ theo giao thức CAN TP ISO 15765-2.

Cấu trúc đóng gói khung yêu cầu dữ liệu:

Cấu trúc khung tin yêu cầu đọc mã lỗi chẩn đoán từ thiết bị kiểm thử được thiết lập dựa trên quy định tầng ứng dụng của tiêu chuẩn, Mạch kiểm thử sẽ đóng gói một thông điệp khung đơn SF và phát lên mạng bus qua địa chỉ định danh CAN ID 0x7DF với cấu trúc phân bổ các byte được mô tả chi tiết tại Bảng 2.4.

Bảng 2.4. Định danh cấu trúc các byte dữ liệu trong khung yêu cầu Mode 03,07 và 0A

Hướng khung tin		Thiết bị kiểm thử → ECU	
ID		0x7DF (địa chỉ quảng bá) 0x7E0 (địa chỉ đích danh ECU động cơ) 0x7E1 (địa chỉ đích danh ECU hộp số)	
Loại khung tin		Yêu cầu	
Số thứ tự Byte	Trường dữ liệu	Giá trị (Hex)	Ý nghĩa
Byte 0	PCI	0x01	Khung đơn SF, thông báo độ dài dữ liệu kế tiếp là 1 byte
Byte 1	SID	0x03/ 0x07/0x0A	Mã ID
Byte 2	Padding Byte #1	0x00/ 0xFF	Byte đệm
Byte 3	Padding Byte #2	0x00/ 0xFF	Byte đệm
Byte 4	Padding Byte #3	0x00/ 0xFF	Byte đệm
Byte 5	Padding Byte #4	0x00/ 0xFF	Byte đệm
Byte 6	Padding Byte #5	0x00/ 0xFF	Byte đệm
Byte 7	Padding Byte #6	0x00/ 0xFF	Byte đệm

Cấu trúc đóng gói khung phản hồi dữ liệu:

Trường hợp 1: Phương tiện phản hồi tiêu cực (Negative Response Frame)

Nếu hộp ECU của dòng xe thử nghiệm không tương thích hoặc không hỗ trợ phân vùng dịch vụ được gọi, ECU sẽ phát trả một khung phản hồi tiêu cực. Khung tin cấu hình dạng Khung đơn SF được mô tả chi tiết tại bảng 2.5.

Bảng 2.5. Định danh cấu trúc các byte trong khung phản hồi tiêu cực Mode 03,07 và 0A

Hướng khung tin		ECU → Thiết bị kiểm thử	
ID		0x7E8 (ECU động cơ) 0x7E9 (ECU hộp số)	
Loại khung tin		Phản hồi tiêu cực	
Số thứ tự Byte	Trường dữ liệu	Giá trị (Hex)	Ý nghĩa
Byte 0	PCI	0x03	Khung đơn SF, thông báo độ dài dữ liệu kế tiếp là 3 byte
Byte 1	Error Marker	0x7F	Mã phản hồi lỗi chẩn đoán
Byte 2	Rejected SID	0x03/ 0x07/ 0x0A	Mã ID Mode bị ECU từ chối thực thi
Byte 3	ResponseCode (NRC)	0xXX	Mã nguyên nhân từ chối đọc lỗi

Danh sách các mã phản hồi tiêu cực NRC theo tiêu chuẩn ISO 15765-4 được mô tả chi tiết tại bảng 2.6.

Bảng 2.6. Danh sách các mã phản hồi tiêu cực NRC

Mã NRC (Hex)	Tên quy chuẩn tiêu chuẩn	Ý nghĩa
0x10	generalReject (GR)	Từ chối chung
0x11	serviceNotSupported (SNS)	Dịch vụ không hỗ trợ
0x12	subFunctionNotSupported-InvalidFormat (SFNSIF)	Định dạng không hợp lệ
0x21	busy-RepeatRequest (BRR)	ECU đang bận
0x22	conditionsNotCorrectOrRequestSequenceError (CNCORSE)	Sai điều kiện hoặc trình tự
0x78	requestCorrectlyReceived-ResponsePending (RCR-RP)	Đang chờ xử lý phản hồi

Trường hợp 2: Phương tiện không có mã lỗi lưu trữ

Khi hệ thống tự chẩn đoán của xe xác nhận các cụm động cơ hoạt động ổn định và không có lỗi, ECU sẽ đóng gói và phản hồi lại trong một khung đơn SF được mô tả chi tiết tại bảng 2.7.

Bảng 2.7. Định danh cấu trúc byte phản hồi khi phương tiện không có mã lỗi

Hướng khung tin	ECU → Thiết bị kiểm thử		
ID	0x7E8 (ECU động cơ) 0x7E9 (ECU hộp số)		
Loại khung tin	Phản hồi tích cực		
Số thứ tự Byte	Trường dữ liệu	Giá trị (Hex)	Ý nghĩa
Byte 0	PCI	0x02	Khung đơn SF, thông báo độ dài dữ liệu kế tiếp là 2 byte
Byte 1	Phản hồi SID	0x43 / 0x47 / 0x4A	Mã phản hồi tích cực
Byte 2	# of DTC	0x00	Số lượng mã lỗi lưu trữ
Byte 3 → Byte 7	Padding Byte	0x00 / 0xFF	Byte đệm

Trường hợp 3: Phương tiện lưu trữ số lượng mã lỗi ít (Từ 1 đến 2 mã lỗi)

Vì mỗi mã lỗi DTC tiêu chuẩn chiếm dụng cố định **2 byte dữ liệu**, khi số lượng lỗi lưu trữ nhỏ hơn hoặc bằng 2 mã, dung lượng Payload này vẫn nằm trong giới hạn chứa tối đa của một khung đơn SF cho phép chứa tối đa 7 byte dữ liệu. Do đó, ECU sẽ đóng gói tập trung toàn bộ danh sách lỗi vào một khung tin CAN duy nhất.

Bảng 2.8. Định danh cấu trúc toàn bộ các byte phản hồi khi có ít hơn 2 mã lỗi

Hướng khung tin	ECU → thiết bị kiểm thử
ID	0x7E8 (ECU động cơ)

		0x7E9 (ECU hộp số)	
Loại khung tin		Phản hồi tích cực	
Số thứ tự Byte	Trường dữ liệu	Giá trị (Hex)	Ý nghĩa
Byte 0	PCI	0x06	Khung đơn SF, thông báo độ dài dữ liệu kế tiếp là 6 byte
Byte 1	Phản hồi SID	0x43/ 0x47/ 0x4A	Mã phản hồi SID
Byte 2	# of DTC	0x0X	X: Số lượng mã lỗi (tối đa 2 mã)
Byte 3	DTC #1 High Byte	0xXX	Byte cao DTC#1
Byte 4	DTC #1 Low Byte	0xXX	Byte thấp DTC#1
Byte 5	DTC #2 High Byte	0xXX	Byte cao DTC#2
Byte 6	DTC #2 Low Byte	0xXX	Byte thấp DTC#2
Byte 7	Padding Byte	0x00	Byte đệm

Trường hợp 4: Phương tiện lưu trữ số lượng mã lỗi lớn (Từ 3 mã lỗi trở lên)

Khi hệ thống phương tiện xảy ra sự cố nghiêm trọng và tích lũy từ 3 mã lỗi trở lên, tổng kích thước dữ liệu bắt buộc phải từ 8 byte trở lên vượt quá khả năng đáp ứng tối đa của khung đơn SF. Do đó phải sử dụng cơ chế truyền nhận đa khung của giao thức CAN TP 15765-2 thông qua chu trình 3 bước được mô tả chi tiết như sau:

Bước 1: ECU phát khung đầu tiên FF

Bảng 2.9. Định danh cấu trúc khung đầu tiên từ ECU Mode 03,07 và 0A

Hướng khung tin		ECU → thiết bị kiểm thử	
ID		0x7E8 (ECU động cơ) 0x7E9 (ECU hộp số)	
Loại khung tin		Phản hồi đa khung – Khung đầu tiên FF	
Số thứ tự Byte	Trường dữ liệu	Giá trị (Hex)	Ý nghĩa
Byte 0	PCI byte 1	0x10	Mã định danh khung đầu tiên FF
Byte 1	PCI Byte 2	0x0E	Tổng chiều dài chuỗi dữ liệu ứng dụng là 14 byte
Byte 2	Phản hồi SID	0x43/ 0x47/ 0x4A	Mã phản hồi SID
Byte 3	# of DTC	0x0X	X: Số lượng mã lỗi
Byte 4	DTC #1 High Byte	0xXX	Byte cao DTC#1
Byte 5	DTC #1 Low Byte	0xXX	Byte thấp DTC#1
Byte 6	DTC #2 High Byte	0xXX	Byte cao DTC#2
Byte 7	DTC #2 Low Byte	0xXX	Byte thấp DTC#2

Bước 2: Thiết bị kiểm thử phản hồi khung điều khiển luồng FC

Bảng 2.10. Định danh cấu trúc khung điều khiển luồng FC từ thiết bị kiểm thử Mode 03,07 và 0A

Hướng khung tin		Thiết bị kiểm thử → ECU	
ID		0x7E0	
Loại khung tin		Khung điều khiển luồng FC	
Số thứ tự Byte	Trường dữ liệu	Giá trị (Hex)	Ý nghĩa
Byte 0	PCI byte	0x30	Định danh khung FC, trạng thái Clear To Send (Sẵn sàng nhận)
Byte 1	Block Size (BS)	0x00	Cho phép ECU gửi liên tục toàn bộ các khung còn lại
Byte 2	Separation_Time (ST_min)	0xXX	Yêu cầu ECU gửi các gói nối tiếp với tốc độ là XX
Byte 3 → Byte 7	Padding Byte	0x00 / 0xFF	Byte đệm

Bước 3: ECU truyền tải các khung nối tiếp CF

Sau khi tiếp nhận xung lệnh thông luồng FC hợp lệ từ thiết bị kiểm thử, ECU xe ngay lập tức giải phóng toàn bộ phần dữ liệu mã lỗi còn lại trong bộ đệm ứng dụng để truyền tải lên mạng bus thông qua các khung nối tiếp.

Nếu số lượng mã lỗi thực tế của phương tiện quá lớn và dung lượng chứa của khung CF1 không thể tải hết, hệ thống phần cứng của ECU sẽ tự động đóng gói phần dữ liệu còn sót lại và chuyển tiếp sang khung nối tiếp thứ hai CF2 (PCI = 0x22), khung thứ ba CF3 (PCI = 0x23), ...

Chuỗi truyền CF này sẽ lặp đi lặp lại liên tục cho đến khi tổng số lượng byte dữ liệu mã lỗi nhận về trên mạng CAN khớp hoàn toàn với tổng chiều dài Payload đã được khai báo trước đó ở Khung đầu tiên (FF). Cấu trúc phân bổ byte tổng quát cho các khung nối tiếp CF được vạch thiết kế chi tiết tại bảng 2.11.

Bảng 2.11. Định danh cấu trúc khung nối tiếp CF từ ECU Mode 03,07 và 0A

Hướng khung tin	ECU → thiết bị kiểm thử
ID	0x7E8 (ECU động cơ) 0x7E9 (ECU hộp số)
Loại khung tin	Phản hồi đa khung – Khung nối tiếp CF

Số thứ tự Byte	Trường dữ liệu	Giá trị (Hex)	Ý nghĩa
Byte 0	PCI Byte	0x2X	2: Mã định danh khung nối tiếp CF X: số lượng khung CF cần thiết
Byte 1	DTC #n High Byte	0xXX	Byte cao DTC#n
Byte 2	DTC #n Low Byte	0xXX	Byte thấp DTC#n
Byte 3	DTC #n+1 High Byte	0xXX	Byte cao DTC#n+1
Byte 4	DTC #n+1 Low Byte	0xXX	Byte thấp DTC#n+1
Byte 5	DTC #n+2 High Byte	0xXX	Byte cao DTC#n+2
Byte 6	DTC #n+2 Low Byte	0xXX	Byte thấp DTC#n+2
Byte 7	DTC #n+3 High Byte / Padding Byte	0xXX / 0x00	Byte thấp DTC#2/ Padding byte nếu không còn lỗi nữa

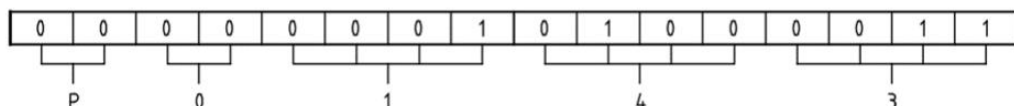
Phương pháp phân tích và giải mã cấu trúc mã lỗi DTC

Mỗi mã lỗi chẩn đoán sự cố DTC lưu trữ trong bộ nhớ ECU được truyền dưới dạng hai byte thông tin cho mỗi mã lỗi bao gồm: Byte cao và Byte thấp. Các mã lỗi đã được tiêu chuẩn hóa cho tất cả các hãng xe và được chia thành 4 loại cơ bản:

- P0xxx: Các lỗi tiêu chuẩn liên quan đến hệ thống động lực.
- C0xxx: Các lỗi tiêu chuẩn liên quan đến khung gầm.
- B0xxx: Các lỗi tiêu chuẩn liên quan đến thân xe.
- U0xxx: Các lỗi tiêu chuẩn liên quan đến mạng truyền thông.

Để chuyển đổi cụm hai byte Hex này thành chuỗi 5 ký tự ký hiệu tiêu chuẩn trực quan hiển thị lên màn hình giao diện, vi điều khiển triển khai giải thuật bóc tách và che mặt nạ bit theo quy chuẩn ISO 15031-6.

Quy luật bóc tách chi tiết cấu trúc bit của từng ký tự đầu ra cùng với ví dụ thực nghiệm giải mã đối với cặp byte thô Byte cao = 0x01 và Byte thấp = 0x43 được tổng hợp chi tiết tại Bảng 2.12.



Hình 2.14. Ví dụ cấu trúc bit mã lỗi DTC P0143

Bảng 2.12. Quy luật bóc tách Bitmasking và giải mã mã lỗi DTC P0143

Ký tự đầu ra	Vị trí bit	Giá trị phân	Kết quả	Quy luật ánh
--------------	------------	--------------	---------	--------------

		tích (Nhị phân)		xạ tiêu chuẩn (ISO 15031-6)
#1 (Hệ thống)	Bit [7:6] của Byte cao	00	P	00→ P 01→C 10→B 11 →U
#2 (Loại mã)	Bit [5:4] của Byte Cao	00	0	0→ Mã lỗi chung quốc tế (SAE/ISO) 1 / 2 / 3 → Mã lỗi riêng do hãng xe quy định
#3 (Hệ thống phụ)	Bit [3:0] của Byte Cao	0001	1	Đổi trực tiếp cụm 4 bit sang ký tự Hex (0 → F)
#4 (Mã lỗi cụ thể)	Bit [7:4] của Byte Thấp	0100	4	Tách nửa byte trên, đổi sang ký tự Hex (0 → F)
#5 (Mã lỗi cụ thể)	Bit [3:0] của Byte Thấp	0011	3	Tách nửa byte dưới, đổi sang ký tự Hex (0 → F)
KẾT QUẢ		0x01 + 0x43	P0143	

2.3.5.3 Quy trình đóng gói và truyền nhận Mode 09

Vì chuỗi mã số VIN quy chuẩn quốc tế luôn có độ dài cố định là 17 ký tự dạng mã ASCII, khi cộng thêm các byte quản lý tầng ứng dụng (mã phản hồi SID và mã xác nhận PID), tổng chiều dài chuỗi dữ liệu logic sẽ lên tới 20 byte. Do dung lượng này vượt xa giới hạn payload của một gói tin đơn SF, nên ở chiều phản hồi, ECU phương tiện bắt buộc phải sử dụng cơ chế truyền đa khung phân mảnh tầng mạng theo giao thức CAN TP ISO 15765-2.

Cấu trúc đóng gói khung yêu cầu dữ liệu:

Bảng 2.13. Định danh cấu trúc các byte dữ liệu trong khung yêu cầu Mode 09

Hướng khung tin	Thiết bị kiểm thử → ECU
ID	0x7DF

Loại khung tin		Yêu cầu	
Số thứ tự Byte	Trường dữ liệu	Giá trị (Hex)	Ý nghĩa
Byte 0	PCI	0x02	Khung đơn SF, thông báo độ dài dữ liệu kế tiếp là 2 byte
Byte 1	SID	0x09	Byte cao DTC#n
Byte 2	PID	0x02	Mã Parameter ID chỉ định truy xuất chuỗi mã số VIN xe
Byte 3 → Byte 7	Padding Byte	0x00 / 0xFF	Byte đệm

Cấu trúc đóng gói các khung phản hồi dữ liệu:

Bước 1: ECU phát khung đầu tiên FF

Bảng 2.14. Định danh cấu trúc khung đầu tiên FF chứa mã VIN từ ECU

Hướng khung tin		ECU → thiết bị kiểm thử	
ID		0x7E8	
Loại khung tin		Phản hồi đa khung - Khung đầu tiên FF	
Số thứ tự Byte	Trường dữ liệu	Giá trị (Hex)	Ý nghĩa
Byte 0	PCI Byte 1	0x10	Định danh Khung đầu tiên (FF)
Byte 1	PCI Byte 2	0x14	Tổng chiều dài chuỗi dữ liệu ứng dụng trả về là 20 byte
Byte 2	Phản hồi SID	0x49	Mã phản hồi Mode 09
Byte 3	Xác nhận PID	0x02	PID 02 (Mã VIN)
Byte 4	Data Items	0x01	Số lượng chuỗi báo cáo dữ liệu
Byte 5 → Byte 7	VIN Byte 0 → 2	0xXX	Ký tự ASCII thứ 1 → 3

Bước 2: Thiết bị kiểm thử phản hồi khung điều khiển luồng FC

Bảng 2.15. Định danh cấu trúc khung điều khiển luồng FC Mode 09

Hướng khung tin		Thiết bị kiểm thử → ECU	
ID		0x7E0	
Loại khung tin		Khung điều khiển luồng FC	
Số thứ tự Byte	Trường dữ liệu	Giá trị (Hex)	Ý nghĩa
Byte 0	PCI Byte	0x30	Định danh khung Flow Control, trạng thái Sẵn sàng nhận (CTS)
Byte 1	Block Size (BS)	0x00	Cho phép ECU gửi liên tục toàn bộ các mảnh dữ liệu còn lại
Byte 2	Separation_Time (ST_min)	0xXX	Yêu cầu ECU gửi các gói nối tiếp với tốc độ là XX

Byte 3 → Byte 7	Padding Byte	0x00 / 0xFF	Byte đệm

Bước 3: ECU truyền tải các khung nối tiếp CF

Định danh cấu trúc khung nối tiếp số CF1

Bảng 2.16. Định danh cấu trúc khung nối tiếp số CF1 Mode 09

Hướng khung tin		ECU → thiết bị kiểm thử	
ID		0x7E8	
Loại khung tin		Khung nối tiếp CF1	
Số thứ tự Byte	Trường dữ liệu	Giá trị (Hex)	Ý nghĩa
Byte 0	PCI Byte	0x21	Định danh khung nối tiếp (CF), số thứ tự gói số 1
Byte 1 → Byte 7	VIN Byte 3 → 9	0xFF	Ký tự ASCII thứ 4 → 10

Bảng 2.17. Định danh cấu trúc khung nối tiếp số CF2 Mode 09

Hướng khung tin		ECU → thiết bị kiểm thử	
ID		0x7E8	
Loại khung tin		Khung nối tiếp CF2	
Số thứ tự Byte	Trường dữ liệu	Giá trị (Hex)	Ý nghĩa
Byte 0	PCI Byte	0x22	Định danh khung nối tiếp (CF), số thứ tự gói số 2
Byte 1 → Byte 7	VIN Byte 10 → 16	0xFF	Ký tự ASCII thứ 11 → 17

2.4 Chuẩn giao tiếp SPI

2.4.1 Tổng quan về SPI

Serial Peripheral Interface (SPI) là chuẩn giao tiếp nối tiếp đồng bộ, cho phép truyền dữ liệu ở tốc độ cao giữa một thiết bị Master và một hoặc nhiều thiết bị Slave.

SPI hỗ trợ chế độ truyền full-duplex, cho phép dữ liệu được gửi và nhận đồng thời trong cùng một chu kỳ truyền. Nhờ đó, SPI có thể đạt tốc độ truyền cao và đáp ứng tốt các ứng dụng yêu cầu thời gian thực.

2.4.2 Cấu trúc tín hiệu trong

SPI sử dụng cấu hình tối thiểu là 4 đường dây tín hiệu [12]:

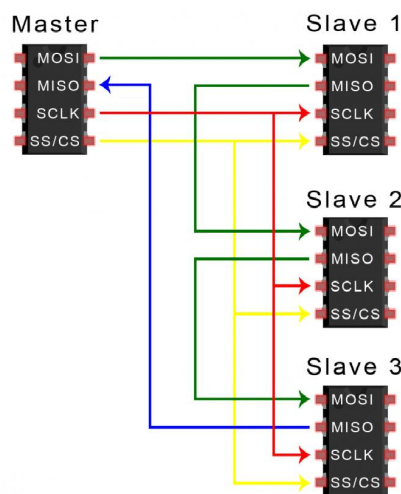
- SCLK (Serial Clock): Xung giữ nhịp đồng bộ do Master tạo ra.
- MOSI (Master Out Slave In): Đường truyền dữ liệu từ Master đến Slave.
- MISO (Master In Slave Out): Đường truyền dữ liệu từ Slave về Master.

- SS / CS (Slave Select / Chip Select): Tín hiệu chọn Slave cần giao tiếp (tích cực ở mức thấp 0).

Các tín hiệu này giúp đảm bảo việc truyền dữ liệu diễn ra đồng bộ và có kiểm soát giữa các thiết bị trong hệ thống.

2.4.3 Nguyên lý hoạt động của SPI

SPI hoạt động theo mô hình Master–Slave, trong đó Master giữ vai trò điều khiển toàn bộ quá trình giao tiếp. Master sẽ tạo xung clock và lựa chọn Slave cần giao tiếp thông qua tín hiệu CS.



Hình 2.15. Giao tiếp SPI giữa Master và nhiều Slave

Trong quá trình truyền:

- Master kéo CS xuống mức thấp để chọn Slave
- Master tạo xung SCLK để đồng bộ dữ liệu
- Dữ liệu được truyền từ Master đến Slave qua MOSI
- Đồng thời, Slave gửi dữ liệu ngược lại qua MISO

Hình 2.15 mô tả các thiết bị Slave chia sẻ các đường tín hiệu chung như MOSI, MISO và SCLK, trong khi mỗi Slave được điều khiển bằng một đường Chip Select (CS) riêng biệt. Khi Master kích hoạt một đường CS ở mức thấp, chỉ Slave tương ứng mới tham gia giao tiếp, các Slave còn lại sẽ ở trạng thái không hoạt động.

2.5 Nền tảng phát triển ứng dụng Flutter

Flutter là một framework mã nguồn mở do Google phát triển, được sử dụng để xây dựng ứng dụng đa nền tảng trên các hệ điều hành như Android, iOS, Web và

Desktop. Flutter cho phép lập trình viên phát triển ứng dụng với cùng một codebase, giúp giảm thời gian và chi phí phát triển.

Flutter sử dụng ngôn ngữ lập trình Dart và cung cấp một hệ thống giao diện người dùng mạnh mẽ dựa trên cơ chế widget, giúp xây dựng giao diện linh hoạt và dễ mở rộng.

Flutter được xây dựng theo kiến trúc nhiều tầng, bao gồm:

- Framework layer: chứa các widget và thư viện để xây dựng giao diện
- Engine layer: xử lý rendering, animation và giao tiếp với hệ điều hành
- Embedder: giúp Flutter chạy trên từng nền tảng cụ thể

Kiến trúc này cho phép Flutter kiểm soát trực tiếp việc hiển thị giao diện mà không phụ thuộc vào các thành phần UI native của hệ điều hành, từ đó đảm bảo tính nhất quán giữa các nền tảng.

Trong phạm vi đồ án, Flutter được sử dụng để xây dựng giao diện ứng dụng phía người dùng, cho phép hiển thị dữ liệu chẩn đoán nhận được (ví dụ: thông tin lỗi, trạng thái hệ thống, dữ liệu cảm biến...). Flutter đóng vai trò là lớp giao tiếp giữa người dùng và hệ thống bên dưới, giúp trực quan hóa dữ liệu và nâng cao trải nghiệm sử dụng.

2.6 Hệ điều hành thời gian thực (RTOS)

2.6.1 Tổng quan về RTOS

RTOS (Real-Time Operating System) là hệ điều hành được thiết kế để xử lý các tác vụ trong hệ thống với yêu cầu về thời gian chặt chẽ. Trong các hệ thống nhúng và automotive, RTOS đóng vai trò đảm bảo các tác vụ được thực thi đúng thời điểm và trong khoảng thời gian xác định, đặc biệt quan trọng đối với các ứng dụng liên quan đến điều khiển và chẩn đoán.

Khác với các hệ điều hành thông thường, RTOS không chỉ quan tâm đến việc hoàn thành tác vụ mà còn phải đảm bảo thời gian phản hồi ổn định và có thể dự đoán được.

2.6.2 Kiến trúc cơ bản của RTOS

RTOS bao gồm một số thành phần chính:

- Kernel: lõi hệ điều hành, chịu trách nhiệm quản lý tài nguyên và lập lịch
- Task/Thread: các đơn vị thực thi trong hệ thống

- Scheduler: bộ lập lịch quyết định task nào sẽ được chạy
- Interrupt handler: xử lý các sự kiện ngắt từ phần cứng
- Inter-Process Communication (IPC): cơ chế giao tiếp giữa các task

Các thành phần này phối hợp với nhau để đảm bảo hệ thống hoạt động ổn định và đáp ứng các yêu cầu thời gian thực.

2.6.3 Nguyên lý hoạt động của

RTOS hoạt động dựa trên cơ chế lập lịch (scheduling) để quản lý nhiều task trong hệ thống. Mỗi task có thể được gán một mức ưu tiên khác nhau, và scheduler sẽ quyết định task nào được thực thi tại từng thời điểm.

Trong quá trình hoạt động:

- RTOS liên tục theo dõi trạng thái của các task
- Task có độ ưu tiên cao hơn sẽ được thực thi trước
- Khi một sự kiện xảy ra (ví dụ interrupt), hệ điều hành có thể chuyển sang task khác ngay lập tức
- Sau khi hoàn thành, hệ thống quay lại xử lý các task còn lại

2.6.4 Các cơ chế chính trong RTOS

Lập lịch ưu tiên (Priority Scheduling): RTOS thường sử dụng cơ chế lập lịch dựa trên độ ưu tiên, trong đó task có mức ưu tiên cao hơn sẽ được xử lý trước.

Điều khiển ngắt (Interrupt Handling): RTOS hỗ trợ xử lý ngắt nhanh để phản ứng với các sự kiện từ phần cứng, từ đó đảm bảo hệ thống có thể phản hồi tức thời.

Đồng bộ và giao tiếp giữa các task: RTOS cung cấp các cơ chế hỗ trợ giao tiếp và đồng bộ dữ liệu giữa các task, gồm:

- Semaphore: điều khiển truy cập tài nguyên
- Mutex: tránh xung đột khi dùng chung tài nguyên
- Queue: truyền dữ liệu giữa các task

CHƯƠNG 3

THIẾT KẾ VÀ THI CÔNG HỆ THỐNG

3.1 Đặc tả hệ thống

Dựa trên nhu cầu thực tiễn của đề tài, các yêu cầu của hệ thống được phân chia thành bốn nhóm chính.

3.1.1 Yêu cầu chức năng

Đây là các yêu cầu mô tả những tác vụ cốt lõi mà hệ thống bắt buộc phải thực hiện được:

- Đối với thiết bị thu thập dữ liệu qua OBD-II

Thiết lập giao tiếp thành công với mạng CAN của ô tô thông qua cổng vật lý OBD-II

Khởi tạo khung yêu cầu và xử lý dữ liệu phản hồi để truy xuất thông số vận hành động cơ theo thời gian thực (Mode 01).

Truy xuất chính xác các mã lỗi chẩn đoán DTC đã lưu trong ECU (Mode 03) và các mã lỗi đang chờ xử lý (Mode 07).

Thu thập thành công mã định danh phương tiện VIN (Mode 09).

Đọc được các mã lỗi vĩnh viễn không thể xóa bằng phương pháp thông thường (Mode 0A).

- Đối với hệ thống mô phỏng ECU

Có khả năng đóng vai trò như một ECU thực tế, phản hồi chính xác các gói tin CAN khi nhận được lệnh truy vấn từ thiết bị chẩn đoán theo các Mode đã quy định.

- Đối với ứng dụng di động

Cung cấp cơ chế quản lý tài khoản người dùng an toàn.

Tiếp nhận dữ liệu cấu trúc và trực quan hóa các thông số vận hành của xe thông qua giao diện đồ họa.

Cảnh báo trạng thái phương tiện và hiển thị danh sách các mã lỗi DTC cho người dùng.

3.1.2 Yêu cầu phi chức năng

Các yêu cầu này ràng buộc về mặt hiệu năng, độ ổn định và tiêu chuẩn hoạt động của hệ thống.

- Hiệu năng và độ trễ: Giao tiếp mạng CAN phải được cấu hình ở tốc độ truyền chuẩn 500 kbps để đảm bảo đồng bộ với hệ thống trên xe. Luồng dữ liệu hiển thị trên ứng dụng di động và màn hình cục bộ phải được cập nhật liên tục.
- Độ tin cậy và tính chính xác: Dữ liệu thu thập từ thiết bị chẩn đoán phải tuân thủ nghiêm ngặt định dạng của tiêu chuẩn ISO 15031-5 và SAE J1979. Kết quả truyền nhận phải được kiểm chứng và đối chiếu chính xác hoàn toàn với luồng dữ liệu thô phân tích qua thiết bị chuyên dụng Logic Analyzer.
- Khả năng hoạt động thực tế: Thiết bị phần cứng phải có khả năng hoạt động ổn định trong môi trường thực tế trên ô tô, chống nhiễu tín hiệu tốt.

3.1.3 Yêu cầu phần cứng

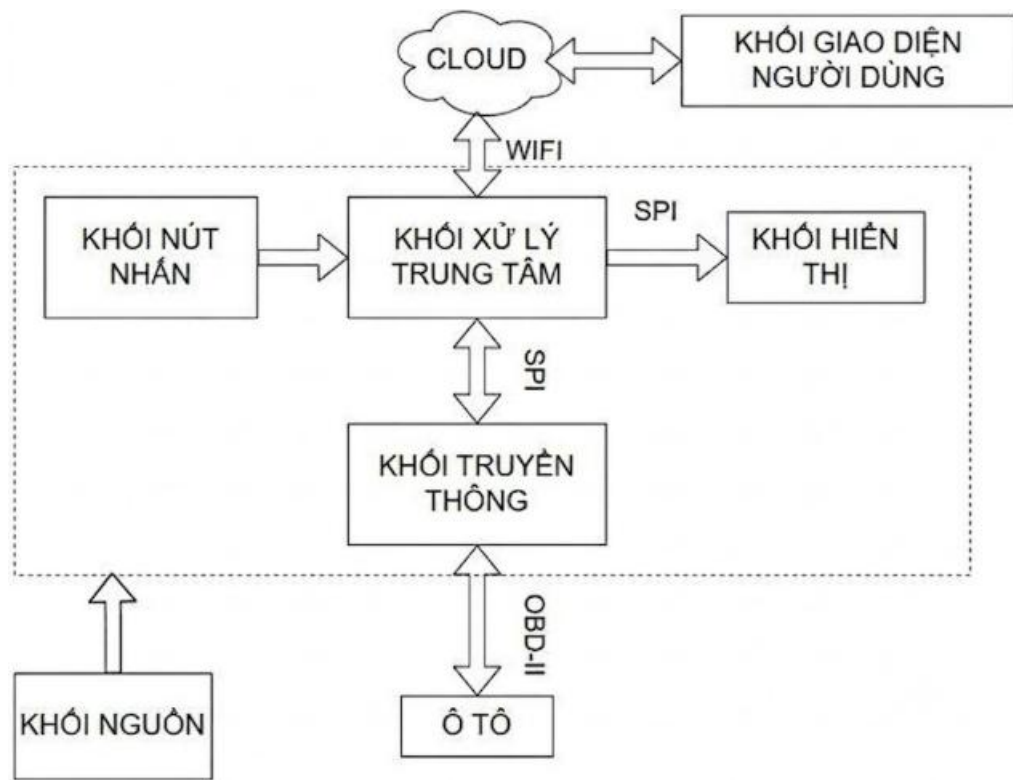
Để đáp ứng toàn diện các yêu cầu chức năng và phi chức năng, cấu trúc phần cứng của hệ thống được xác định dựa trên các tiêu chí kỹ thuật sau:

- Khối xử lý trung tâm: Sử dụng vi điều khiển cấu trúc 32-bit hiệu năng cao, sở hữu kiến trúc nhân kép hỗ trợ tính toán song song. Cấu trúc này bắt buộc phải đáp ứng khả năng xử lý đa tác vụ trong thời gian thực.
- Khối giao tiếp mạng CAN: Yêu cầu tích hợp bộ điều khiển CAN (CAN Controller) phần cứng, kết hợp cùng IC thu phát ngoại vi (CAN Transceiver). Khối này đóng vai trò là cầu nối vật lý, thực hiện nhiệm vụ chuyển đổi hai chiều giữa tín hiệu mức logic của vi điều khiển và tín hiệu điện áp vi sai CAN_H và CAN_L theo đúng tiêu chuẩn mạng chẩn đoán trên xe hơi.
- Khối hiển thị cục bộ: Tích hợp màn hình hiển thị TFT LCD sử dụng giao tiếp nối tiếp SPI đồng bộ tốc độ cao để trực quan hóa dữ liệu chẩn đoán và trạng thái hoạt động ngay tại thiết bị.

3.1.4 Yêu cầu phần mềm

- Firmware thiết bị nhúng: Sử dụng ngôn ngữ C/C++ để lập trình firmware cho vi điều khiển, đảm nhận các nhiệm vụ: khởi tạo kết nối CAN, giải mã dữ liệu OBD-II, điều khiển màn hình và giao tiếp với nền tảng di động.
- Phần mềm ứng dụng di động: Được xây dựng và phát triển dựa trên bộ công cụ Flutter và ngôn ngữ Dart.

3.2 Sơ đồ khối và chức năng của các khối



Hình 3.1. Sơ đồ khối hệ thống

Dựa trên sơ đồ khối hệ thống hình 3.1, chức năng của từng khối được quy định như sau:

- **Khối xử lý trung tâm:** Đóng vai trò như bộ não điều khiển toàn bộ hệ thống. Khối này chịu trách nhiệm tiếp nhận các tín hiệu tương tác từ khối nút nhấn, xử lý thuật toán chẩn đoán, đóng gói dữ liệu để điều khiển khối hiển thị cục bộ qua giao tiếp SPI, đồng thời phối hợp với khối truyền thông để điều khiển luồng dữ liệu OBD-II. Ngoài ra, khối này còn đảm nhiệm vai trò kết nối mạng không dây Wifi để đồng bộ dữ liệu thu thập được lên hệ thống Cloud.
- **Khối truyền thông:** Là cầu nối giao tiếp trực tiếp giữa thiết bị chẩn đoán và phương tiện ô tô. Khối này nhận các lệnh truy vấn chẩn đoán từ khối xử lý trung tâm thông qua giao tiếp nối tiếp SPI, thực hiện chuyển đổi mức tín hiệu và đóng gói theo đúng định dạng giao thức để giao tiếp với hộp ECU trên ô tô qua cổng vật lý OBD-II, sau đó nhận dữ liệu phản hồi trả về cho khối xử lý trung tâm.
- **Khối hiển thị:** Đảm nhận vai trò trực quan hóa dữ liệu chẩn đoán tại chỗ cho thiết bị. Khối này tiếp nhận dữ liệu xuất ra từ khối xử lý trung tâm qua đường

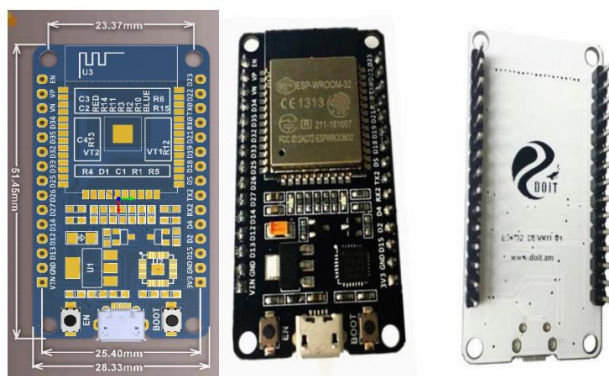
truyền SPI tốc độ cao để hiển thị các thông số động học, mã lỗi DTC và mã định danh phương tiện.

- **Khối nút nhấn:** Là khối tiếp nhận tương tác vật lý từ người dùng, đóng vai trò đầu vào điều khiển của thiết bị. Khi người dùng thao tác bấm nút (như chuyển trang dữ liệu, chọn chế độ đọc lỗi), khối này sẽ gửi tín hiệu kích hoạt về khối xử lý trung tâm để thực thi các lệnh tương ứng.
- **Khối nguồn:** Cung cấp điện năng cho toàn bộ hệ thống, đảm bảo sự hoạt động liên tục và ổn định của tất cả các khối chức năng.
- **Cloud:** Đóng vai trò là nền tảng lưu trữ và trung chuyển dữ liệu trung gian trên Internet. Khối này tiếp nhận luồng dữ liệu chẩn đoán được gửi lên từ khối xử lý trung tâm qua kết nối Wi-Fi, thực hiện lưu trữ dữ liệu và sẵn sàng cung cấp dữ liệu đồng bộ cho ứng dụng giám sát.
- **Khối giao diện người dùng:** Là ứng dụng di động kết nối với Cloud để trực quan hóa thông số xe, đồng thời cho phép người dùng trực tiếp gửi các yêu cầu thực thi các Mode chẩn đoán từ xa.

3.3 Phân tích và thiết kế các khối phần cứng

3.3.1 Khối xử lý trung tâm

Khối xử lý trung tâm sử dụng vi điều khiển ESP32 DevKit V1 là một board phát triển phổ biến dựa trên chip ESP32 của Espressif Systems.



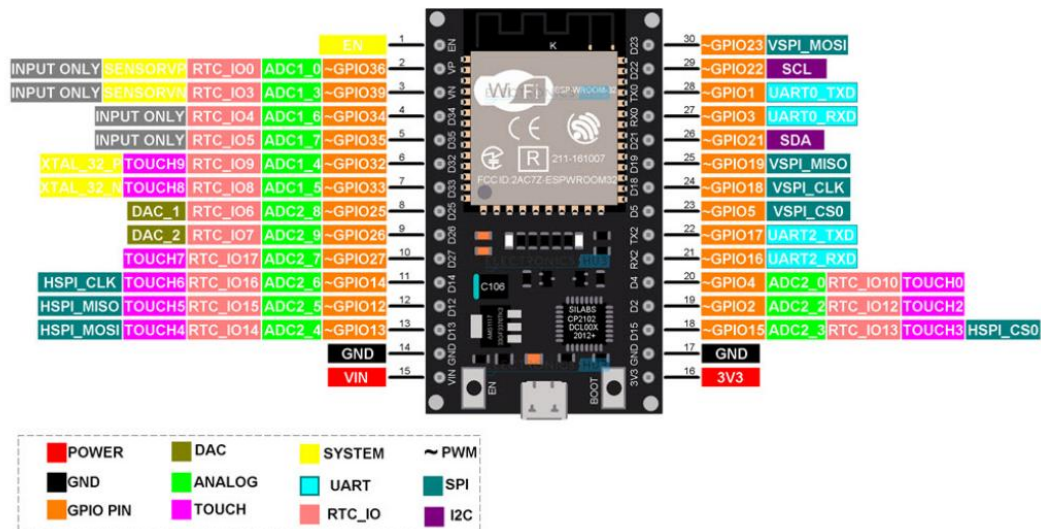
Hình 3.2. Module ESP32 devkit v1

Bảng 3.1. Thông số kỹ thuật module ESP32 devkit v1

Đặc tính	Thông số
Bộ nhớ	520KB SRAM
Wi-Fi	802.11 b/g/n (2.4GHz)
Bluetooth	v4.2 BR/EDR và BLE

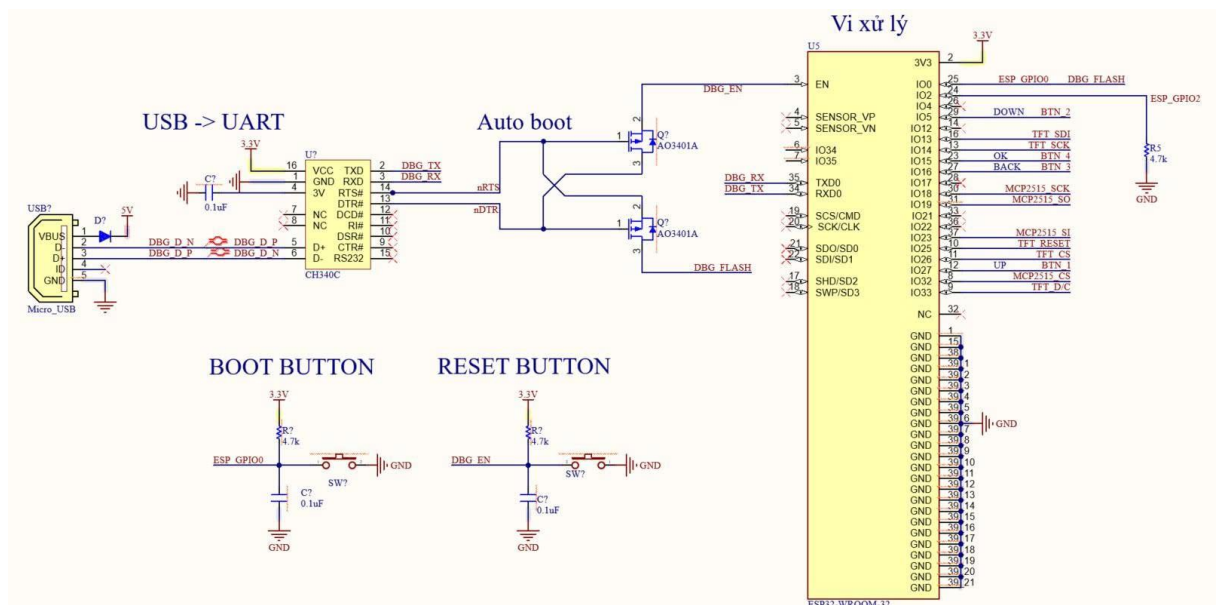
GPIO	30 chân I/O
ADC	12-bit, 18 kênh
DAC	8-bit, 2 kênh
Giao tiếp	SPI, I2C, I2S, UART, CAN, PWM
Điện áp hoạt động	3.3V

ESP32 DevKit V1 có tổng cộng 30 chân GPIO (General Purpose Input/Output) và các chân chức năng khác. Dưới đây là hình ảnh thẻ thông tin chi tiết về sơ đồ chân.



Hình 3.3. Sơ đồ chân module ESP32 devkit v1

Sơ đồ nguyên lý khối điều khiển trung tâm:



Hình 3.4. Sơ đồ nguyên lý khối điều khiển trung tâm

Khối xử lý trung tâm chia thành ba thành phần chính gồm: Khối chuyển đổi tín hiệu USB - UART, khối tự động nạp chương trình và khối vi xử lý sử dụng ESP32-WROOM-32E-N4.

- Khối chuyển đổi tín hiệu USB - UART: Khối này sử dụng IC CH340C với chức năng chuyển đổi tín hiệu USB từ máy tính sang giao tiếp UART để vi điều khiển ESP32 có thể tiếp nhận và xử lý dữ liệu. Bên cạnh đó, khối còn có chức năng debug chương trình thông qua cổng Serial Monitor.
- Khối tự động nạp sử dụng hai MOSFET kênh P AO3401A để điều khiển các chân EN, IO0 và IO2 của ESP32 thông qua hai tín hiệu nRTS và nDTR từ IC CH340C. Cơ chế này cho phép tự động đưa ESP32 vào chế độ UART Download Mode khi người dùng thực hiện lệnh nạp chương trình trên IDE, cụ thể là Arduino IDE. Nhờ đó, quá trình nạp chương trình được thực hiện hoàn toàn tự động mà không cần thao tác thủ công trên các nút BOOT và RESET. Bảng trạng thái điều khiển của khối tự động nạp được trình bày trong hình 3.5.

Auto program				
DTR	RTS-->	EN	IO0	IO2
1	1	1	1	1
0	0	1	1	1
1	0	0	1	1
0	1	1	0	0

Hình 3.5. Bảng trạng thái điều khiển chân EN, IO0 và IO2 từ tín hiệu DTR, RTS

- Khối vi xử lý sử dụng chip ESP32-WROOM-32E-N4 được thiết kế dưới dạng linh kiện dán nhằm tối ưu diện tích bo mạch và vẫn sử dụng đầy đủ các chân GPIO của vi điều khiển. ESP32 đóng vai trò là bộ điều khiển trung tâm của hệ thống, chịu trách nhiệm xử lý dữ liệu, thực hiện giao tiếp với các ngoại vi, điều khiển quá trình truyền nhận dữ liệu trên mạng CAN và điều phối hoạt động của toàn bộ hệ thống.

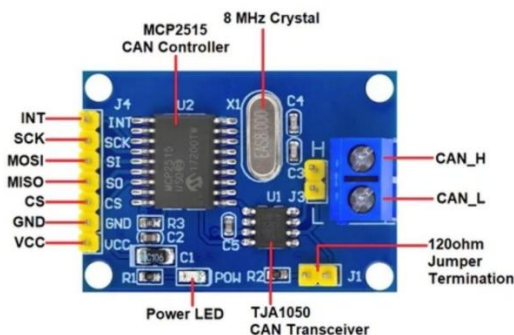
3.3.2 Khối truyền thông

Module CAN Bus MCP2515 với TJA1050 Transceiver là một giao tiếp truyền thông thiết yếu cho các dự án cần tương tác với mạng điều khiển ô tô hoặc công nghiệp. Bo mạch nhỏ gọn với kích thước 40x28x11 mm nhưng mạnh mẽ này tích hợp

bộ điều khiển CAN Microchip MCP2515 và bộ thu phát CAN tốc độ cao NXP TJA1050, mang lại một giải pháp hoàn chỉnh, cách ly để kết nối vi điều khiển với mạng CAN. MCP2515 xử lý ngăn xếp giao thức CAN phức tạp, quản lý việc lọc và truyền bản tin, trong khi TJA1050 đảm bảo truyền thông tin cậy và mạnh mẽ qua bus hai dây vật lý.

Các đặc điểm chính:

- Giải pháp CAN tích hợp: Kết hợp bộ điều khiển CAN MCP2515 và bộ thu phát CAN tốc độ cao TJA1050 trên một bo mạch đơn, nhỏ gọn.
- Giao tiếp SPI: Sử dụng giao tiếp ngoại vi nối tiếp SPI phổ biến và nhanh chóng để dễ dàng truyền thông với hầu hết các vi điều khiển.
- Truyền thông tốc độ cao: Hỗ trợ tốc độ truyền thông CAN chuẩn lên đến 1 Mb/s.
- Bộ đệm bản tin: Có ba bộ đệm truyền và hai bộ đệm nhận, cho phép module xếp hàng các bản tin một cách hiệu quả.
- Jumper điện trở đầu cuối: Bao gồm một điện trở kết thúc 120-ohm trên bo mạch, có thể dễ dàng bật hoặc tắt bằng cách sử dụng một jumper để phù hợp với cấu trúc mạng.



Hình 3.6. Module MCP2515 CAN Bus

Bảng 3.2. Thông số kỹ thuật module MCP2515 CAN Bus

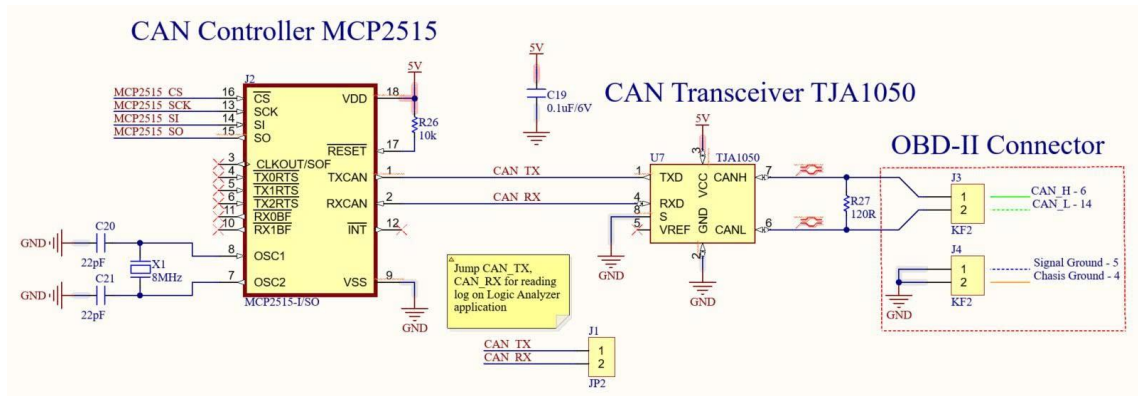
Đặc tính	Thông số
CAN Controller IC	Microchip MCP2515
CAN Transceiver IC	NXP TJA1050
Giao thức truyền thông	CAN V2.0B
Tốc độ dữ liệu	Lên đến 1 Mb/s
Điện áp bộ điều khiển	DC 5V
Điện áp bộ thu phát	DC 12V
Giao tiếp	SPI

Sơ đồ chân và đấu dây (Pinout and Wiring)

- VCC: Đầu vào nguồn cấp (5V DC) cho khối logic của module
- GND: Kết nối nối đất (Ground connection)
- CS: Chip select, kết nối với bất kỳ chân I/O số nào trên vi điều khiển
- SO (MISO): Master In Slave Out
- SI (MOSI): Master Out Slave In
- SCK: SPI Clock
- CAN_H: Kết nối với đường cao của CAN bus.
- CAN_L: Kết nối với đường thấp của CAN bus.

Lưu ý đấu: Kết nối VCC với nguồn cấp 5V của vi điều khiển. Đảm bảo kết nối các đường CAN_H và CAN_L với các dây tương ứng trong mạng CAN. Nếu module nằm ở một trong hai đầu của mạng CAN bus, hãy bật jumper điện trở kết thúc 120-ohm.

Sơ đồ nguyên lý khối truyền thông:



Hình 3.7. Sơ đồ nguyên lý khối truyền thông

Khối truyền thông CAN bao gồm hai thành phần chính là CAN Controller MCP2515 và CAN Transceiver TJA1050. Khối này đóng vai trò trung gian giữa vi xử lý ESP32 và mạng CAN Bus, thực hiện chức năng truyền nhận dữ liệu theo chuẩn CAN.

- MCP2515 (CAN Controller): MCP2515 được sử dụng để triển khai giao thức CAN và giao tiếp với vi xử lý ESP32 thông qua chuẩn SPI. Các chân kết nối gồm: chân CS nối với GPIO32, chân SCK nối với GPIO18, chân SI (MOSI) nối

với GPIO23 và chân SO (MISO) nối với GPIO19. IC MCP2515 chịu trách nhiệm cho việc đóng gói, xử lý và chuyển đổi dữ liệu từ giao thức SPI thành các tín hiệu Tx và Rx.

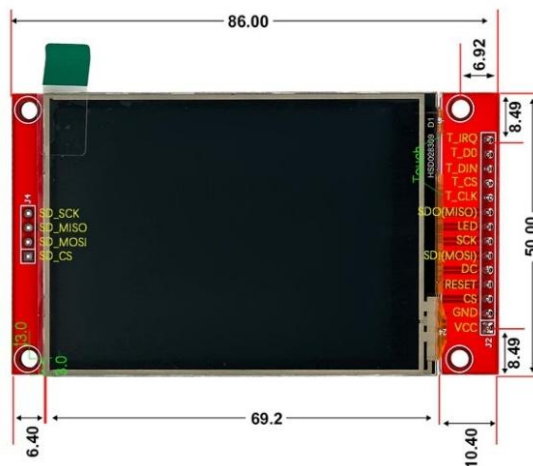
- TJA1050 (CAN Transceiver): TJA1050 được kết nối trực tiếp với các chân Tx và Rx của MCP2515. Nhiệm vụ của IC này là chuyển đổi tín hiệu logic mức TTL từ bộ điều khiển CAN thành tín hiệu vi sai trên hai đường CAN_H và CAN_L để truyền lên mạng CAN Bus, đồng thời thực hiện quá trình nhận và chuyển đổi tín hiệu vi sai từ mạng CAN Bus về dạng tín hiệu logic để bộ điều khiển CAN xử lý.

Ngoài ra, một đầu nối phụ cũng được bố trí trên các tín hiệu Tx và Rx để phục vụ quá trình kiểm tra và phân tích tín hiệu bằng Logic Analyzer trong giai đoạn phát triển và kiểm thử hệ thống.

3.3.3 Khôi hiển thị

Màn hình cảm ứng TFT 2.8 inch SPI ILI9341 là một giải pháp hiển thị sống động và linh hoạt, lý tưởng cho các hệ thống nhúng yêu cầu giao diện đồ họa người dùng (GUI) phong phú. Sử dụng IC ILI9341 phổ biến, cung cấp độ phân giải 240x320 pixels với khả năng hiển thị lên đến 262K màu. Thiết bị được tích hợp tám nền cảm ứng điện trở điều khiển bởi IC XPT2046, cho phép tiếp nhận các tương tác đầu vào từ người dùng.

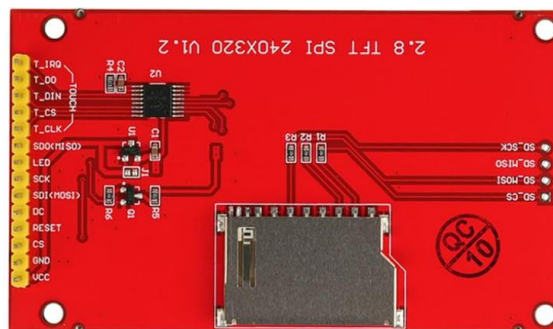
Bo mạch còn tích hợp sẵn một khe cắm thẻ nhớ SD ở mặt sau, thuận tiện cho việc lưu trữ các tệp đồ họa bitmap, phông chữ chữ và nhật ký dữ liệu từ bên ngoài.



Hình 3.8. Màn hình cảm ứng TFT 2.8 inch SPI ILI934

Bảng 3. 3 Thông số kỹ thuật màn hình cảm ứng TFT SPI ILI9341

Đặc tính	Thông số
IC điều khiển hiển thị	ILI9341
IC điều khiển cảm ứng	XPT2046
Loại màn hình cảm ứng	Cảm ứng điện trở
Điện áp nguồn VCC	3.3V ~ 5V
Điện áp cổng I/O logic	3.3V (TTL)



Hình 3.9. Mặt sau và sơ đồ ký hiệu chân cắm thực tế của module màn hình TFT 2.8" SPI

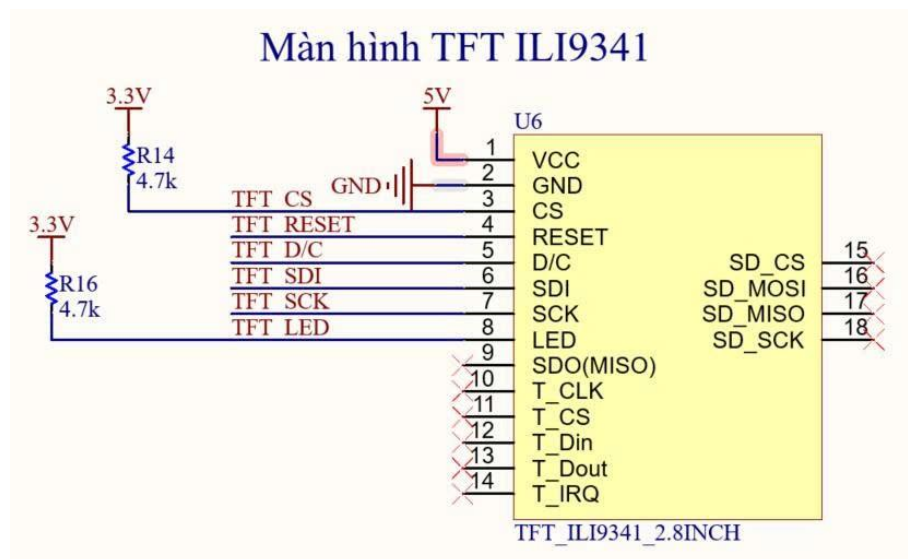
Bảng 3.4. Mô tả chức năng các chân cắm của module màn hình

Tên Chân	Chức năng
VCC	Đầu vào nguồn cấp điện cho module
GND	Chân nối mass chung
CS	Chip Select, kích hoạt ở mức thấp '0'
RESET	Tín hiệu khởi động lại màn hình LCD, kích hoạt ở mức thấp '0'
DC (RS)	Tín hiệu chọn Dữ liệu ở mức cao '1' hoặc lệnh ở mức thấp '0'

SDI (MOSI)	Đường truyền dữ liệu từ vi điều khiển đến màn hình trên bus SPI
SCK	Tín hiệu xung nhịp giữ nhịp đồng bộ của bus SPI.
LED	Mạch định kết nối nguồn 3.3V để luôn sáng.
SDO (MISO)	Đường xuất dữ liệu từ màn hình về vi điều khiển trên bus SPI
T_CLK	Tín hiệu xung nhịp đồng bộ bus SPI dành riêng cho khối cảm ứng.
T_CS	Tín hiệu chọn chip của khối cảm ứng, kích hoạt ở mức thấp '0'
T_DIN	Đường vào dữ liệu SPI của khối cảm ứng
T_DO	Đường ra dữ liệu SPI của khối cảm ứng
T_IRQ	Tín hiệu phát ngắt phần cứng của màn hình cảm ứng, tự động kéo xuống mức thấp khi phát hiện có thao tác chạm.

Việc kết nối và đấu dây cho nhóm tín hiệu điều khiển cảm ứng (từ chân số 10 đến chân số 14) chỉ cần thiết khi ứng dụng yêu cầu chức năng tương tác chạm. Trong trường hợp hệ thống không sử dụng tính năng cảm ứng, các chân này có thể để trống để tiết kiệm tài nguyên chân I/O của vi điều khiển trung tâm.

Sơ đồ nguyên lý khối hiển thị:



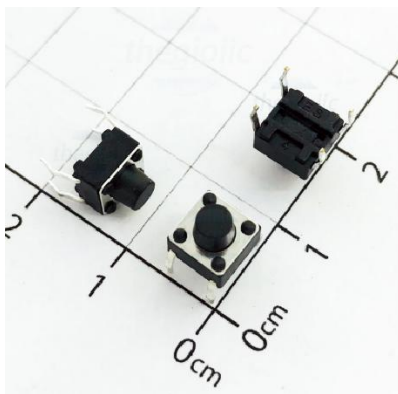
Hình 3.10. Sơ đồ nguyên lý khối hiển thị

Màn hình TFT được kết nối với vi xử lý ESP32 thông qua giao thức SPI nhằm phục vụ chức năng hiển thị thông tin. Các chân kết nối bao gồm: chân CS nối với GPIO26, chân RESET nối với GPIO25, chân D/C nối với GPIO33, chân SDI nối với GPIO13 và chân SCK nối với GPIO14. Thông qua giao thức SPI, ESP32 thực hiện việc truyền dữ liệu và các lệnh điều khiển đến màn hình TFT để hiển thị nội dung theo

yêu cầu của ứng dụng.. Điện trở được sử dụng làm điện trở kéo lên cho chân CS của màn hình TFT nhằm tránh hiện tượng floating trong quá trình khởi động hệ thống. Khi CS ở mức logic cao, màn hình TFT không được chọn trên bus SPI. Đồng thời, điện trở này còn giúp giới hạn dòng điện khi vi điều khiển kéo chân CS xuống mức thấp để thực hiện giao tiếp dữ liệu với màn hình.

3.3.4 Khối nút nhấn

Khối nút nhấn được thiết kế để người dùng tương tác trực tiếp với thiết bị thông qua màn hình trên khối hiển thị. Hệ thống sử dụng nút nhấn dạng Tact Switch 6x6mm cao 6mm, 4 chân xuyên lỗ, có kích thước nhỏ gọn, độ bền cao và dễ dàng tích hợp trên mạch PCB. Nút nhấn hoạt động theo nguyên lý nhấn để đóng mạch và nhả để ngắt mạch.



Hình 3.11. Nút nhấn 6x6mm cao 6mm 4 chân xuyên lỗ

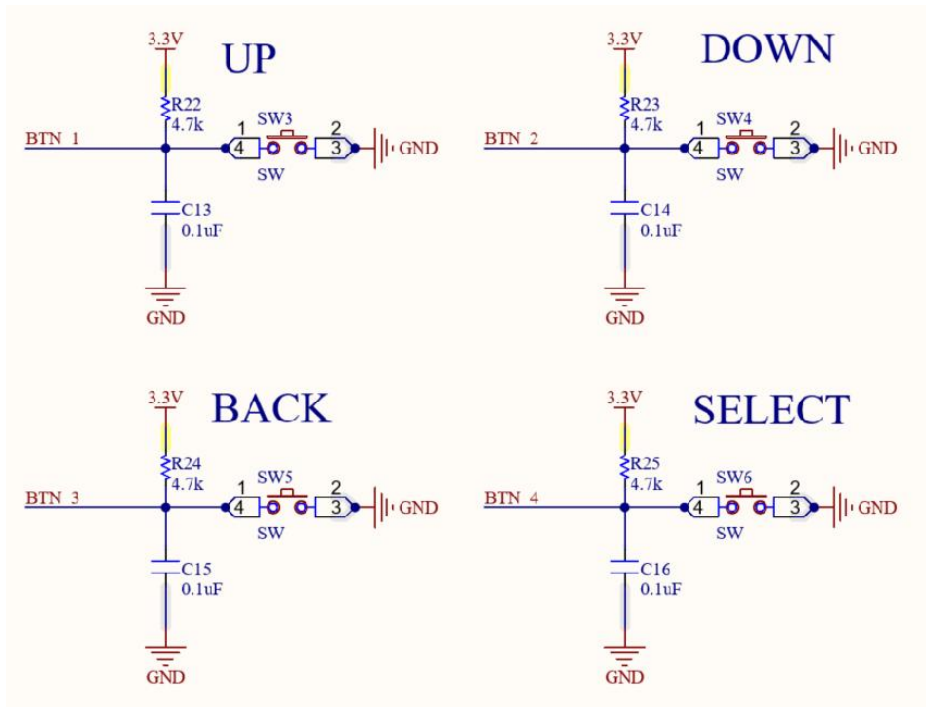
Bảng 3.5. Thông số kỹ thuật nút nhấn

Đặc tính	Thông số
Loại nút nhấn	Tact Switch
Số chân	4
Kiểu chân	Xuyên lỗ
Hoạt động	Nhấn nhả
Kích thước	6x6x6mm
Dòng và áp định mức	50mA và 12V DC
Nhiệt độ hoạt động	-35 -> 85 °C
Tuổi thọ	100000 lần

Sơ đồ nguyên lý khối nút nhấn

Khối nút nhấn bao gồm 4 chức năng chính (UP, DOWN, BACK, SELECT) tương ứng với các đường tín hiệu từ BTN1 đến BTN4 như hình 3. Để phục vụ quá trình điều hướng và điều khiển giao diện

- Nút Up (BTN1 - GPIO 27): Di chuyển lên các mục phía trên trong Menu.
- Nút Down (BTN2 - GPIO 5): Di chuyển xuống các mục phía dưới trong Menu.
- Nút Back (BTN3 - GPIO 15): Thoát chế độ đọc và quay về giao diện Menu chính, ngoài ra để phục vụ cho việc đồng bộ trên ứng dụng di động, nút BACK còn được sử dụng như cách để chiếm quyền điều khiển thiết bị vật lý.
- Nút Select (BTN4 - GPIO 4): Lựa chọn và truy cập vào chế độ chẩn đoán muốn đọc.



Hình 3.12. Sơ đồ nguyên lý khối nút nhấn

Các nút nhấn được thiết kế theo cấu hình pull-up. Ở trạng thái bình thường khi không nhấn nút, chân tín hiệu được điện trở 4,7 kΩ kéo lên mức logic cao. Khi nhấn nút, chân tín hiệu được nối thẳng xuống GND, tạo mức logic thấp để vi điều khiển nhận biết trạng thái nhấn. Điện trở 4,7 kΩ đóng vai trò điện trở pull-up, giúp tránh trạng thái trôi nổi của tín hiệu cũng như hạn dòng khi nhấn nút. Ngoài ra, tụ điện 0,1 μF được mắc song song với nút nhấn đóng vai trò như là một bộ lọc phần cứng, giúp giảm hiện tượng rung phím cơ khí và lọc nhiễu tần số cao, đảm bảo tín hiệu đầu vào luôn ổn định và chính xác hơn.

3.3.5 Khối nguồn

Khối nguồn đóng vai trò cung cấp năng lượng cho toàn bộ hệ thống và ảnh hưởng trực tiếp đến độ ổn định của thiết bị. Trong hệ thống sử dụng pin Lithium,

nguồn điện thường trải qua nhiều giai đoạn xử lý nhằm đảm bảo điện áp cung cấp phù hợp với yêu cầu của từng khối chức năng.

Pin 18650 là một loại pin Lithium, có thể sạc được nhiều lần. Pin 18650 cung cấp một công suất khoảng 1800mAh đến 3500mAh tùy từng loại pin và điện áp khoảng 3.7V – 4.2V. Pin 18650 được sử dụng rất rộng rãi trong cuộc sống từ máy tính xách tay, đèn pin, sạc pin dự phòng, ...



Hình 3.13. Sơ đồ nguyên lý khối nút nhấn

Hệ thống sử dụng mạch sạc sử dụng IC TP4056, hoạt động theo thuật toán sạc CC/CV (Constant Current/Constant Voltage) gồm ba giai đoạn chính như sau:

- Giai đoạn 1 (Trickle Charge): Khi điện áp của pin thấp hơn khoảng 2.9V, TP4056 sẽ giới hạn dòng sạc ở mức khoảng 10% của dòng sạc cài đặt để bảo vệ pin khỏi hư hỏng do dòng sạc lớn.
- Giai đoạn 2 (Constant Current Phase): Khi vượt quá ngưỡng sạc nhỏ giọt, bộ sạc sẽ cung cấp dòng sạc tối đa được cài đặt. Trong giai đoạn này, điện áp tăng đều đặn.
- Giai đoạn 3 (Constant Voltage Phase): Khi điện áp đạt ngưỡng quy định, nó sẽ duy trì ổn áp và giảm dần dòng sạc cho đến khi pin bão hoà.

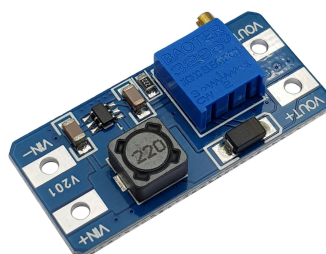


Hình 3.14. Module sạc pin TP4056

Bảng 3. 6 Thông số kỹ thuật module sạc pin TP4056

Đặc tính	Thông số
Nguồn đầu vào	4.5~8VC: Nguồn vào +/- 5VDC qua cổng USB-C
Điện áp sạc đầy	4.2VDC
Dòng sạc tối đa	1A
Trạng thái đèn sạc	Đèn đỏ: Đang sạc, chưa đầy Đèn xanh: Đã sạc đầy
Pin điện áp bảo vệ quá tải	4.28V
Điện áp bảo vệ xả pin	3V
Điện áp ngưng xả pin	3.2V
Bảo vệ quá dòng	3A

Do điện áp đầu ra của pin Lithium 18650 chỉ nằm trong khoảng 3.0 V đến 4.2 V nên hệ thống sử dụng thêm mạch tăng áp để nâng điện áp lên mức 5 V nhằm đáp ứng yêu cầu cấp nguồn cho các khối chức năng khác. Mạch tăng áp sử dụng IC MT3608, hoạt động theo nguyên lý tích trữ năng lượng trong cuộn cảm và giải phóng năng lượng thông qua quá trình đóng cắt tốc độ cao, từ đó tạo ra điện áp đầu ra lớn hơn điện áp đầu vào. Có một lưu ý là mạch sẽ bị cháy nếu điện áp đầu vào lớn hơn điện áp đầu ra.



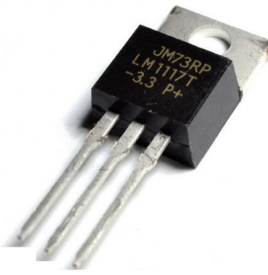
Hình 3.15. Module tăng áp MT3608

Bảng 3.7. Thông số kỹ thuật module tăng áp MT3608

Đặc tính	Thông số
Tần số	1.2Mhz
Điện áp đầu vào	2~24VDC
Điện áp đầu ra	Max 28VDC
Công suất tải	8W
Dòng đáp ứng	2A

Tương tự như mạch tăng áp, một số khối chức năng trong hệ thống chỉ hoạt động ở mức điện áp 3.3 V. Do đó, hệ thống sử dụng IC ổn áp LM1117-3.3 để chuyển

đổi điện áp 5 V từ mạch tăng áp xuống mức điện áp 3.3 V ổn định, cung cấp nguồn cho các linh kiện yêu cầu điện áp hoạt động 3.3 V.



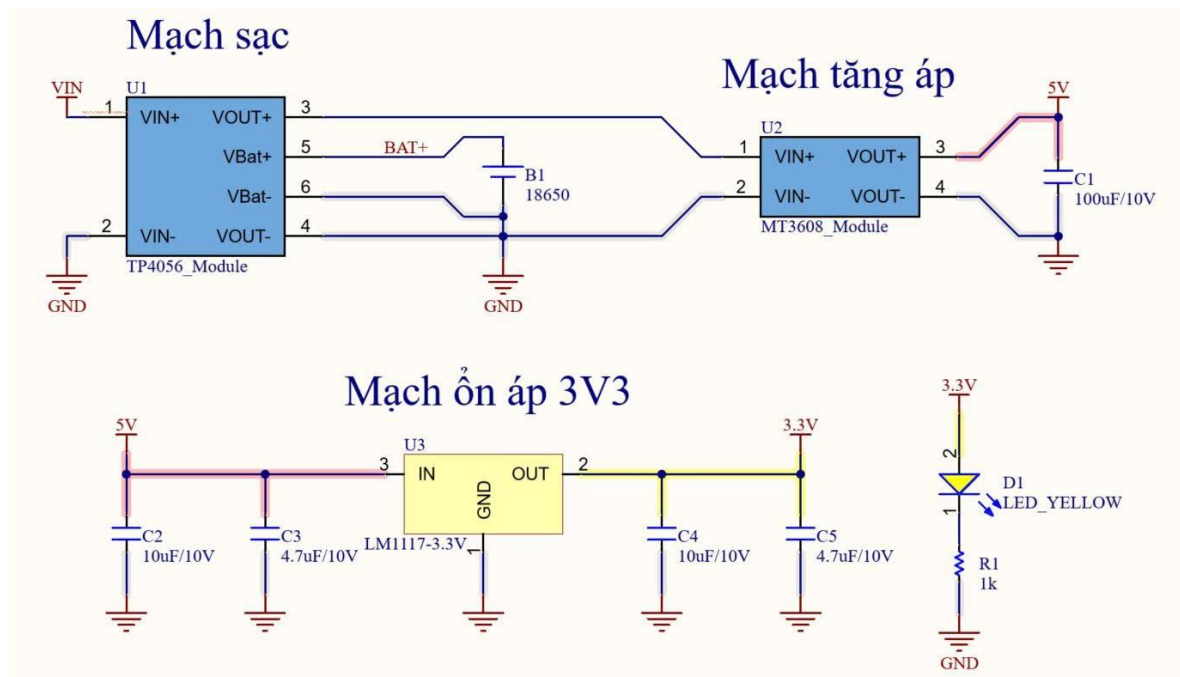
Hình 3.16. IC ổn áp LM 1117 3.3V

Bảng 3.8. Thông số kỹ thuật IC ổn áp LM1117 3.3V

Đặc tính	Thông số
Điện áp ngõ ra	3.3V
Dòng điện ngõ ra	800mA
Điện áp ngõ vào Max	15V
Loại ổn áp	Linear voltage
Phân cực âm dương	Điện áp dương
Loại ngõ ra	Cố định
Số ngõ ra	1
Nhiệt độ hoạt động	0°C ~ 125°C

Sơ đồ nguyên lý khối nguồn:

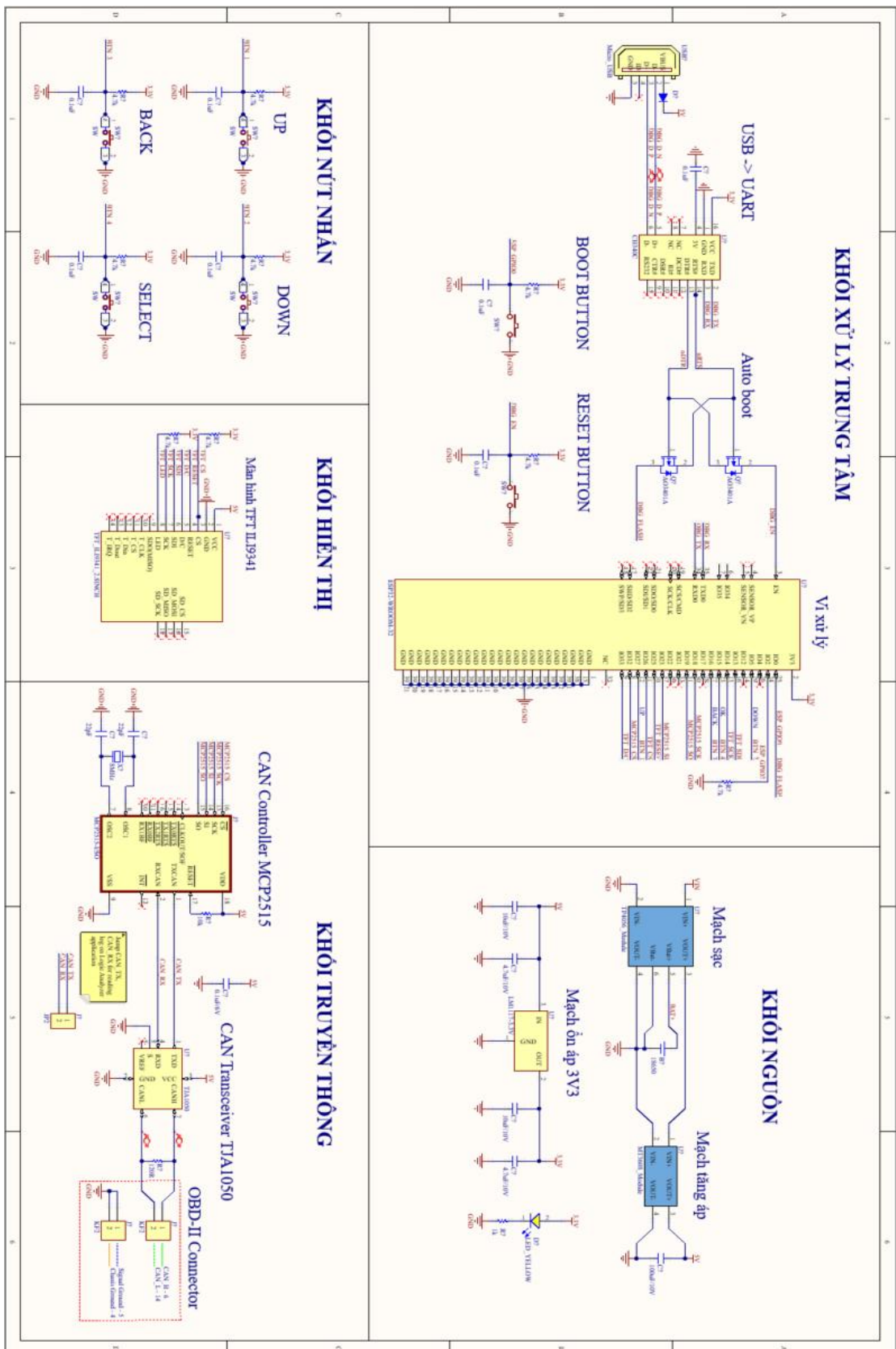
Hình 3.17 mô tả chi tiết khối nguồn có nhiệm vụ cung cấp các mức điện áp ổn định cho toàn bộ hệ thống, bao gồm 3 thành phần chính: Mạch sạc pin, mạch tăng áp và mạch ổn áp 3.3V.



Hình 3.17. Sơ đồ nguyên lý khối nguồn

- Mạch sạc pin: Sử dụng module TP4056 để thực hiện chức năng sạc cho pin Lithium 18650. Nguồn điện đầu vào được cấp vào chân VIN của TP4056, sau đó IC thực hiện quá trình sạc theo các giai đoạn để đảm bảo pin được sạc an toàn.
- Mạch tăng áp: Điện áp từ pin 18650 được đưa vào module tăng áp MT3608 để chuyển đổi từ mức điện áp pin lên mức điện áp ổn định 5 V. Mức điện áp 5 V này được sử dụng để cấp nguồn cho các khối chức năng yêu cầu điện áp hoạt động 5 V trong hệ thống. Tụ điện 100 µF được bố trí tại đầu ra nhằm giảm nhiễu và ổn định điện áp nguồn.
- Mạch ổn áp 3.3 V: Nguồn 5 V từ mạch tăng áp được đưa vào IC ổn áp LM1117-3.3 để tạo ra điện áp 3.3 V cung cấp cho các linh kiện ngoại vi hoạt động ở mức điện áp tương ứng. Theo khuyến nghị của nhà sản xuất, các tụ điện được bố trí tại đầu vào và đầu ra của IC nhằm giảm nhiễu, cải thiện độ ổn định điện áp và đảm bảo mạch hoạt động đáng tin cậy. Ngoài ra, LED kết hợp với điện trở hạn dòng được sử dụng làm đèn báo nguồn, giúp người dùng dễ dàng kiểm tra trạng thái cấp nguồn của hệ thống.

3.4 Sơ đồ nguyên lý toàn hệ thống



Hình 3.18. Sơ đồ nguyên lý toàn hệ thống

3.5 Thiết kế phần mềm và ứng dụng di động

3.5.1 Thiết kế cơ sở dữ liệu Firebase

Hệ thống sử dụng cơ sở dữ liệu Firebase Realtime Database làm nền tảng trung gian để trao đổi dữ liệu giữa thiết bị phân cứng và ứng dụng di động Android. Dữ liệu được lưu trữ theo cấu trúc JSON dạng cây, cho phép truy xuất và đồng bộ theo thời gian thực. Để thiết lập kết nối với Firebase Realtime Database, chương trình cần khai báo hai tham số quan trọng là FIREBASE_HOST và FIREBASE_AUTH. Trong đó, FIREBASE_HOST xác định địa chỉ của cơ sở dữ liệu trên nền tảng Firebase, còn FIREBASE_AUTH là mã xác thực dùng để cấp quyền truy cập và bảo mật quá trình trao đổi dữ liệu giữa thiết bị và cơ sở dữ liệu.



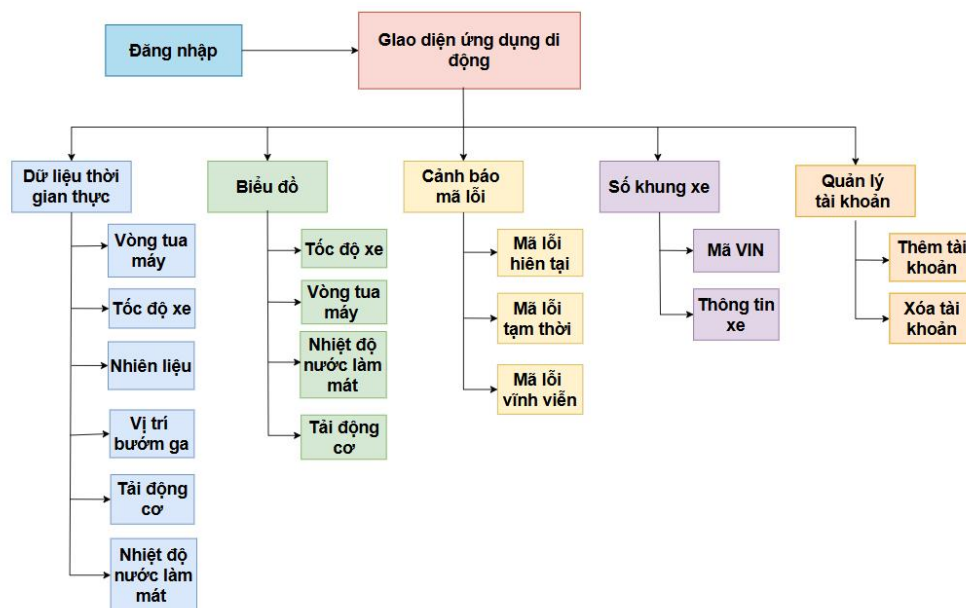
Hình 3.19. Cấu trúc cây dữ liệu thực tế trên Firebase

Cơ sở dữ liệu Firebase được phân chia làm ba nhánh chức năng chính bao gồm: Nhánh /control quản lý quyền tương tác hai chiều thông qua chế độ điều khiển (từ ứng

dụng hoặc từ thiết bị phần cứng) và chế độ yêu cầu giữa ứng dụng di động và thiết bị. Nhánh /data lưu trữ các thông số vận hành thời gian thực của xe do thiết bị ESP32 truyền lên như nhiên liệu, tải động cơ, vòng tua, tốc độ, nhiệt độ nước làm mát, góc mở bướm ga, cùng mã VIN và danh sách mã lỗi động cơ. Nhánh /users quản lý danh sách các tài khoản người dùng đã đăng ký thành công gồm thông tin họ tên, email, tên đăng nhập và mật khẩu phục vụ quy trình xác thực truy cập.

3.5.2 Thiết kế giao diện ứng di động

Sau khi thực hiện phần cứng có khả năng thu thập được dữ liệu và có thể truyền dữ liệu đến các nền tảng cơ sở dữ liệu thì công việc tiếp theo đó là tiến hành xây dựng giao diện cho ứng dụng di động với các thông số để hiển thị dữ liệu thu thập được. Để ứng dụng di động hiển thị dữ liệu một cách nhanh chóng, đầy đủ và đáp ứng được các yêu cầu của người dùng, một mô hình sẽ được xây dựng như hình bên dưới.



Hình 3.20. Tổng quan về giao diện ứng dụng

3.6 Lưu đồ giải thuật

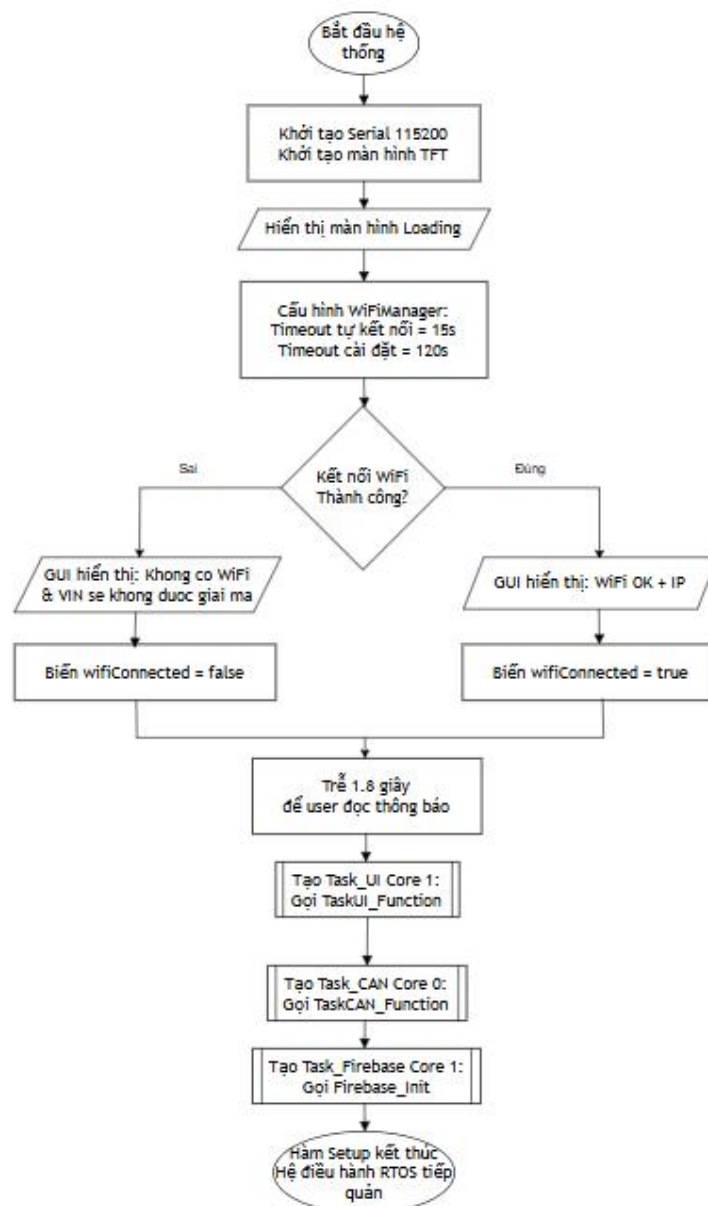
3.6.1 Lưu đồ giải thuật hệ thống

Quy trình vận hành của hệ thống được bắt đầu bằng việc cấu hình các ngoại vi phần cứng cơ bản, bao gồm khởi tạo giao tiếp Serial với tốc độ baudrate 115200 bit/s và khởi động màn hình hiển thị TFT. Ngay sau đó, tiến trình quản lý mạng WiFiManager được kích hoạt với hai chế độ: Tự động và thủ công. Hệ thống sẽ ưu tiên thực hiện tiến trình kết nối mạng tự động dựa trên cấu hình đã lưu trong thời gian chờ

tối đa 15 giây. Nếu kết nối tự động thất bại, hệ thống tự động phát trạm phát để chuyển sang chế độ kết nối thủ công với thời gian chờ là 120 giây.

Trong trường hợp cả hai phương thức đều không thể thiết lập mạng thành công, hệ thống vẫn duy trì hoạt động cục bộ bình thường. Tuy nhiên, tính năng đồng bộ dữ liệu với ứng dụng di động và tra cứu số VIN trực tuyến qua API sẽ tạm thời bị vô hiệu hóa.

Sau bước thiết lập mạng, hệ thống ứng dụng hệ điều hành thời gian thực để khởi tạo và quản lý ba phân luồng thực thi độc lập, nhằm tối ưu hóa tài nguyên phần cứng đa nhân



Hình 3.21. Lưu đồ giải thuật toàn hệ thống

- Phân luồng thứ nhất (Task_UI): Chịu trách nhiệm nhận luồng dữ liệu hệ thống, xử lý tương tác người dùng và kết xuất thông tin trực quan lên màn hình TFT.
- Phân luồng thứ hai (Task_CAN): Được ghim hoạt động song song trên một Core riêng biệt, chuyên đảm nhận nhiệm vụ truyền nhận và giải mã dữ liệu chẩn đoán xe, tuân thủ nghiêm ngặt theo tiêu chuẩn giao thức ISO 15765-2.
- Phân luồng thứ ba (Task_Firebase): Khởi tạo tiến trình giao tiếp cơ sở dữ liệu đám mây chạy ngầm ở Core 1, đảm nhiệm việc đẩy dữ liệu thời gian thực lên Firebase và đồng bộ tín hiệu điều khiển ngược lại từ ứng dụng điện thoại.

3.6.2 Lưu đồ giải thuật từng chức năng

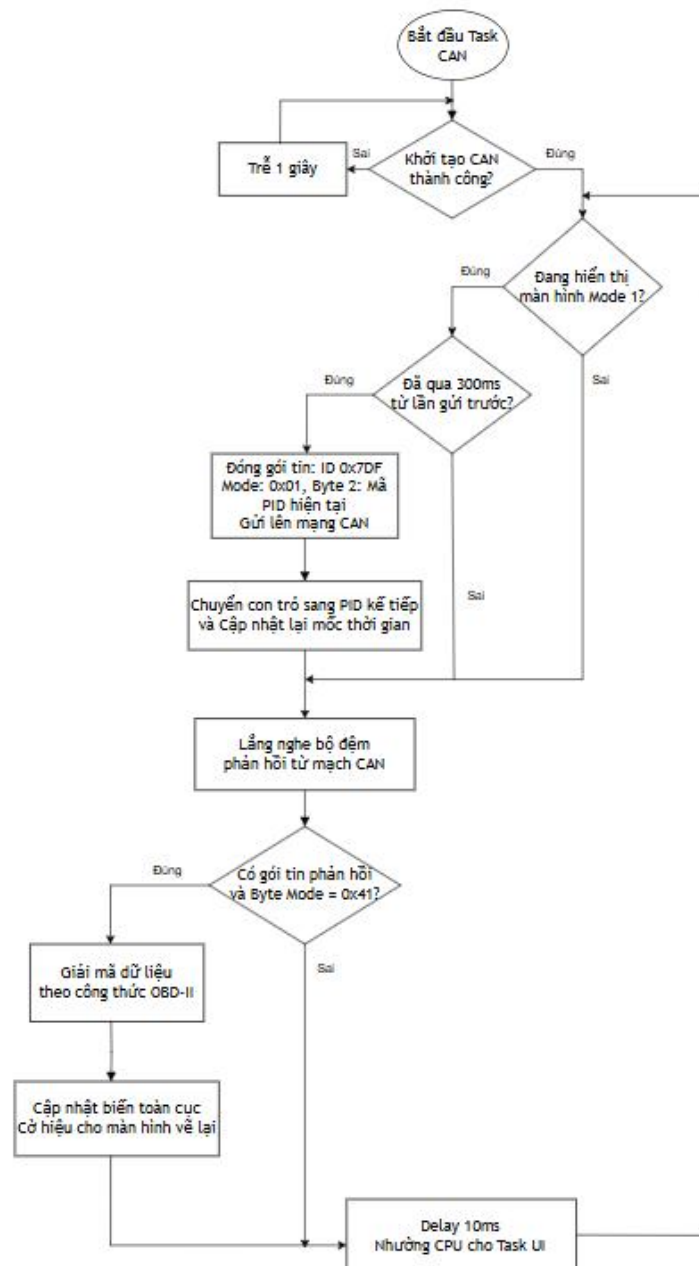
3.6.2.1 Lưu đồ giải thuật Mode 01

Ngay sau khi khởi động phân luồng thực thi Task_CAN, hệ thống tiến hành cấu hình bộ điều khiển giao tiếp CAN với tốc độ Baudrate tiêu chuẩn là 500kbps. Quá trình khởi tạo này được thực hiện trong một vòng lặp kín với chu kỳ 1s cho đến khi phần cứng thiết lập thành công, nhằm đảm bảo tính toàn vẹn của đường truyền trước khi hệ thống chính thức đi vào vòng lặp vô tận của RTOS.

Tại mỗi chu kỳ lặp, hệ thống kiểm tra trạng thái màn hình để xác định xem thiết bị có đang hoạt động ở Mode 01 hay không. Nếu điều kiện này thỏa mãn, hệ thống tiếp tục kiểm tra mốc thời gian xem đã trôi qua 300ms kể từ lần hỏi trước hay chưa. Cơ chế định thời này đóng vai trò then chốt giúp ngăn ngừa hiện tượng nghẽn cổ chai trên mạng CAN Bus. Khi đủ thời gian, chương trình đóng gói tin yêu cầu chẩn đoán chức năng với ID 0x7DF, định cấu hình Mode 0x01 kết hợp cùng mã PID hiện hành. Ngay sau khi phát gói tin, hệ thống tự động xoay vòng con trỏ sang PID kế tiếp và cập nhật lại mốc thời gian để chuẩn bị cho chu kỳ tiếp theo.

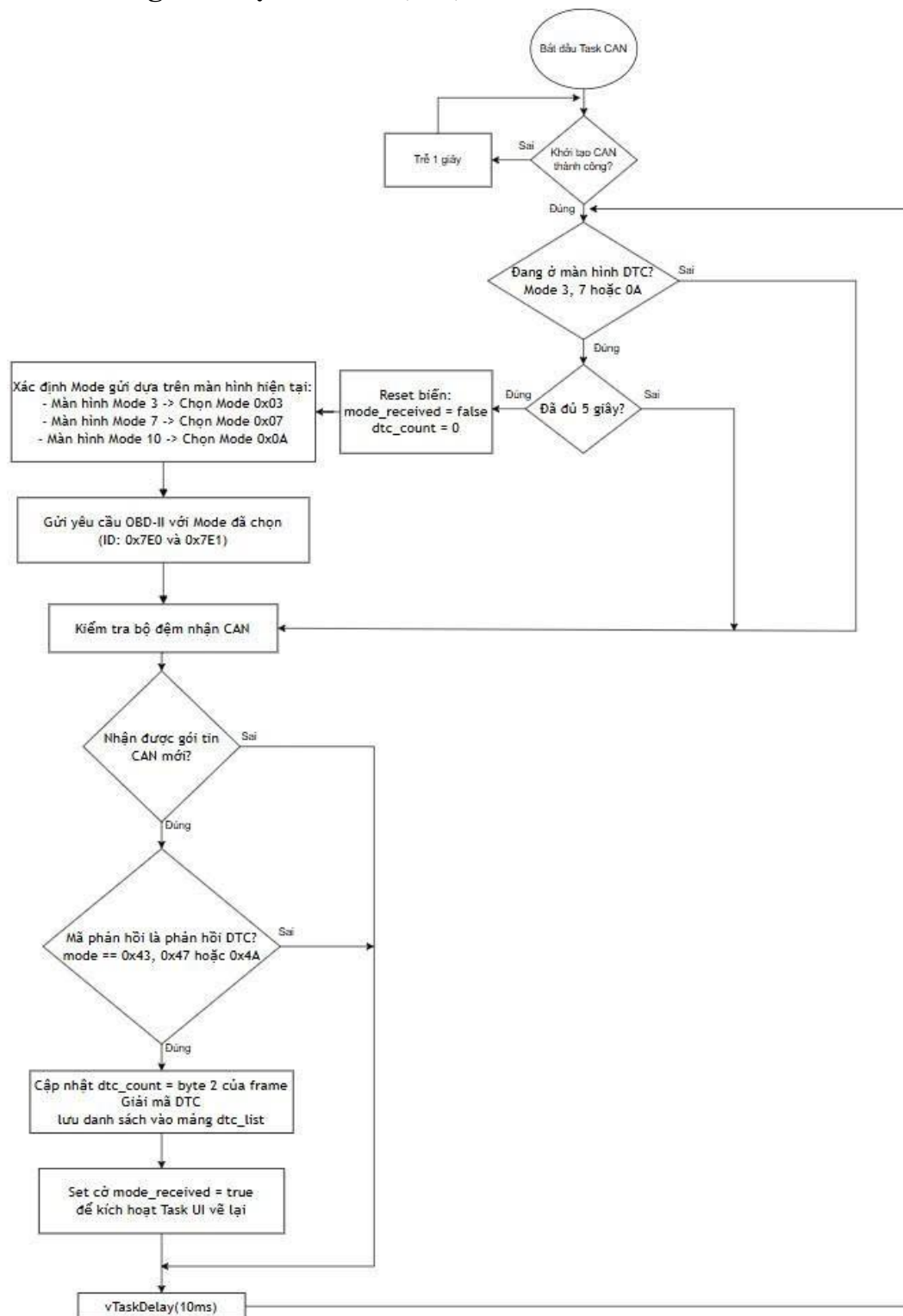
Độc lập với quá trình gửi, bước kế tiếp luôn được thực thi là quá trình lắng nghe và lọc phản hồi từ bộ đệm CAN. Hệ thống tiến hành xác thực gói tin nhận được và chỉ chấp nhận xử lý nếu thông điệp trả về mang mã dịch vụ phản hồi tích cực là 0x41, đồng thời mang mã PID trùng khớp hoàn toàn với các PID hệ thống hỗ trợ. Khi điều kiện được đáp ứng, thuật toán tiến hành giải mã các byte dữ liệu thô nhận được theo chuẩn ISO 15031-5, lưu vào biến toàn cục và phát cờ hiệu để phân luồng giao diện Task_UI nhận biết, từ đó kết xuất thông tin trực quan lên màn hình TFT. Trước khi kết thúc một chu

kỳ lặp, phân luồng CAN sẽ chủ động tạo một khoảng trễ 10ms để nhường tài nguyên CPU cho các tác vụ khác xử lý, đảm bảo hệ thống vận hành đa nhiệm mượt mà.



Hình 3.22. Lưu đồ giải thuật Mode 01

3.6.2.2 Lưu đồ giải thuật Mode 03, 07, 0A

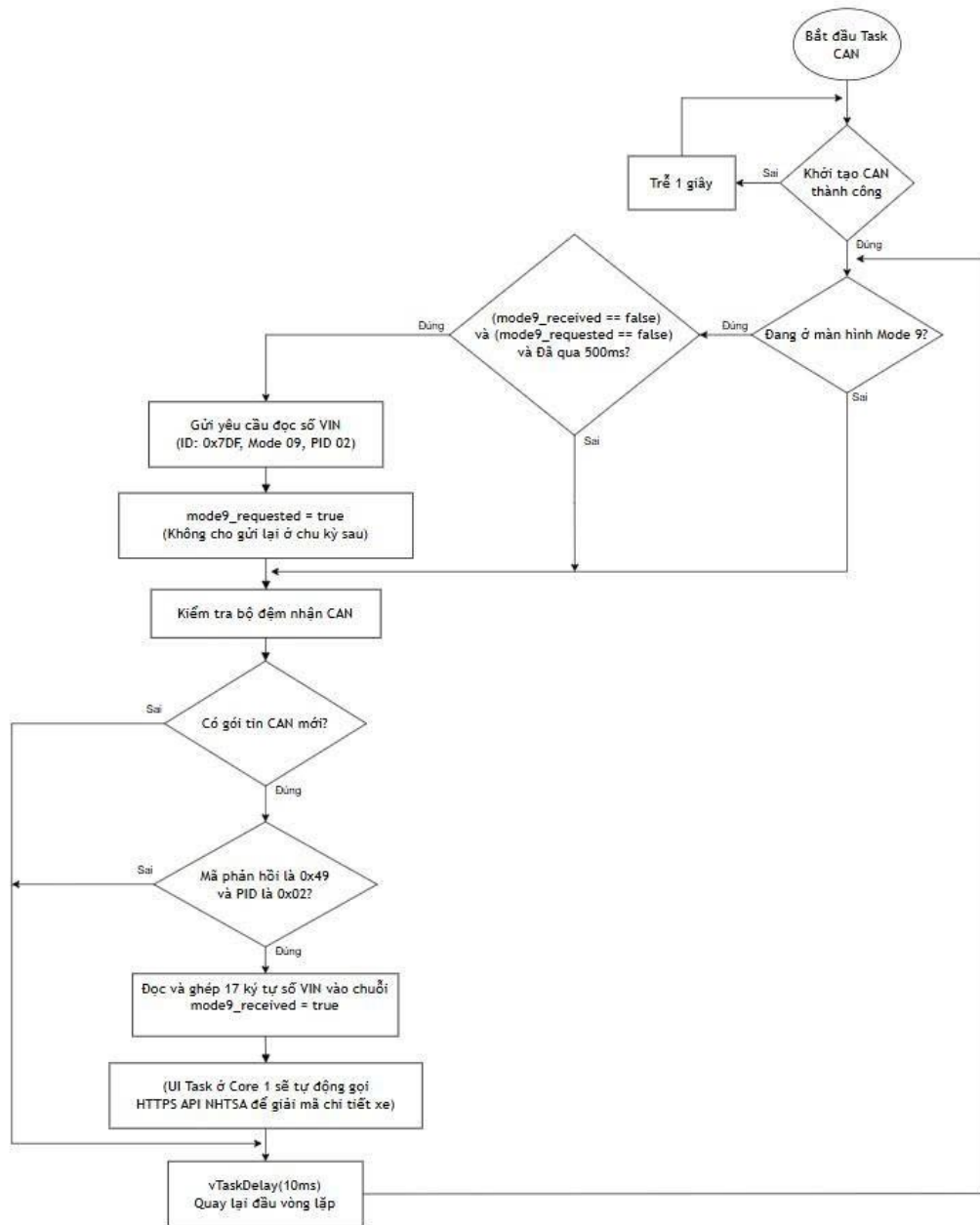


Hình 3.23. Lưu đồ giải thuật Mode 03, 07, 0A

Tiến trình chẩn đoán mã lỗi cho Mode 03, 07 và 0A được triển khai tích hợp trên một luồng xử lý chung của Task_CAN. Định kỳ mỗi 5 giây khi người dùng ở màn hình chẩn đoán tương ứng, hệ thống sẽ xóa dữ liệu cũ (dtc_count = 0, mode3_received

= false), xác định mã dịch vụ chẩn đoán (0x03 hoặc 0x07 hoặc 0x0A) và gửi gói tin yêu cầu qua ID 0x7DF lên mạng CAN bus. Song song đó, hệ thống liên tục kiểm tra bộ đệm nhận để lắng nghe phản hồi từ ECU động cơ qua ID 0x7E8. Nếu nhận được phản hồi hợp lệ trùng khớp với chế độ đang sử dụng (0x43 hoặc 0x47 hoặc 0x4A), hệ thống sẽ trích xuất số lượng lỗi, giải mã các byte thô thành mã lỗi chuẩn OBD-II lưu vào mảng `dtc_list`, đồng thời dựng cờ `mode_received = true` để kích hoạt `Task_UI` cập nhật giao diện màn hình TFT. Ngược lại, nếu không có phản hồi hoặc phản hồi không hợp lệ, bước giải mã sẽ tự động bị bỏ qua để hệ thống tiếp tục chu kỳ quét tiếp theo.

3.6.2.3 Lưu đồ giải thuật Mode 09



Hình 3.24. Lưu đồ giải thuật Mode 09

Tiến trình đọc số VIN ở Mode 09 hoạt động theo cơ chế gửi yêu cầu một lần duy nhất để tiết kiệm băng thông CAN. Khi người dùng mở màn hình, nếu chưa gửi và chưa nhận dữ liệu, hệ thống sẽ phát gói tin yêu cầu đọc VIN qua ID 0x7DF với Mode 09 và PID 02 rồi lập tức khóa gửi bằng cờ mode9_requested = true. Khi nhận được phản hồi hợp lệ từ ECU động cơ qua ID 0x7E8 với mã 0x49 và PID 02, hệ thống trích xuất 17 ký tự VIN và dựng cờ mode9_received = true. Cờ này kích hoạt Task_UI ở

Core 1 tự động gọi HTTPS API của NHTSA để giải mã chi tiết thông tin xe và hiển thị lên màn hình TFT.

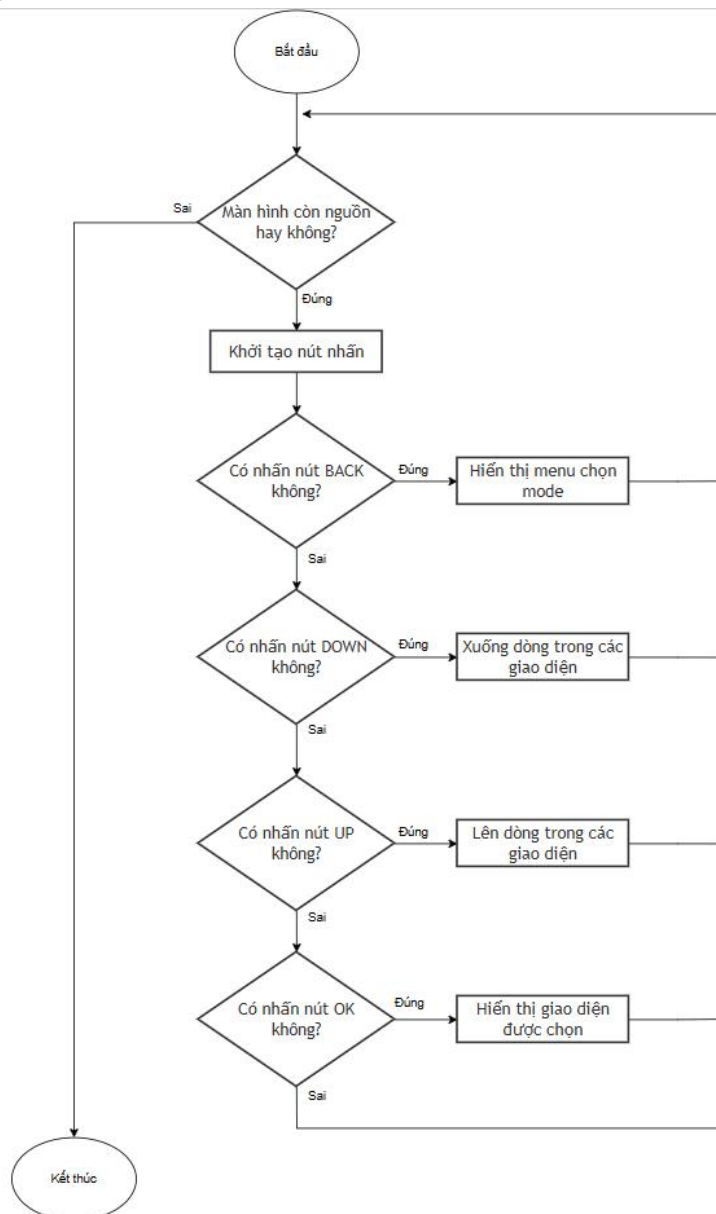
3.6.2.4 Lưu đồ giải thuật tiến trình đồng bộ dữ liệu lên Firebase



Hình 3.25. Lưu đồ giải thuật tiến trình đồng bộ dữ liệu lên Firebase

Ngay sau khi hàm khởi tạo Firebase được gọi, một tiến trình nền sẽ chạy song song với luồng UI để thu thập dữ liệu. Quy trình truyền tải bao gồm hai bước chính: Trước tiên, hệ thống kiểm tra trạng thái kết nối Wi-Fi, đây là điều kiện bắt buộc để thiết lập liên lạc với server; sau đó, dữ liệu sẽ được đóng gói theo định dạng cấu trúc Json và đồng bộ hóa lên nền tảng Firebase.

3.6.2.5 Lưu đồ giải thuật UI



Hình 3.26. Lưu đồ giải thuật UI

Phân luồng giao diện chịu trách nhiệm quản lý hiển thị và xử lý tương tác người dùng trên màn hình TFT qua 6 giao diện chính: Menu tổng, Mode 01, Mode 03, Mode 07, Mode 0A và Mode 09. Trong điều kiện bình thường tức là nguồn cấp đủ, chương trình tiến hành quét và phản hồi tuần tự theo trạng thái của 4 nút nhấn điều hướng: nút BACK để thoát chế độ hiện tại và quay về giao diện Menu; nút DOWN và UP để dịch chuyển con trỏ xuống dòng và lên dòng trên giao diện; nút OK để xác nhận kích hoạt giao diện của Mode được chọn. Sau khi thực thi lệnh tương ứng hoặc khi không có sự kiện nhấn nút, hệ thống quay lại vòng lặp để tiếp tục cập nhật trạng thái tương tác thời gian thực.

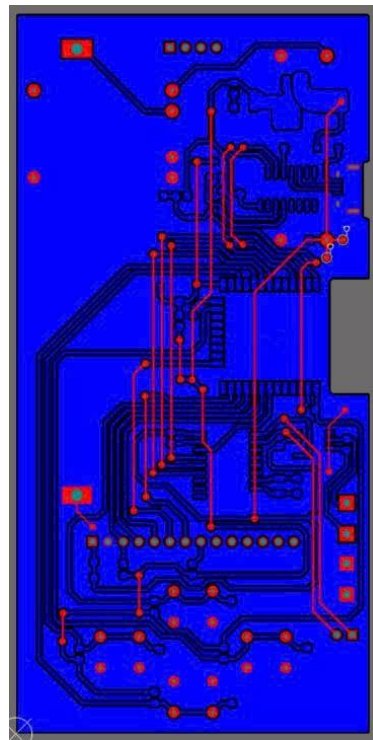
CHƯƠNG 4

KẾT QUẢ THỰC HIỆN

4.1 Kết quả thực hiện trên phần cứng

Sau khi hoàn tất quá trình thiết kế sơ đồ nguyên lý và xây dựng ý tưởng hệ thống, nhóm tiến hành hiện thực hóa các thành phần phần cứng thông qua việc mô phỏng, thiết kế mạch in PCB, xuất mô hình 3D kiểm tra không gian, in mạch và lắp ráp các linh kiện thực tế.

Từ sơ đồ nguyên lý, nhóm thực hiện triển khai thiết kế bản vẽ mạch in PCB hai lớp như hình 4.1.



Hình 4.1. Bản vẽ thiết kế đường mạch in PCB

Sau khi hoàn thiện xong bố cục mạch in, nhóm thực hiện bắt đầu xây dựng mô hình 3D nhằm tối ưu hóa không gian cho bo mạch bố trí màn hình hiển thị trực quan, dễ nhìn và định vị các cổng cắm giao tiếp vật lý sao cho nó thuận tiện nhất khi thao tác kiểm thử trên xe như hình 4.2.



a)

b)

Hình 4.2. a) Mô hình 3D mặt trước và b) mô hình 3D mặt sau của thiết bị

Sau khi hoàn thành xong bước mô phỏng và thiết kế mạch in PCB và xây dựng mô hình 3D, nhóm thực hiện tiến hành gia công và hàn lắp linh kiện thô và linh kiện dán lên bo mạch. Hình 4.3 mô tả hoàn chỉnh thiết bị đã được gia công xong.



a)

b)

Hình 4.3. a) Mô hình mặt trước và b) mặt sau thực tế của thiết bị

Để tăng độ bền và tránh va đập cho các linh kiện điện tử trong quá trình sử dụng thực tế với phương tiện nhóm thực hiện còn thiết kế lớp vỏ bảo vệ bằng nhựa bên ngoài để bảo vệ bo mạch.



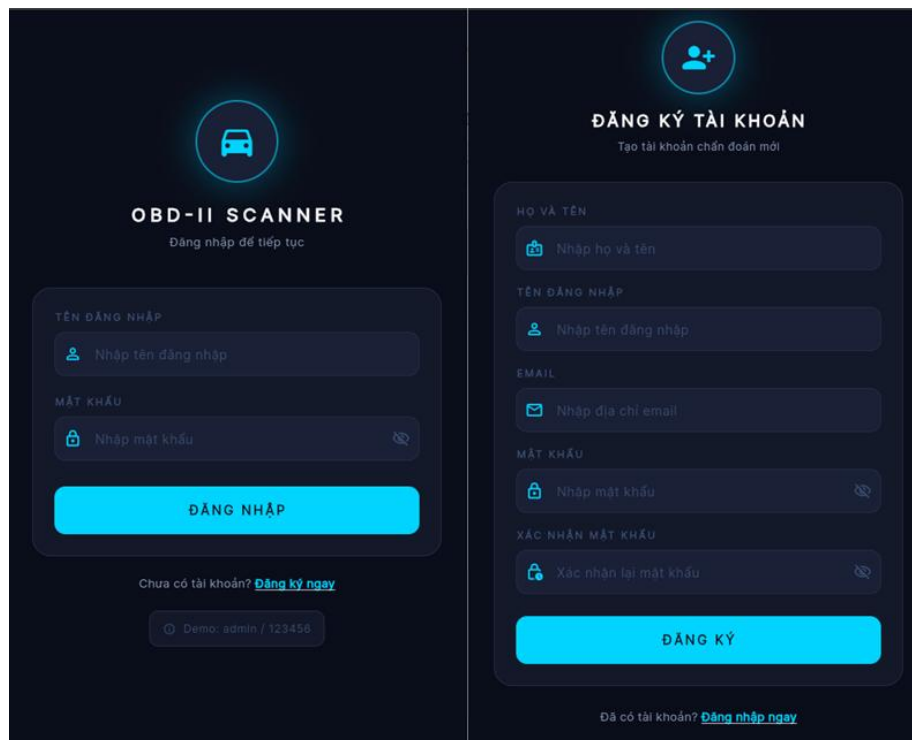
Hình 4.4. Thiết bị hoàn thiện được đóng vỏ bảo vệ

4.2 Kết quả thực hiện trên phần mềm

Trong khuôn khổ đề tài, nhóm đã thiết kế ứng dụng di động Android với các chức năng như sau:

- Quản lý tài khoản: Đăng ký, đăng nhập tài khoản và phân quyền hệ thống gồm 1 tài khoản admin và nhiều tài khoản dưới quyền.
- Yêu cầu đọc dữ liệu: Chủ động gửi yêu cầu truy xuất dữ liệu thay cho phần cứng qua các Mode của OBD-II: Mode 01, 03, 07, 09, 0A.
- Cảnh báo tự động: Cảnh báo tức thời ngay khi tốc độ, vòng tua, nhiên liệu và nhiệt độ nước làm mát vượt ngưỡng cho phép.

Ứng dụng quản lý tài khoản thông qua cơ sở dữ liệu đám mây Firebase. Khi người dùng mới bắt đầu việc đăng ký, hệ thống sẽ tự động đối chiếu dữ liệu để ngăn chặn việc tạo tài khoản có tên đăng nhập trùng lặp. Chỉ những tài khoản có tên đăng nhập duy nhất mới được ghi nhận thành công lên Firebase và được phép sử dụng để đăng nhập vào ứng dụng. Giao diện đăng nhập ở hình 4.5a và giao diện đăng ký tài khoản ở hình 4.5b.



a)

b)

Hình 4.5. a) Giao diện đăng nhập và b) Giao diện đăng ký tài khoản



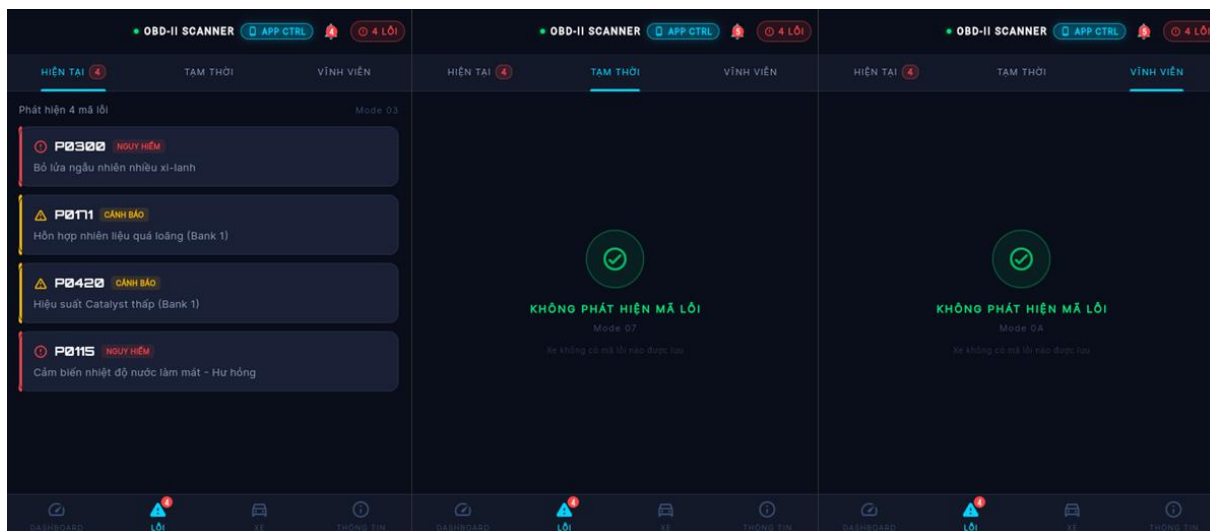
a)

b)

Hình 4.6. a) Giao diện Mode 01 và b) Thông báo các cảnh báo ngưỡng

Hình 4.6a trình bày giao diện giám sát thời gian thực các thông số vận hành của xe truy xuất từ Mode 01 OBD-II như tốc độ xe, vòng tua máy, nhiệt độ nước làm mát, và mức nhiên liệu.

vị trí bướm ga, tải động cơ và mức nhiên liệu. Khi phát hiện các thông số vận hành này vượt ngưỡng an toàn đã thiết lập (tốc độ > 120 km/h, vòng tua > 6000 vòng/phút, nhiệt độ nước > 105°C hoặc mức nhiên liệu < 15%), hệ thống sẽ tự động kích hoạt cảnh báo khẩn cấp và lưu lại trong danh sách lịch sử cảnh báo được mô tả ở hình 4.6b.



a)

b)

c)

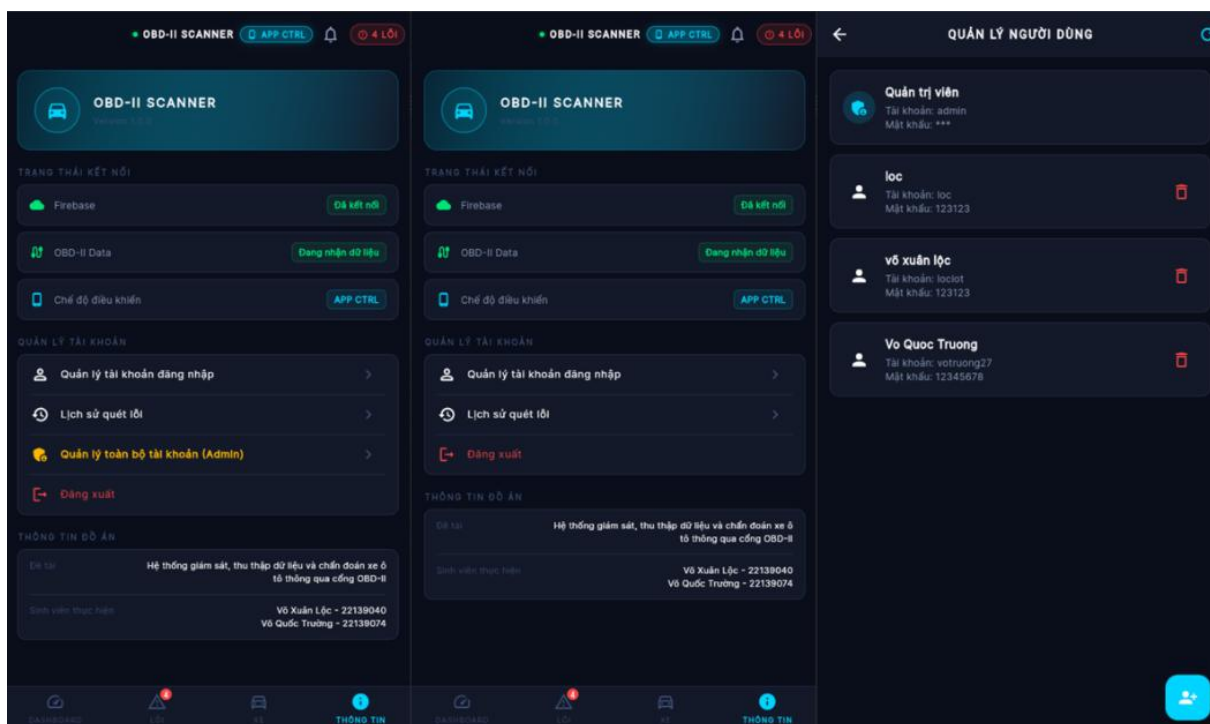
Hình 4.7. a) Giao diện các Mode 03, b) Giao diện Mode 07 và c) Giao diện Mode 0A

Hình 4.7 thể hiện trực quan giao diện chẩn đoán lỗi động cơ hiển thị mã lỗi kỹ thuật qua ba Mode OBD-II gồm Mode 03, Mode 07 và Mode 0A. Tại mỗi Mode, nếu phát hiện sự cố, hệ thống sẽ hiển thị chi tiết mã lỗi, mức độ cảnh báo cùng mô tả nguyên nhân lỗi; ngược lại, nếu không có sự cố nào xảy ra, màn hình sẽ hiển thị thông báo “Không có mã lỗi” để người dùng dễ dàng nhận biết.

Hình 4.8 thể hiện trực quan giao diện chi tiết giao diện Mode 09 hiển thị mã định danh số khung xe được lấy trực tiếp từ Firebase và giải mã để cung cấp các thông tin cơ bản của phương tiện bao gồm: Hãng xe, mẫu xe, năm sản xuất và loại động cơ.



Hình 4.8. Giao diện Mode 09



a)

b)

c)

Hình 4.9. a) Giao diện thông tin của admin b) Giao diện thông tin của User và c) Giao diện quản lý tài khoản người dùng

Tính năng phân quyền tài khoản được thể hiện trực quan ở hình 4.8. Giao diện của tài khoản Admin ở hình 4.8a hiển thị thêm tính năng “Quản lý toàn bộ tài khoản” dành cho quản trị viên, trong khi giao diện của tài khoản thường ở hình 4.8b tự động ẩn tính năng này để giới hạn quyền hạn. Quyền này cho phép xem toàn bộ các trạng thái đang được đăng ký hiện tại và cho phép xóa tài khoản, giao diện ở hình 4.8c

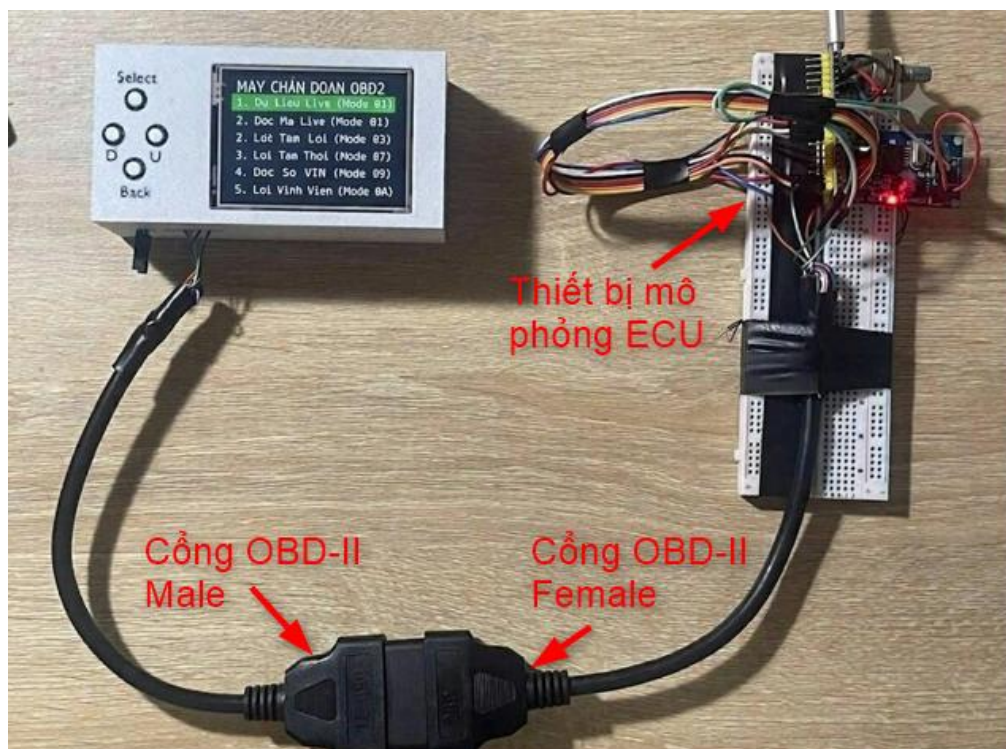
4.3 Kết quả thực hiện quy trình kiểm thử

Sau khi hoàn thiện xong phần cứng, hệ thống chẩn đoán được đưa vào giai đoạn thử nghiệm. Quá trình kiểm thử này đóng vai trò cốt lõi nhằm đảm bảo được hệ thống chạy ổn định, đọc được dữ liệu một cách chính xác, ngoài ra đánh giá được năng lực xử lý thời gian thực, độ chính xác của các thuật toán giải mã dữ liệu.

Quá trình này bao gồm hai giai đoạn kiểm thử độc lập: kiểm thử trên bộ mô phỏng giả lập và kiểm thử thực tế trên phương tiện giao thông. Trong đó việc kiểm thử trên bộ mô phỏng giả lập đóng vai trò là tiền đề và cơ sở an toàn, giúp kiểm chứng độ ổn định của hệ thống trước khi kết nối trực tiếp vào hệ thống điện và mạng CAN trên xe thật.

4.3.1 Thử nghiệm trên bộ mô phỏng giả lập

Trước khi kết nối với thiết bị với xe thật, thiết bị bắt buộc phải trải qua bước thử nghiệm tiền đề trên bộ mô phỏng giả lập. Bước này nhằm kiểm tra tính ổn định của phần cứng và xác định xem quy trình truyền nhận, xử lý các khung tin yêu cầu và phản hồi có tuân thủ hoàn toàn đúng theo các quy chuẩn quốc tế ISO 15765-2 (về giao thức truyền tải tầng mạng CAN) và ISO 15031 (về chuẩn dịch vụ chẩn đoán khí thải OBD-II) hay không, phòng tránh các nguy cơ chập cháy tín hiệu hoặc xung đột làm treo hệ thống mạng Bus của phương tiện do lỗi phần mềm phát sinh.



Hình 4.10. Hệ thống mô phỏng giả lập

Kết quả đạt được: Hệ thống vận hành ổn định, thực hiện truyền nhận các khung tin CAN thô chính xác và xử lý thời gian thực tốt. Thiết bị kiểm chứng được quy trình truyền nhận dữ liệu hoàn toàn trùng khớp với cấu trúc tiêu chuẩn, nhận diện và giải mã chính xác các kịch bản lệnh giả lập thuộc các chế độ Mode 01, Mode 03 và Mode 09 mà không xảy ra hiện tượng mất gói tin hay treo vi điều khiển.

Quá trình thử nghiệm giả lập thành công đã khẳng định độ tin cậy của firmware và phần cứng, hoàn thành xuất sắc vai trò tiền đề và tạo cơ sở vững chắc để triển khai thực nghiệm trực tiếp trên xe thật.

4.3.2 Thử nghiệm thực tế trên phương tiện và phân tích kết quả



Hình 4.11. Thử nghiệm thực tế trên xe ô tô Mazda 3 2022

Sau khi đã kiểm thử trên bộ mô phỏng giả lập xong, thiết bị được đưa vào thử nghiệm trực tiếp trên xe ô tô Mazda 3 2022 như hình 4.10. Quá trình kiểm thử được thực hiện bằng cách kết nối thiết bị kiểm thử với cổng OBD-II của xe, tiến hành khởi động động cơ để thiết lập luồng truyền thông dữ liệu thực tế trên mạng CAN Bus với tốc độ 500 kbps.



Hình 4.12. Kết nối vật lý giữa thiết bị chẩn đoán với cổng OBD-II của xe Mazda 3

Để giám sát và bóc tách luồng truyền, máy hiện sóng Logic Analyzer được đầu nối trực tiếp vào đường truyền vật lý, thực hiện bắt luồng xung tín hiệu đồng thời trên hai kênh giám sát độc lập:

- Kênh Channel 0: Thực hiện bắt luồng và giám sát xung tín hiệu của khung lệnh yêu cầu phát đi từ thiết bị chẩn đoán.
- Kênh Channel 1: Thực hiện bắt luồng và giám sát xung tín hiệu của khung tin phản hồi trả về từ hộp ECU xe.

4.3.2.1 Kết quả kiểm thử Mode 01

Tiến hành nổ máy xe ô tô và duy trì động cơ ở trạng thái không tải ổn định, sử dụng máy hiển sóng Logic Analyzer để trích xuất các khung tin CAN thô trên mạng Bus của xe nhằm đối chiếu trực tiếp với các giá trị hiển thị thực tế trên màn hình LCD của thiết bị và cụm đồng hồ Taplo trên xe ô tô.



Hình 4.13. Ảnh chụp màn hình LCD của thiết bị kiểm thử thu thập dữ liệu Mode 01



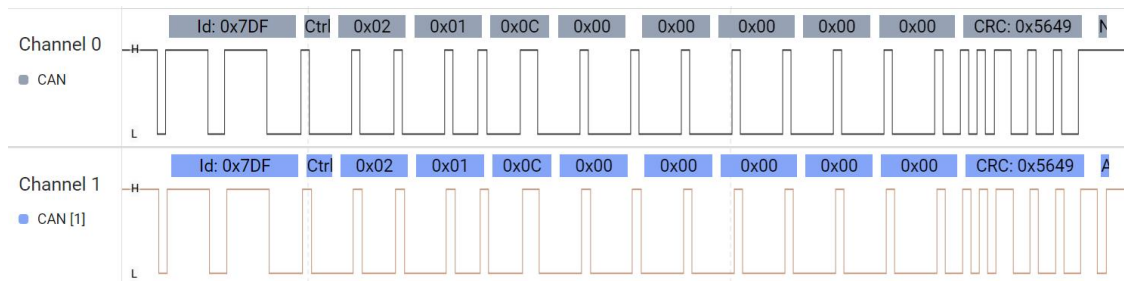
Hình 4.14. Ảnh chụp cụm đồng hồ Taplo trên xe ô tô tại thời điểm thực nghiệm đối chiếu

Đối chiếu Log dữ liệu CAN bus và giá trị hiển thị thực tế của Mode 01

1. Tốc độ vòng tua động cơ

Từ dữ liệu thu thập của máy hiện sóng Logic Analyzer, cấu trúc chuỗi mã Hex của cặp khung tin yêu cầu và phản hồi cho thông số vòng tua máy được trích xuất trong hai hình 4.14.

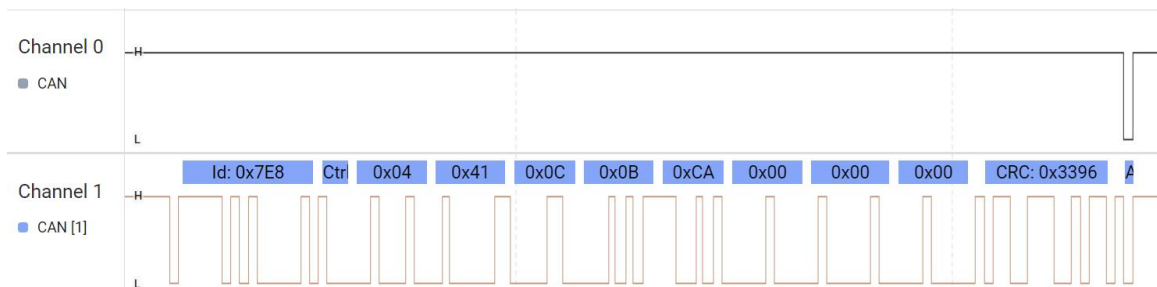
Khung tin yêu cầu:



Hình 4.15. Giải đồ xung của lệnh yêu cầu dữ liệu tốc độ vòng tua của Mode 01

- ID: 0x7DF
- Data Field: 02 | 01 | 0C | 00 | 00 | 00 | 00 | 00
 - Byte 0 (0x02): Khai báo 2 byte dữ liệu hiệu dụng phía sau.
 - Byte 1 (0x01): Khai báo chế độ chẩn đoán Mode 01
 - Byte 2 (0x0C): Mã PID yêu cầu dữ liệu tốc độ vòng quay động cơ
 - Byte 3 – Byte 6 (0x00): Byte đệm

Khung tin phản hồi:



Hình 4.16. Giải đồ xung phản hồi tốc độ vòng tua máy từ ECU

- ID: 0x7E8
- Data Field: 04 | 41 | 0C | 0B | BC | 00 | 00 | 00
 - Byte 0 (0x04): Khai báo khung phản hồi có 4 byte dữ liệu
 - Byte 1 (0x41): Xác nhận phản hồi hợp lệ cho Mode 01
 - Byte 2 (0x0C): Xác nhận mã PID
 - Byte 3 (0x0B) và Byte 4 (0xBC): Hai byte dữ liệu thô vật lý, ký hiệu lần lượt là Byte A và Byte B chứa giá trị mã hóa của vòng tua máy.
 - Byte 5 – Byte 6 (0x00): Byte đệm

Chuyển đổi hai byte dữ liệu Hex thô sang hệ thập phân A=11 và B=188. Áp dụng công thức trong bảng 2.3 định nghĩa và công thức giải mã các thông số Mode 01 PID của hệ thống.

$$\text{RPM} = \frac{(A \times 256) + B}{4} = \frac{(11 \times 256) + 188}{4} = \frac{2816 + 188}{4} = \frac{3004}{4} = 751 \text{ (vòng/phút)}$$

Kết quả:

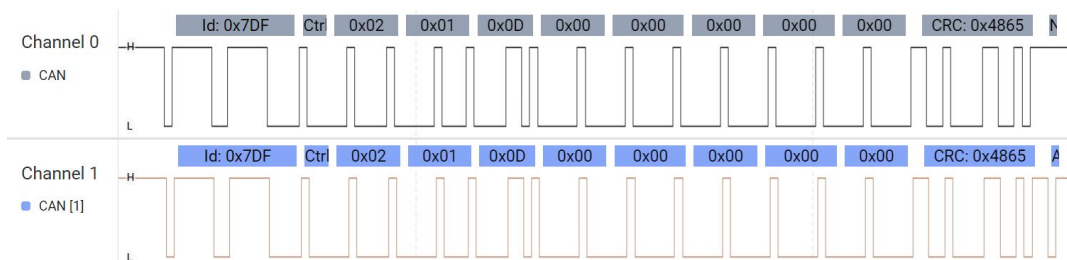
Trên màn hình LCD của thiết bị: Hệ thống xử lý giải mã và xuất ra thông số **Vòng tua : 751 vòng/p** đúng với tính toán trên.

Trên cụm đồng hồ Taplo: Kim báo vòng tua cơ học chỉ ổn định ở vùng dưới vạch 1000 RPM (nhằm sát mức 750 RPM), khớp hoàn toàn 100% với giá trị lý thuyết tính toán từ mạng Bus.

2. Tốc độ xe

Từ dữ liệu thu thập của máy hiện sóng Logic Analyzer, cấu trúc chuỗi mã Hex của cặp khung tin yêu cầu và phản hồi cho thông số tốc độ xe được trích xuất như sau:

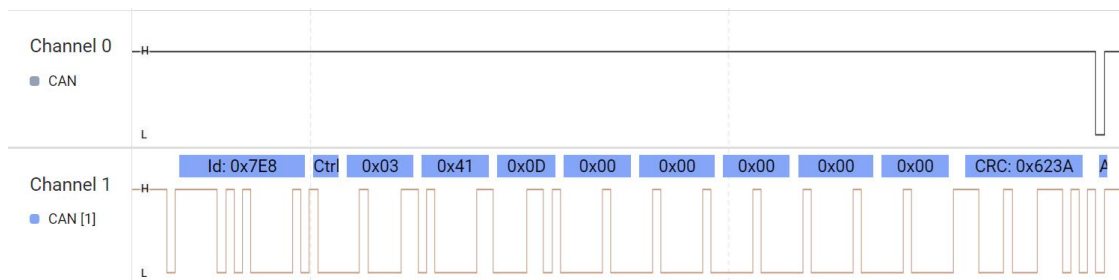
Khung tin yêu cầu:



Hình 4. 17 Giải đồ xung của lệnh yêu cầu dữ liệu tốc độ xe Mode 01

- ID: 0x7DF
- Data Field: 02 | 01 | 0D | 00 | 00 | 00 | 00 | 00
 - Byte 0 (0x02): Khai báo 2 byte dữ liệu hiệu dụng phía sau.
 - Byte 1 (0x01): Khai báo chế độ chẩn đoán Mode 01.
 - Byte 2 (0x0D): Mã PID yêu cầu dữ liệu tốc độ xe (Vehicle Speed).
 - Byte 3 – Byte 7 (0x00): Byte đệm.

Khung tin phản hồi:



Hình 4.18. Giải đồ xung phản hồi tốc độ xe từ ECU

- ID: 0x7E8
- Data Field: 03 | 41 | 0D | 00 | 00 | 00 | 00 | 00
 - Byte 0 (0x03): Khai báo khung phản hồi có 3 byte dữ liệu.
 - Byte 1 (0x41): Xác nhận phản hồi hợp lệ cho Mode 01.
 - Byte 2 (0x0D): Xác nhận mã PID tốc độ xe.
 - Byte 3 (0x00): Byte dữ liệu thô vật lý (ký hiệu là Byte A) chứa giá trị mã hóa của tốc độ xe.
 - Byte 4 – Byte 7 (0x00): Byte đệm.

Chuyển đổi byte dữ liệu Hex thô sang hệ thập phân $A = 0$. Áp dụng công thức trong bảng 2.3 định nghĩa và công thức giải mã các thông số Mode 01 PID của hệ thống.

$$\text{Speed} = A = 0 \quad (\text{km/h})$$

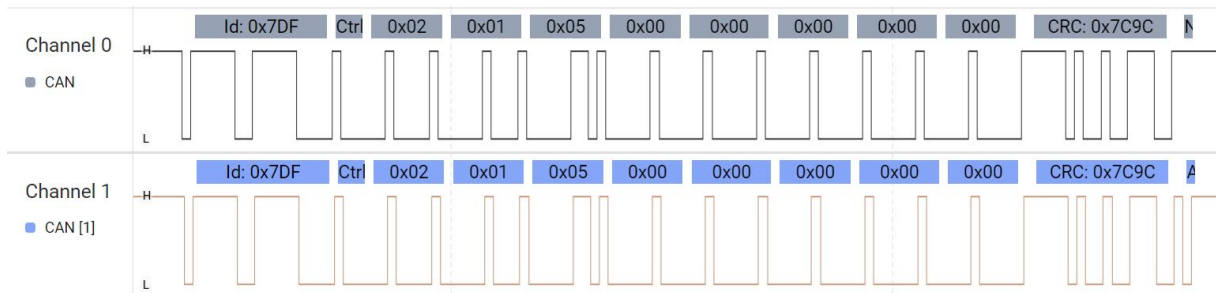
Kết quả:

Trên màn hình LCD của thiết bị: Hệ thống xử lý giải mã và xuất ra thông số **Tốc độ: 0 km/h** đúng với tính toán trên.

Trên cụm đồng hồ Taplo: Đồng hồ hiển thị tốc độ xe đạt trạng thái tĩnh 0 km/h và hộp số đang giữ ở vị trí P khi xe nổ máy tại chỗ, khớp hoàn toàn 100% với giá trị lý thuyết tính toán từ mạng Bus.

3. Nhiệt độ nước làm mát động cơ

Khung tin yêu cầu:



Hình 4.19. Giải đồ xung của lệnh yêu cầu nhiệt độ nước làm mát động cơ

- ID: 0x7DF
- Data Field: 02 | 01 | 05 | 00 | 00 | 00 | 00 | 00
 - Byte 0 (0x02): Khai báo 2 byte dữ liệu hiệu dụng phía sau.
 - Byte 1 (0x01): Khai báo chế độ chẩn đoán Mode 01.
 - Byte 2 (0x05): Mã PID yêu cầu dữ liệu nhiệt độ nước làm mát (Engine Coolant Temperature).
 - Byte 3 – Byte 7 (0x00): Byte đệm.

Khung tin phản hồi:



Hình 4.20. Giải đồ xung phản hồi nhiệt độ nước từ ECU

- ID: 0x7E8
- Data Field: 03 | 41 | 05 | 8088 | 00 | 00 | 00 | 00
 - Byte 0 (0x03): Khai báo khung phản hồi có 3 byte dữ liệu chẩn đoán.
 - Byte 1 (0x41): Xác nhận phản hồi hợp lệ cho Mode 01.
 - Byte 2 (0x05): Xác nhận mã PID nhiệt độ nước làm mát.
 - Byte 3 (0x80): Byte dữ liệu thô vật lý (ký hiệu là Byte A) chứa giá trị mã hóa của nhiệt độ nước.
 - Byte 4 – Byte 7 (0x00): Byte đệm.

Chuyển đổi byte dữ liệu Hex thô sang hệ thập phân $A = 128$. Áp dụng công thức trong bảng 2.3 định nghĩa và công thức giải mã các thông số Mode 01 PID của hệ thống:

$$\text{Temp} = A - 40 = 128 - 40 = 88 \quad (^\circ \text{C})$$

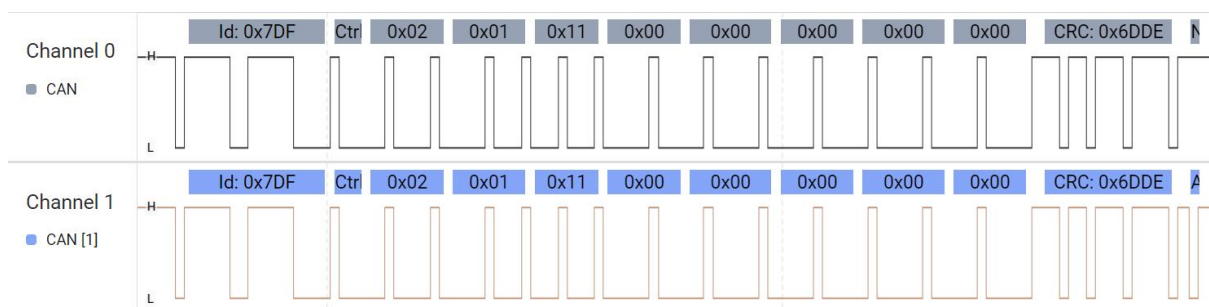
Kết quả:

Trên màn hình LCD của thiết bị: Hệ thống xử lý giải mã và xuất ra thông số **Nhiệt độ : 88 °** đúng với tính toán trên.

Trên cụm đồng hồ Taplo: Kim báo nhiệt độ nước làm mát nằm ở vị trí tiệm cận sát dưới vạch cơ bản giữa 100°C tương đương với ngưỡng nhiệt làm việc thực tế ổn định của xe, khớp hoàn toàn 100% với giá trị lý thuyết.

4. Vị trí bướm ga tuyệt đối

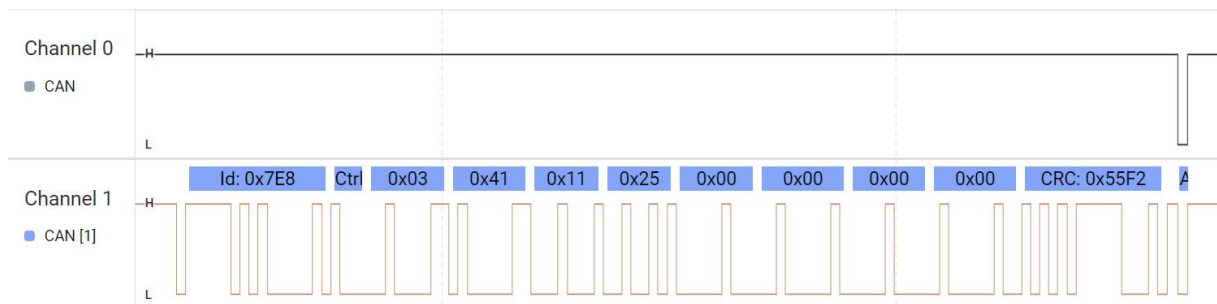
Khung tin yêu cầu:



Hình 4.21. Giản đồ xung của lệnh yêu cầu dữ liệu vị trí bướm ga tuyệt đối

- ID: 0x7DF
- Data Field: 02 | 01 | 11 | 00 | 00 | 00 | 00 | 00
 - Byte 0 (0x02): Khai báo 2 byte dữ liệu hiệu dụng phía sau.
 - Byte 1 (0x01): Khai báo chế độ chẩn đoán Mode 01.
 - Byte 2 (0x11): Mã PID yêu cầu dữ liệu vị trí bướm ga tuyệt đối.
 - Byte 3 – Byte 7 (0x00): Byte đệm.

Khung tin phản hồi:



Hình 4.22. Giải đồ xung phản hồi vị trí bướm ga tuyệt đối từ ECU

- ID: 0x7E8
- Data Field: 03 | 41 | 11 | 25 | 00 | 00 | 00 | 00
 - Byte 0 (0x03): Khai báo khung phản hồi có 3 byte dữ liệu chuẩn đoán.
 - Byte 1 (0x41): Xác nhận phản hồi hợp lệ cho Mode 01.
 - Byte 2 (0x11): Xác nhận mã PID vị trí bướm ga.
 - Byte 3 (0x25): Byte dữ liệu thô vật lý (ký hiệu là Byte A) chứa giá trị mã hóa độ mở bướm ga.
 - Byte 4 – Byte 7 (0x00): Byte đệm.

Chuyển đổi byte dữ liệu Hex thô sang hệ thập phân $A = 37$. Áp dụng công thức trong bảng 2.3 định nghĩa và công thức giải mã các thông số Mode 01 PID của hệ thống:

$$TP = \frac{A \times 100}{255} = \frac{36 \times 100}{255} = \frac{3600}{255} \approx 14.5 \quad (\%)$$

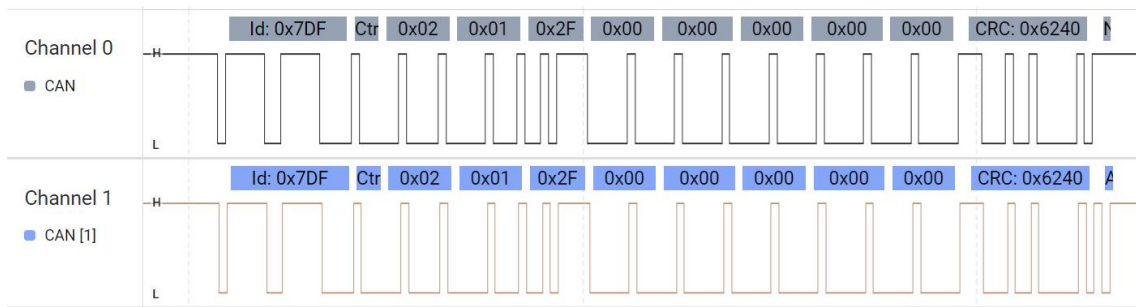
Kết quả:

Trên màn hình LCD của thiết bị: Hệ thống xử lý giải mã và xuất ra thông số Buom ga : 14.5 % đúng với tính toán trên.

Chế độ Idling ổn định: Vòng tua máy đạt chuẩn khoảng 750; góc mở thực tế của bướm ga khớp hoàn toàn với thông số tính toán từ ECU.

5. Mức nhiên liệu

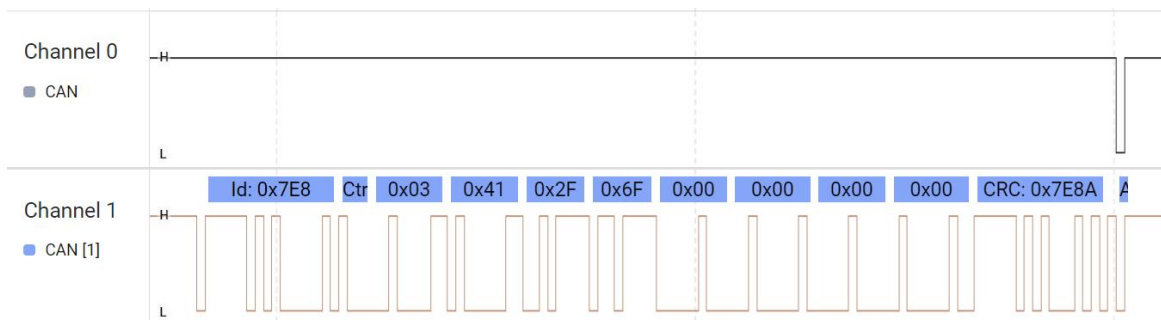
Khung tin yêu cầu:



Hình 4.23. Giải đồ xung của lệnh yêu cầu dữ liệu mức nhiên liệu

- ID: 0x7DF
- Data Field: 02 | 01 | 2F | 00 | 00 | 00 | 00 | 00
 - Byte 0 (0x02): Khai báo 2 byte dữ liệu hiệu dụng phía sau.
 - Byte 1 (0x01): Khai báo chế độ chẩn đoán Mode 01.
 - Byte 2 (0x2F): Mã PID yêu cầu dữ liệu mức nhiên liệu đầu vào (Fuel Level Input).
 - Byte 3 – Byte 7 (0x00): Byte đệm.

Khung tin phản hồi:



Hình 4.24. Giải đồ xung phản hồi mức nhiên liệu từ ECU

- ID: 0x7E8
- Data Field: 03 | 41 | 2F | 6F | 00 | 00 | 00 | 00
 - Byte 0 (0x03): Khai báo khung phản hồi có 3 byte dữ liệu chẩn đoán.
 - Byte 1 (0x41): Xác nhận phản hồi hợp lệ cho Mode 01.
 - Byte 2 (0x2F): Xác nhận mã PID mức nhiên liệu.
 - Byte 3 (0x6F): Byte dữ liệu thô vật lý (ký hiệu là Byte A) chứa giá trị mã hóa lượng nhiên liệu.
 - Byte 4 – Byte 7 (0x00): Byte đệm.

Chuyển đổi byte dữ liệu Hex thô sang hệ thập phân $A = 0x6F \rightarrow 111$. Áp dụng công thức trong bảng 2.3 định nghĩa và công thức giải mã các thông số Mode 01 PID của hệ thống:

$$FLI = \frac{A \times 100}{255} = \frac{111 \times 100}{255} = \frac{11100}{255} \approx 43.5 \Rightarrow (\%)$$

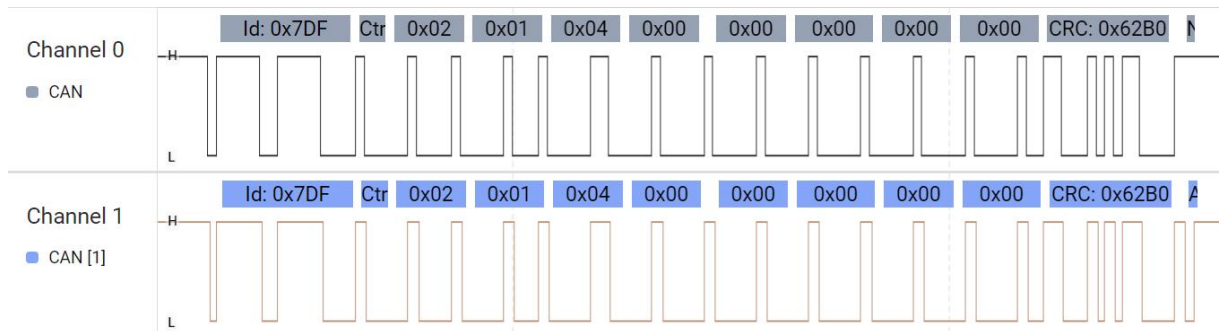
Kết quả:

Trên màn hình LCD của thiết bị: Hệ thống xử lý giải mã và xuất ra thông số **Nhiên liệu : 43.5 %** đúng với tính toán trên.

Trên cụm đồng hồ Taplo: Kim xăng chỉ báo ở mức dưới $\frac{1}{2}$ bình một chút, đồng bộ hoàn hảo với giá trị thực tế đo đạc.

6. Tải trọng động cơ

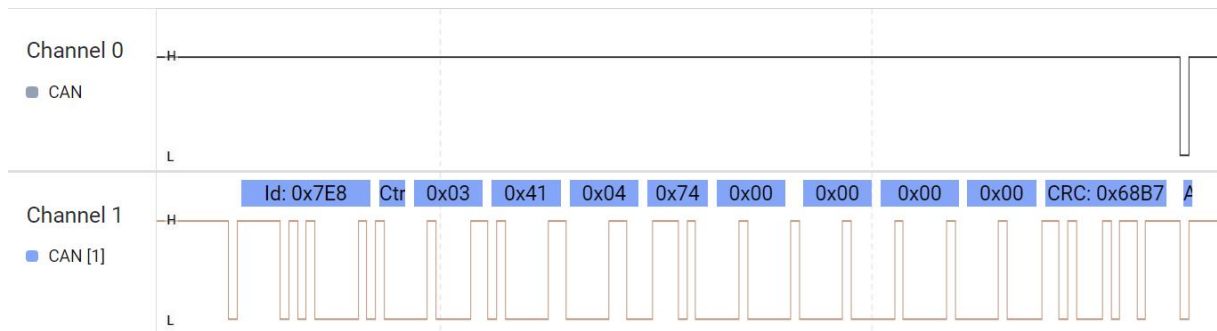
Khung tin yêu cầu:



Hình 4.25. Giải đồ xung của lệnh yêu cầu dữ liệu tải trọng động cơ

- ID: 0x7DF
- Data Field: 02 | 01 | 04 | 00 | 00 | 00 | 00 | 00
 - Byte 0 (0x02): Khai báo 2 byte dữ liệu hiệu dụng phía sau.
 - Byte 1 (0x01): Khai báo chế độ chẩn đoán Mode 01.
 - Byte 2 (0x04): Mã PID yêu cầu dữ liệu tải trọng động cơ tính toán.
 - Byte 3 – Byte 7 (0x00): Byte đệm.

Khung tin phản hồi:



Hình 4.26. Giải đồ xung phản hồi tải trọng động cơ từ ECU

- ID: 0x7E8
- Data Field: 03 | 41 | 04 | 74 | 00 | 00 | 00 | 00
 - Byte 0 (0x03): Khai báo khung phản hồi có 3 byte dữ liệu chẩn đoán.
 - Byte 1 (0x41): Xác nhận phản hồi hợp lệ cho Mode 01.
 - Byte 2 (0x04): Xác nhận mã PID tải trọng động cơ.
 - Byte 3 (0x74): Byte dữ liệu thô vật lý (ký hiệu là Byte A) chứa giá trị mã hóa tải trọng động cơ.
 - Byte 4 – Byte 7 (0x00): Byte đệm.

Chuyển đổi byte dữ liệu Hex thô sang hệ thập phân $A = 0x74 \rightarrow 116$. Áp dụng công thức trong bảng 2.3 định nghĩa và công thức giải mã các thông số Mode 01 PID của hệ thống:

$$\text{LOAD} = \frac{A \times 100}{255} = \frac{116 \times 100}{255} = \frac{11600}{255} \approx 45.5 \quad (\%)$$

Kết quả:

Trên màn hình LCD của thiết bị: Hệ thống xử lý giải mã và xuất ra thông số **Tải máy : 45.5 %** đúng với tính toán trên.

Trên cụm đồng hồ Taplo: Hiệu suất tải máy tính toán phù hợp cho kích bản xe nổ máy không tải kết hợp bật các hệ thống phụ tải phụ (điều hòa, đèn xe...), khớp hoàn toàn 100% với giá trị lý thuyết.

4.3.2.2 Kết quả kiểm thử Mode 03, Mode 07

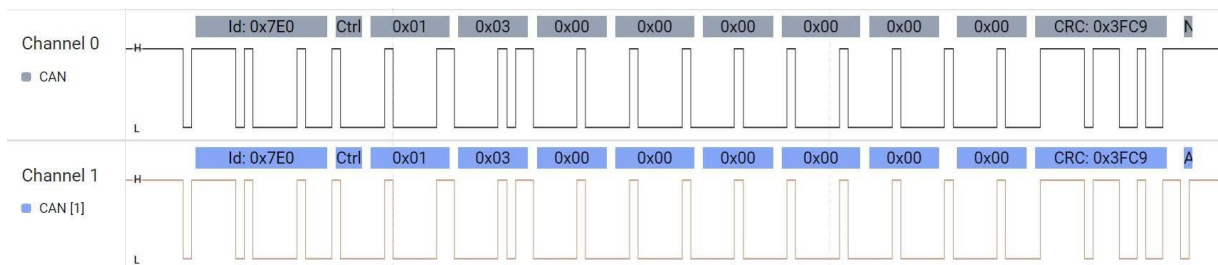
Do phương tiện thử nghiệm đang ở trạng thái hoạt động bình thường, không phát sinh lỗi, luồng dữ liệu trả về ngắn nên quá trình truyền nhận tuân thủ theo cấu trúc khung đơn của tầng mạng ISO 15765-2, ngoại trừ trường hợp dịch vụ không được hỗ trợ sẽ trả về khung tin báo lỗi phân tích sâu.



Hình 4.27. Ảnh chụp màn hình LCD của thiết bị chẩn đoán đọc mã lỗi

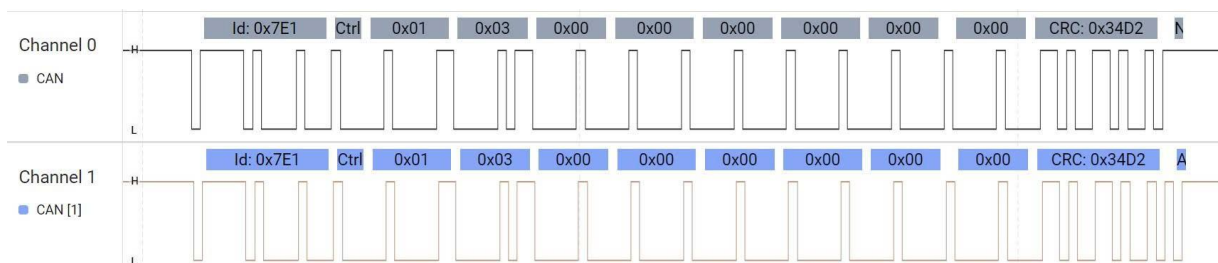
Quá trình kiểm thử được thực hiện lần lượt với các hộp ECU động cơ và ECU hộp số

Khung tin yêu cầu:



Hình 4.28. Giải đồ xung của lệnh yêu cầu mã lỗi đến ECU động cơ Mode 03,

07

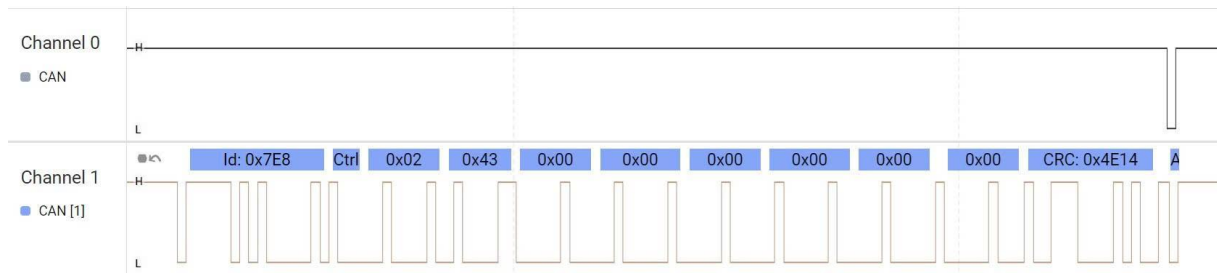


Hình 4.29. Giải đồ xung của lệnh yêu cầu mã lỗi đến ECU hộp số Mode 03, 07

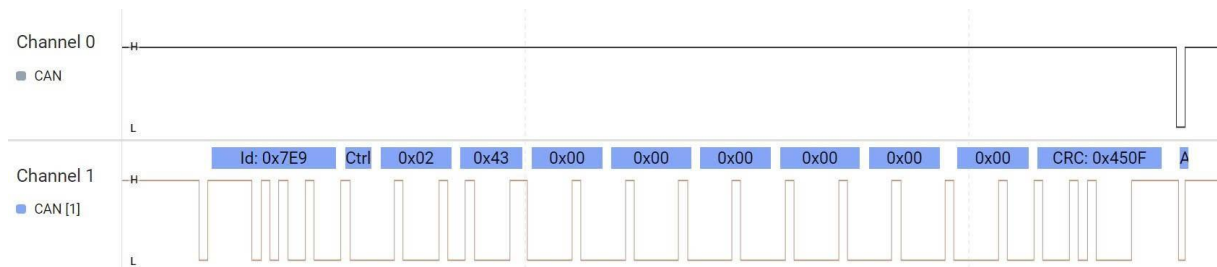
- ID: 0x7E0 (ECU động cơ)/ 0x7E1 (ECU hộp số)
- Data Field: 01 | 03 | 00 | 00 | 00 | 00 | 00 | 00

- Byte 0 (0x01): Khai báo loại khung đơn với 1 byte dữ liệu hiệu dụng phía sau.
- Byte 1 (0x03): Khai báo chế độ chẩn đoán Mode 03.
- Byte 2 - Byte 7 (0x00): Byte đệm.

Khung tin phản hồi:



Hình 4.30. Giải đồ xung phản hồi từ ECU động cơ Mode 03, 07



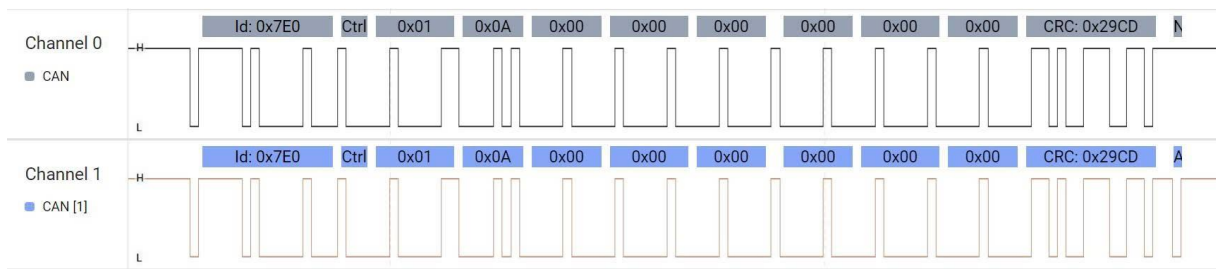
Hình 4.31. Giải đồ xung phản hồi từ ECU hộp số Mode 03, 07

- D: 0x7E8 (Phản hồi từ ECU động cơ)/ 0x7E9 (Phản hồi từ ECU hộp số)
- Data Field: 02 | 43 | 00 | 00 | 00 | 00 | 00 | 00
 - Byte 0 (0x02): Khai báo khung đơn chứa 2 byte dữ liệu hiệu dụng phía sau.
 - Byte 1 (0x43): Phản hồi tích cực cho Mode .
 - Byte 2 - Byte 3 (0x00 | 0x00): Cặp byte biểu thị mã lỗi (DTC). Giá trị 00 00 xác nhận hệ thống hoàn toàn không có mã lỗi lưu trữ nào.
 - Byte 4 - Byte 7 (0x00): Byte đệm.

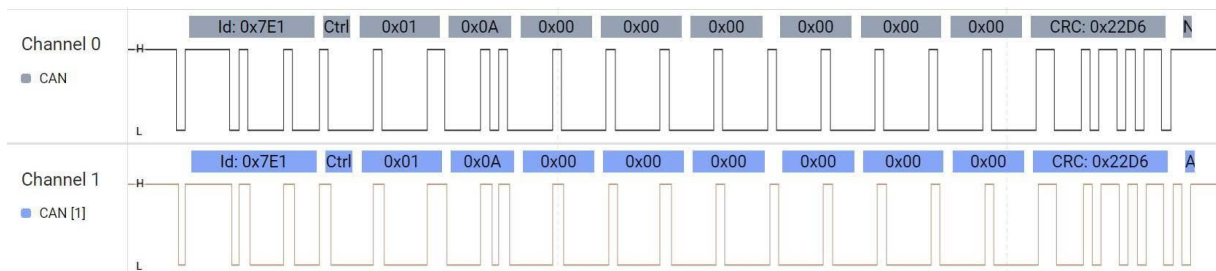
4.3.2.3 Kết quả kiểm thử Mode 0A

Chế độ Mode 0A được sử dụng để truy vấn các mã lỗi vĩnh lưu trữ trong bộ nhớ không khả biến của các hộp ECU. Quá trình thực nghiệm ghi nhận hai kịch bản phản hồi hoàn toàn khác nhau giữa hộp ECU hộp số và hộp ECU động cơ.

Khung tin yêu cầu:



Hình 4.32. Giải đồ xung của lệnh yêu cầu mã lỗi đến ECU động cơ Mode 0A

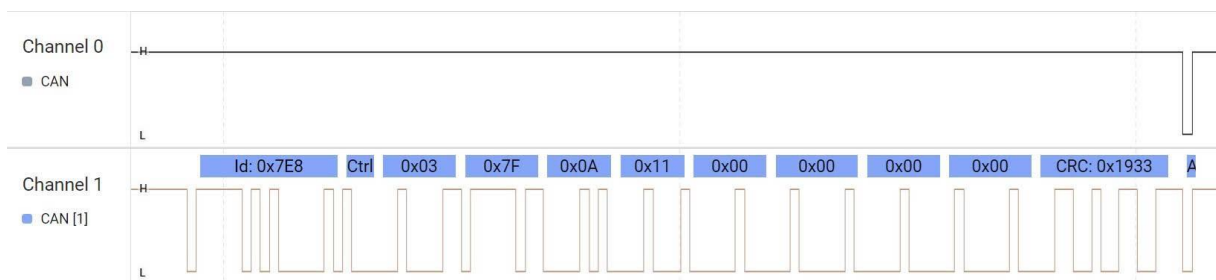


Hình 4.33. Giải đồ xung của lệnh yêu cầu mã lỗi đến ECU hộp số Mode 0A

- ID: 0x7E1 (ECU hộp số) / 0x7E0 (ECU động cơ)
- Data Field: 01 | 0A | 00 | 00 | 00 | 00 | 00 | 00
 - Byte 0 (0x01): Khai báo loại khung đơn 1 byte dữ liệu hiệu dụng phía sau.
 - Byte 1 (0x0A): Khai báo chế độ chẩn đoán Mode 0A.
 - Byte 2 - Byte 7 (0x00): Byte đệm.

Khung tin phản hồi:

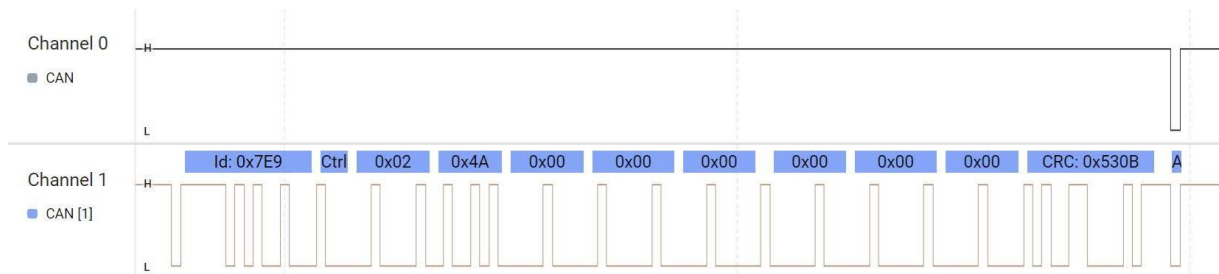
Quá trình tiếp nhận dữ liệu từ các hộp điều khiển ghi nhận hai trạng thái đối lập: Hộp số phản hồi tích cực và động cơ trả về phản hồi phủ định do không hỗ trợ dịch vụ.



Hình 4.34. Giải đồ xung phản hồi của ECU động cơ Mode 0A

- ID: 0x7E8
- Data Field: 03 | 7F | 0A | 11 | 00 | 00 | 00 | 00
 - Byte 0 (0x03): Khai báo độ dài dữ liệu hiệu dụng gồm 3 bytes kế tiếp
 - Byte 1 (0x7F): Ký hiệu nhận diện khung phản hồi phủ, thông báo yêu cầu dịch vụ chẩn đoán đã bị từ chối.

- Byte 2 (0x0A): Chế độ chẩn đoán Mode 0A bị từ chối.
- Byte 3 (0x11): Mã lỗi phủ định NRC. Theo tiêu chuẩn ISO 15765 và SAE J1979, giá trị 0x11 biểu thị lỗi serviceNotSupported (Dịch vụ không được hỗ trợ).



Hình 4.35. Giải đồ xung phản hồi của ECU hộp số Mode 0A

- ID: 0x7E9
- Data Field: 02 | 4A | 00 | 00 | 00 | 00 | 00 | 00
 - Byte 0 (0x02): Khai báo khung đơn chứa 2 byte dữ liệu hiệu dụng phía sau.
 - Byte 1 (0x4A): Phản hồi tích cực cho Mode 0A.
 - Byte 2 - Byte 3 (0x00 | 0x00): Cặp byte biểu thị mã lỗi (DTC). Giá trị 00 00 xác nhận hệ thống bộ nhớ không khả biến của hộp số không lưu trữ mã lỗi vĩnh viễn nào.
 - Byte 4 - Byte 7 (0x00): Byte đệm.

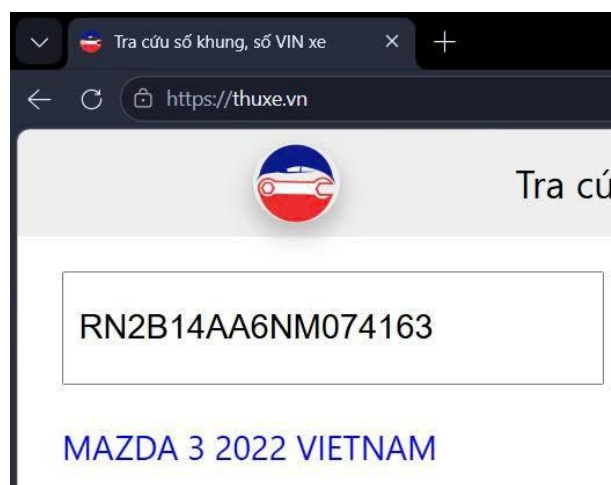
4.3.2.4 Kết quả kiểm thử Mode 09

Tiến hành bật khóa điện phương tiện. Do thông tin số VIN của xe có độ dài 17 ký tự, luồng dữ liệu phản hồi từ hộp ECU không thể nằm trọn trong một khung đơn. Do đó, quá trình truyền nhận bắt buộc phải thực hiện qua giải thuật giao tiếp đa khung tuân thủ quy chuẩn giao thức tầng mạng ISO 15765-2.

Kết quả thực nghiệm trực quan thu được từ hệ thống chẩn đoán và cơ sở dữ liệu đối chiếu trước khi tiến hành phân tích sâu luồng xung logic.



Hình 4.36. Ảnh chụp màn hình LCD của thiết bị chẩn đoán thu thập dữ liệu
Mode 09

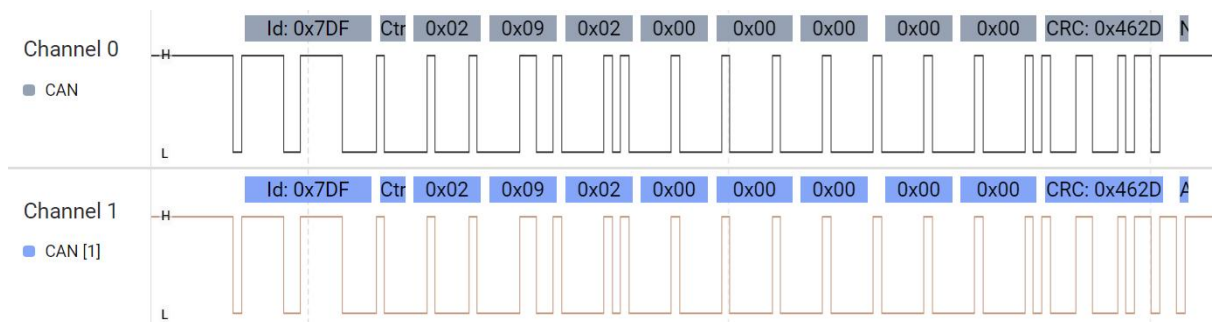


Hình 4.37. Kết quả đối chiếu thông tin mã VIN trên hệ thống tra cứu cơ sở dữ
liệu xe

Để giám sát và bóc tách luồng truyền, máy hiện sóng Logic Analyzer được đầu nối trực tiếp vào đường truyền vật lý, thực hiện bắt luồng xung tín hiệu đồng thời trên hai kênh giám sát độc lập:

- Kênh Channel 0: Thực hiện bắt luồng và giám sát xung tín hiệu của khung lệnh yêu cầu phát đi từ thiết bị chẩn đoán.
- Kênh Channel 1: Thực hiện bắt luồng và giám sát xung tín hiệu của khung tin phản hồi trả về từ hộp ECU xe.

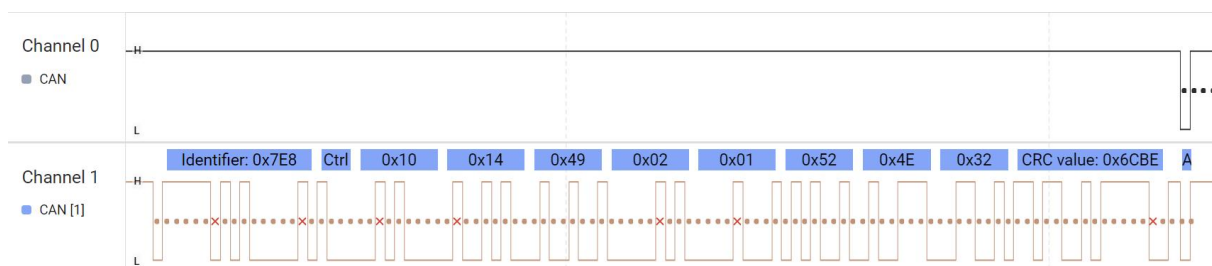
Khung tin yêu cầu:



Hình 4.38. Giải đồ xung của lệnh yêu cầu thông tin mã VIN

- ID: 0x7DF
- Data Field: 02 | 09 | 02 | 00 | 00 | 00 | 00 | 00
 - Byte 0 (0x02): Khai báo 2 byte dữ liệu hiệu dụng phía sau.
 - Byte 1 (0x09): Khai báo chế độ chẩn đoán Mode 09.
 - Byte 2 (0x02): Mã PID yêu cầu trích xuất thông tin số VIN của phương tiện.
 - Byte 3 – Byte 7 (0x00): Byte đệm

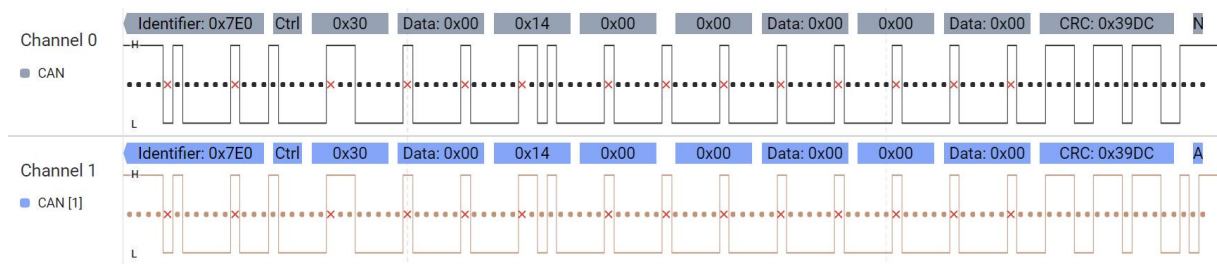
Khung tin phản hồi đầu tiên FF:



Hình 4.39. Giải đồ xung phản hồi khung đầu tiên FF từ ECU

- ID: 0x7E8
- Data Field: 10 | 14 | 49 | 02 | 01 | 52 | 4E | 32
 - Byte 0 (0x10) và Byte 1 (0x14): Khai báo loại Khung đầu tiên (First Frame) với tổng độ dài dữ liệu cần truyền của gói tin là 0x14 (tương đương 20 byte).
 - Byte 2 (0x49): Xác nhận phản hồi hợp lệ cho Mode 09.
 - Byte 3 (0x02): Xác nhận phản hồi mã PID số VIN.
 - Byte 4 (0x01): Khai báo số lượng chuỗi khối dữ liệu (Data block).
 - Byte 5 (0x52): Ký tự VIN thứ 1 dưới dạng mã ASCII (giải mã ký tự: R).
 - Byte 6 (0x4E): Ký tự VIN thứ 2 dưới dạng mã ASCII (giải mã ký tự: N).
 - Byte 7 (0x32): Ký tự VIN thứ 3 dưới dạng mã ASCII (giải mã ký tự: 2).

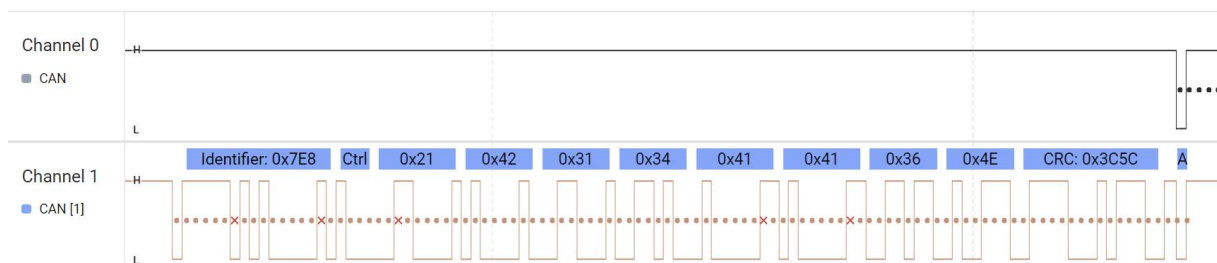
Khung điều khiển luồng FC:



Hình 4.40. Giải đồ xung khung điều khiển luồng FC từ thiết bị chẩn đoán gửi đi

- ID: 0x7E0
- Data Field: 30 | 00 | 14 | 00 | 00 | 00 | 00 | 00
 - Byte 0 (0x30): Định nghĩa loại Khung điều khiển lưu lượng (Flow Control) với trạng thái sẵn sàng tiếp nhận dữ liệu tiếp theo.
 - Byte 1 (0x00): Yêu cầu ECU truyền toàn bộ khối dữ liệu còn lại mà không cần chia nhỏ dải block (Block Size = 0).
 - Byte 2 (0x14): Khoảng thời gian nghỉ tối thiểu yêu cầu giữa các khung nối tiếp (Separation Time = 0x14 tương đương 20 ms).
 - Byte 3 – Byte 7 (0x00): Byte đệm.

Khung tin phản hồi nối tiếp CF1:

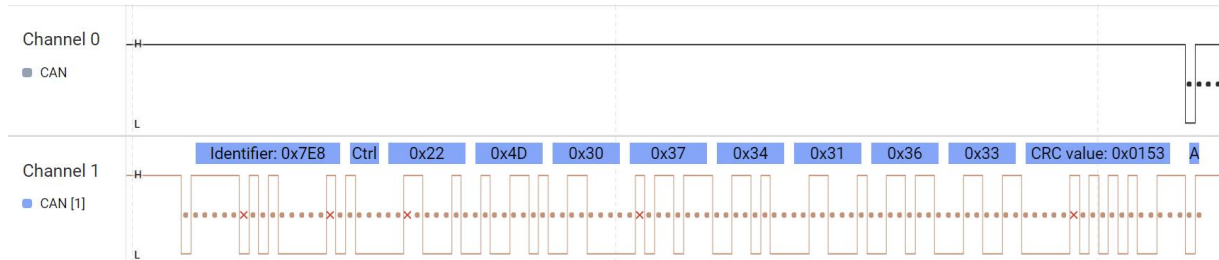


Hình 4.41. Giải đồ xung phản hồi nối tiếp CF1

- ID: 0x7E8
- Data Field: 21 | 42 | 31 | 34 | 41 | 41 | 36 | 4E
 - Byte 0 (0x21): Định nghĩa loại Khung nối tiếp (Consecutive Frame) với số chỉ mục tuần tự (Sequence Number) là 1.
 - Byte 1 (0x42): Ký tự VIN thứ 4 dưới dạng mã ASCII (giải mã ký tự: B).
 - Byte 2 (0x31): Ký tự VIN thứ 5 dưới dạng mã ASCII (giải mã ký tự: 1).
 - Byte 3 (0x34): Ký tự VIN thứ 6 dưới dạng mã ASCII (giải mã ký tự: 4).
 - Byte 4 (0x41): Ký tự VIN thứ 7 dưới dạng mã ASCII (giải mã ký tự: A).

- Byte 5 (0x41): Ký tự VIN thứ 8 dưới dạng mã ASCII (giải mã ký tự: A).
- Byte 6 (0x36): Ký tự VIN thứ 9 dưới dạng mã ASCII (giải mã ký tự: 6).
- Byte 7 (0x4E): Ký tự VIN thứ 10 dưới dạng mã ASCII (giải mã ký tự: N).

Khung phản hồi nối tiếp CF2:



Hình 4.42. Giải đồ xung phản hồi nối tiếp CF2

- ID: 0x7E8
- Data Field: 22 | 4D | 30 | 37 | 34 | 31 | 36 | 33
 - Byte 0 (0x22): Định nghĩa loại Khung nối tiếp (Consecutive Frame) với số chỉ mục tuần tự nhảy cấp lên 2.
 - Byte 1 (0x4D): Ký tự VIN thứ 11 dưới dạng mã ASCII (giải mã ký tự: M).
 - Byte 2 (0x30): Ký tự VIN thứ 12 dưới dạng mã ASCII (giải mã ký tự: 0).
 - Byte 3 (0x37): Ký tự VIN thứ 13 dưới dạng mã ASCII (giải mã ký tự: 7).
 - Byte 4 (0x34): Ký tự VIN thứ 14 dưới dạng mã ASCII (giải mã ký tự: 4).
 - Byte 5 (0x31): Ký tự VIN thứ 15 dưới dạng mã ASCII (giải mã ký tự: 1).
 - Byte 6 (0x36): Ký tự VIN thứ 16 dưới dạng mã ASCII (giải mã ký tự: 6).
 - Byte 7 (0x33): Ký tự VIN thứ 17 dưới dạng mã ASCII (giải mã ký tự: 3).

Sau khi tiếp nhận và tích lũy đầy đủ luồng dữ liệu đa khung, hệ thống tiến hành liên kết chuỗi mã Hex thô nhận được từ Byte 5 của khung FF cho tới Byte 7 của khung CF2 tạo thành chuỗi thông tin đồng nhất:

VIN (Hex) = 52 4E 32 42 31 34 41 41 36 4E 4D 30 37 34 31 36 33

Tiến hành tra cứu, đối chiếu giải thuật dịch chuỗi Hex thô dựa trên bảng mã ký tự ASCII quy chuẩn quốc tế:

Chuỗi ký tự số VIN hoàn chỉnh thu được: **RN2B14AA6NM074163**

Kết quả:

Trên màn hình hiển thị của thiết bị chẩn đoán: Hệ thống xử lý bóc ghép giao thức đa khung hoàn hảo, chuyển đổi ký tự chính xác và hiển thị chuỗi số VIN RN2B14AA6NM074163 rõ ràng, không gặp hiện tượng lỗi ký tự hay mất dòng.

Chuỗi số VIN do thiết bị đọc được trùng khớp hoàn toàn 100% với số VIN định danh thực tế được lưu trữ trong bộ nhớ hộp ECU trên phương tiện Mazda 3 đời 2022 và được xác thực thông tin giải mã thông qua website thuxe.vn. Qua đó, chứng minh giải thuật quản lý luồng truyền tải dữ liệu đa khung thuộc giao thức tầng mạng ISO 15765-2 trên thiết bị đạt độ ổn định và chính xác tuyệt đối.

4.4 Đánh giá và nhận xét hệ thống

Đánh giá tính tương thích giữa kết quả thực nghiệm và cơ sở lý thuyết

Toàn bộ dữ liệu xung logic và cách đóng gói khung tin bắt được trên mạng CAN vật lý từ máy hiện sóng Logic Analyser hoàn toàn trùng khớp với các cơ sở lý thuyết đã nghiên cứu ở chương 2 về tiêu chuẩn OBD-II, cụ thể là ISO 15765-4 và tầng mạng ISO CAN TP ISO 15765-2.

Mạch xử lý chính xác cả hai kịch bản: chạy dạng khung đơn đi đọc dữ liệu ngắn của Mode 03, 07, 0A và tự động chuyển đổi linh hoạt sang cấu trúc đa khung hình khi đọc dữ liệu mã lỗi DTC khi nhiều hơn 2 mã lỗi và giải mã dữ liệu dài như chuỗi ký tự mã VIN Mode 09. Việc phát hiện và bóc tách được mã lỗi tiêu cực NRC 0x11 ở một số hộp điều khiển ECU đời mới cũng chứng minh thuật toán bám sát thực tế đáp ứng của các hãng xe hiện nay.

Đánh giá độ chính xác của thuật toán giải mã dữ liệu

Thuật toán hoạt động cực kỳ chính xác nhờ khả năng sàng lọc và bóc tách các byte điều khiển giao thức, byte đệm thô để lấy chính xác dữ liệu chẩn đoán.

- Giải mã Mode 01: Thuật toán xử lý và tính toán chính xác các mã PID để xuất ra đúng giá trị thực tế của các thông số động học theo thời gian thực như: tốc độ xe, vòng tua máy, nhiệt độ nước làm mát, tải máy, vị trí bướm ga, nhiên liệu xe.
- Xử lý mã lỗi DTC: Quy trình dịch bit và tính toán logic từ các cặp byte dữ liệu thô để chuyển sang định dạng mã lỗi năm ký tự theo tiêu chuẩn quốc tế đạt độ chính xác tuyệt đối.
- Giải mã dữ liệu mã VIN: Bộ dịch mã ASCII vận hành tốt, chuyển đổi toàn vẹn chuỗi dữ liệu thô sang 17 ký tự mã VIN.

Độ ổn định và khả năng tương tích trên nhiều dòng xe

Để khẳng định độ tin cậy và độ ổn định của thiết bị, nhóm thực hiện đã tiến hành kiểm thử thực tế với nhiều phương tiện có hỗ trợ cổng OBD-II khác nhau thuộc các phân khúc và năm sản xuất khác nhau như: Mazda 3 2022, Mazda CX5 2019, Subaru Forester 2023 Thailand và Chevrolet Captiva.

Kết quả cho thấy thiết bị hoạt động rất ổn định, quá trình yêu cầu và đọc dữ liệu hoàn toàn chính xác, kết quả log phân tích của lệnh yêu cầu và phản hồi dữ liệu chính xác với cơ sở lý thuyết.

Đánh giá khả năng đồng bộ dữ liệu Cloud và giao diện ứng dụng

Luồng dữ liệu sau khi được vi điều khiển xử lý thành công được đẩy trực tiếp lên cơ sở dữ liệu đám mây Firebase theo thời gian thực. Hệ thống lưu trữ phản hồi đồng bộ tức thì, đảm bảo dữ liệu truyền từ xe qua mạch không bị thất thoát hay chậm trễ. Ứng dụng di động khi kết nối với Firebase luôn lấy về chính xác tuyệt đối các thông số động học, mã lỗi DTC và mã VIN của xe.

Giao diện ứng dụng được thiết kế rất trực quan, hiện đại. Các dữ liệu động học phức tạp được chuyển đổi để xem dễ dàng, thông số rõ ràng, giúp người dùng hoặc kỹ thuật viên dễ dàng theo dõi, quan sát và đánh giá nhanh tình trạng của xe mà không cần phải am hiểu sâu về các mã Hex thô của mạng CAN.

Ưu điểm và hạn chế của hệ thống

Ưu điểm:

- Thiết bị có kích thước nhỏ gọn, được đóng vỏ nhựa chắc chắn giúp bảo vệ tốt bo mạch bên trong, rất thuận tiện cho việc cầm tay và thao tác linh hoạt khi kiểm thử.
- Tốc độ phản hồi các dòng lệnh chẩn đoán rất nhanh theo đơn vị ms. Quá trình giao tiếp hoàn toàn an toàn, không gây hiện tượng trễ mạng hay xảy ra bất kỳ xung đột tiêu cực nào lên băng thông của mạng CAN Bus trên xe.

Hạn chế:

- Hạn chế việc thu thập log lỗi thực tế trên xe thật: Trong suốt quá trình thực nghiệm, tất cả các phương tiện thực tế đều ở trạng thái hoạt động ổn định, không có lỗi. Vì vậy, nhóm thực hiện không thể thu thập và trích xuất dữ liệu của các mã lỗi DTC từ mạng CAN bus trên xe thật. Tuy nhiên, để khắc phục

điều này và kiểm chứng tính đúng đắn của thuật toán, nhóm thực hiện đã chạy thử nghiệm thiết bị với hệ thống mô phỏng ECU. Dữ liệu log thô và mã lỗi DTC giả lập xuất ra hoàn toàn chính xác, đúng với lý thuyết.

- Phạm vi hệ thống: Hiện tại hệ thống chỉ tập trung xử lý tốt các Mode cơ bản, chưa mở rộng thêm các tính năng nâng cao khác như xóa mã lỗi hệ thống hay cài đặt lại hộp điều khiển,...

CHƯƠNG 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Đề tài “Thiết kế hệ thống giám sát, thu thập dữ liệu và chẩn đoán lỗi xe ô tô qua cổng OBD-II” đã hoàn thành toàn bộ mục tiêu đề ra. Thông qua quy trình kiểm thử hai giai đoạn, từ bộ ECU mô phỏng đến thực nghiệm trên xe thực tế. Hệ thống chứng minh được khả năng vận hành ổn định và thu thập dữ liệu chính xác từ mạng CAN bus. Đồng thời, ứng dụng di động đã kết nối đồng bộ với Firebase, đảm bảo thông tin được cập nhật và hiển thị trực quan theo thời gian thực.

5.1.1 Kết quả đạt được

Hệ thống đã đọc thành công 5 Mode chức năng từ cổng OBD-II của xe, bao gồm: Mode 01, 03, 07, 09 và 0A.

Bộ ECU mô phỏng đóng vai trò là môi trường kiểm thử và phát triển, giúp kiểm soát và xác thực tính chính xác của các gói tin gửi và nhận.

Hệ thống đã được kiểm thử thực tế trên nhiều dòng xe khác nhau như Subaru Forester 2023, Mazda CX-5 2019, Mazda 3 2022, Chevrolet Captiva. Kết quả thực nghiệm đồng nhất với mô phỏng khi toàn bộ dữ liệu đều được truy xuất thành công trên toàn bộ các Mode.

Bên cạnh việc kiểm tra dữ liệu hiển thị trên màn hình TFT và Firebase, nhóm thực hiện đã trích xuất dữ liệu log truyền nhận qua hai chân Tx và Rx. Kết quả được đối chiếu với tiêu chuẩn ISO 15031-5 và SAE J1979 và cho thấy các gói tin hoàn toàn trùng khớp và chính xác theo lý thuyết.

Ứng dụng di động Android đã được xây dựng đúng thiết kế với các chức năng cốt lõi bao gồm: Trực quan hoá dữ liệu truyền nhận từ các Mode, có thể yêu cầu đọc từ xa thay vì sử dụng nút nhấn trên phần cứng và có cảnh báo thời gian thực khi các thông số ở Mode 01 vượt ngưỡng cấu hình.

5.1.2 Hạn chế

Hệ thống phần cứng dừng lại ở việc hoạt động đúng như yêu cầu, chưa được kiểm thử ở điều kiện khắc nghiệt hơn như nhiệt độ cao cũng như tính bảo mật của dữ liệu.

Dữ liệu trên màn hình vật lý và ứng dụng di động chưa đúng theo thời gian thực, còn tồn tại độ trễ so với cụm đồng hồ Taplo trên xe.

5.2 Hướng phát triển

Hệ thống hiện tại đã vận hành ổn định, đáp ứng đầy đủ các yêu cầu chức năng nền tảng. Trong giai đoạn tiếp theo, hệ thống có tiềm năng mở rộng và phát triển theo các định hướng sau:

- Chuyển đổi cơ chế giao tiếp dữ liệu trên thiết bị di động: Nâng cấp ứng dụng Android từ mô hình hiển thị thụ động sang cơ chế tương tác chủ động, cho phép thiết bị gửi các yêu cầu truy xuất dữ liệu trực tiếp theo thời gian thực.
- Triển khai cơ chế phân quyền tài khoản người dùng nhằm đảm bảo tính bảo mật, toàn vẹn dữ liệu và quản lý hiệu quả quyền truy cập hệ thống.
- Có thể phát triển mô hình AI hỗ trợ phân tích mã lỗi hệ thống. Từ các dữ liệu lỗi thu thập được, AI sẽ tự động phân loại và đề xuất giải pháp xử lý tối ưu, nâng cao hiệu quả bảo trì.

TÀI LIỆU THAM KHẢO

- [1] J. R. V. a. R. I. A. Rodriguez, “Implementation of an OBD-II Diagnostics Tool over CAN-BUS with Arduino,” *Sistemas & Telemática*, tập 16, số 45, pp. 31-43, 2018.
- [2] O. M. Rybitskyi, V. V. Golian, N. V. Golian, Z. V. Dudar, O. V. Kalynychenko và D. M. Nikitin, “Using OBD-2 technology for vehicle diagnostic and using it in the information system,” *Вісник Національного технічного університету «ХПІ». Серія: Системний аналіз, управління та інформаційні технології*, số 1 (9), pp. 97-103, 2023.
- [3] A. V. Moshynska và O. G. Khrokalo, “Remote vehicle diagnostic system development based on the internet of things technology,” *Information and Telecommunication Sciences*, tập 15, số 1, pp. 28-32, 2024.
- [4] Robert Bosch GmbH, “CAN Specification Version 2.0,” Robert Bosch GmbH, Stuttgart, Germany, 1991.
- [5] International Organization for Standardization, “Road vehicles — Controller area network (CAN) — Part 2: High-speed medium access unit, ISO 11898-2:2016,” ISO, 2016.
- [6] International Organization for Standardization, “Road vehicles — Diagnostic communication over Controller Area Network (DoCAN) — Part 2: Transport protocol and network layer services, ISO 15765-2:2016,” ISO, 2016.
- [7] SAE International, “E/E Diagnostic Test Modes (SAE J1979),” SAE International, 2014.
- [8] International Organization for Standardization, “Road vehicles — Communication between vehicle and external equipment for emissions-related diagnostics — Part 5: Emissions-related diagnostic services, ISO 15031-5:2015,” ISO, 2015.
- [9] SAE International, “Diagnostic Connector (SAE J1962),” AE International, 2002.
- [10] International Organization for Standardization, “Road vehicles — Communication between vehicle and external equipment for emissions-

related diagnostics — Part 3: Diagnostic connector and related electrical circuits, specification and use, ISO 15031-3,” ISO, 2004.

- [11] International Organization for Standardization, “oad vehicles — Diagnostic communication over Controller Area Network (DoCAN) — Part 4: Requirements for emissions-related systems (ISO 15765-4:2005),” ISO, 2005.
- [12] Motorola Inc., “SPI Block Guide V03.06,” Motorola Inc., 2004.